

High-throughput and Flexible Host Networking for Accelerated Computing

Athinagoras Skiadopoulos¹, Zhiqiang Xie¹, Mark Zhao¹, Qizhe Cai², Saksham Agarwal²,
Jacob Adelman³, David Ahern³, Carlo Contavalli³, Michael Goldflam³, Vitaly Mayatskikh³,
Raghu Raja³, Daniel Walton³, Rachit Agarwal², Shrijeet Mukherjee³, Christos Kozyrakis¹



End-host network stack tradeoff

Performance

Terabit throughput
requirements

vs

Flexibility

improve protocol for
topology/application needs

ZeroNIC achieves both

Design intuition

split **control**/**data** paths, logically couple them

Implementation

FPGA NIC prototype – kernel TCP – GPU zero-copy

Evaluation

RDMA performance + TCP robustness

**AI datacenters are growing:
10,000s of accelerator nodes**



AI datacenters are growing: 10,000s of accelerator nodes

Performance

- End-host requirements: terabit throughput at low cost
- AI nodes today: 3.2Tbps (400Gbps per GPU)

Flexibility

- Enable transport protocol innovation
- Optimize transport and topology for large scale and app patterns

An Ideal Host Network Stack

Performance

high throughput: direct data path for payloads between NIC and application buffers (zero-copy)

Flexibility

process headers using any transport protocol (existing or new), in any execution environment (e.g., in user space, kernel, or accelerator)

An Ideal Host Network Stack

	All in HW (e.g., IB/RoCE)
Performance	✓
Flexibility	✗
	performant data path - fixed protocol

An Ideal Host Network Stack

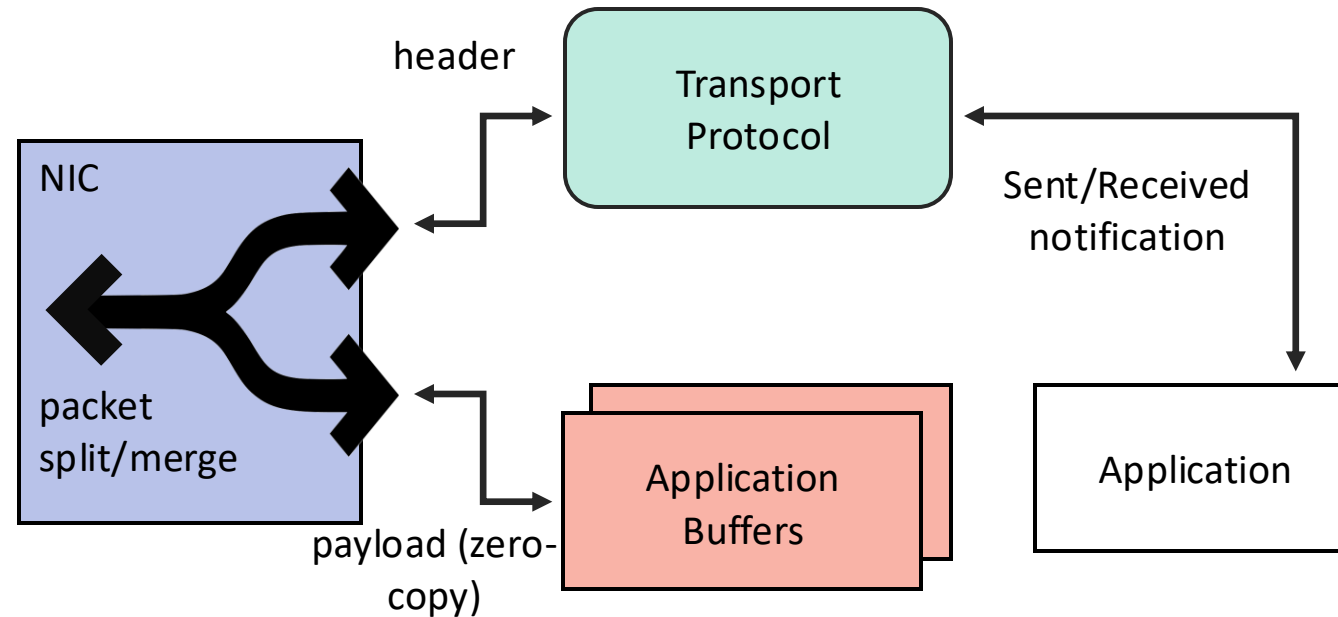
	All in HW (e.g., IB/RoCE)	All in SW (e.g., Linux TCP/IP)
Performance	✓	✗
Flexibility	✗	✓
	performant data path - fixed protocol	flexible control path - slow data path

An Ideal Host Network Stack

	All in HW (e.g., IB/RoCE)	All in SW (e.g., Linux TCP/IP)	ZeroNIC
Performance	✓	✗	✓
Flexibility	✗	✓	✓
	performant data path - fixed protocol	flexible control path - slow data path	zero-copy data path flexible control path

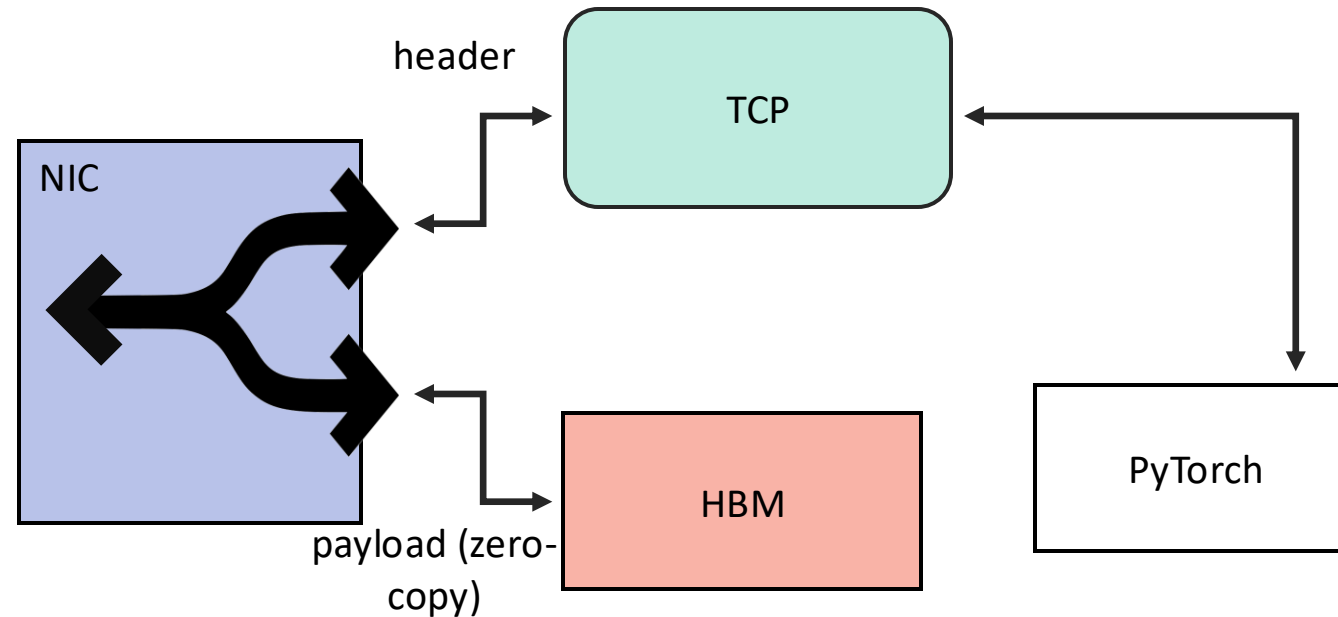
Performant and Flexible **Host Networking**

Performant and Flexible Host Networking



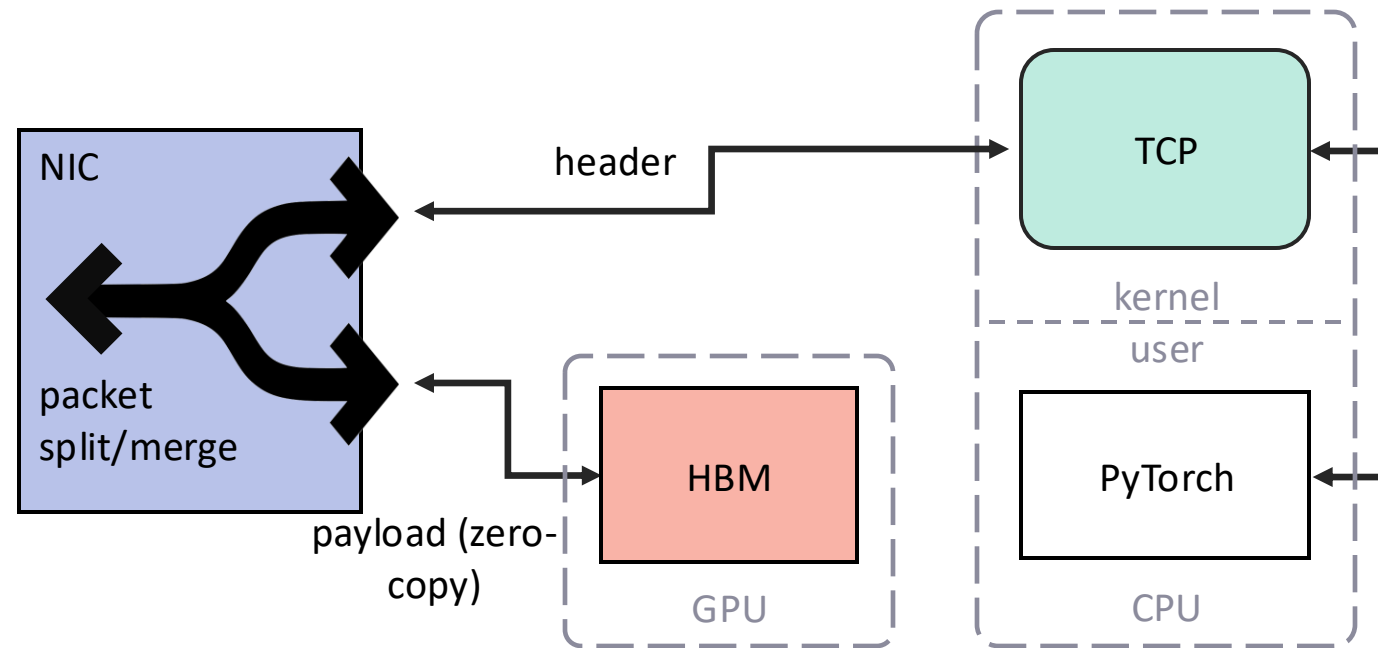
Physically separate the control and data paths
data: **zero-copy** control: **anywhere/anyhow**

Performant and Flexible Host Networking



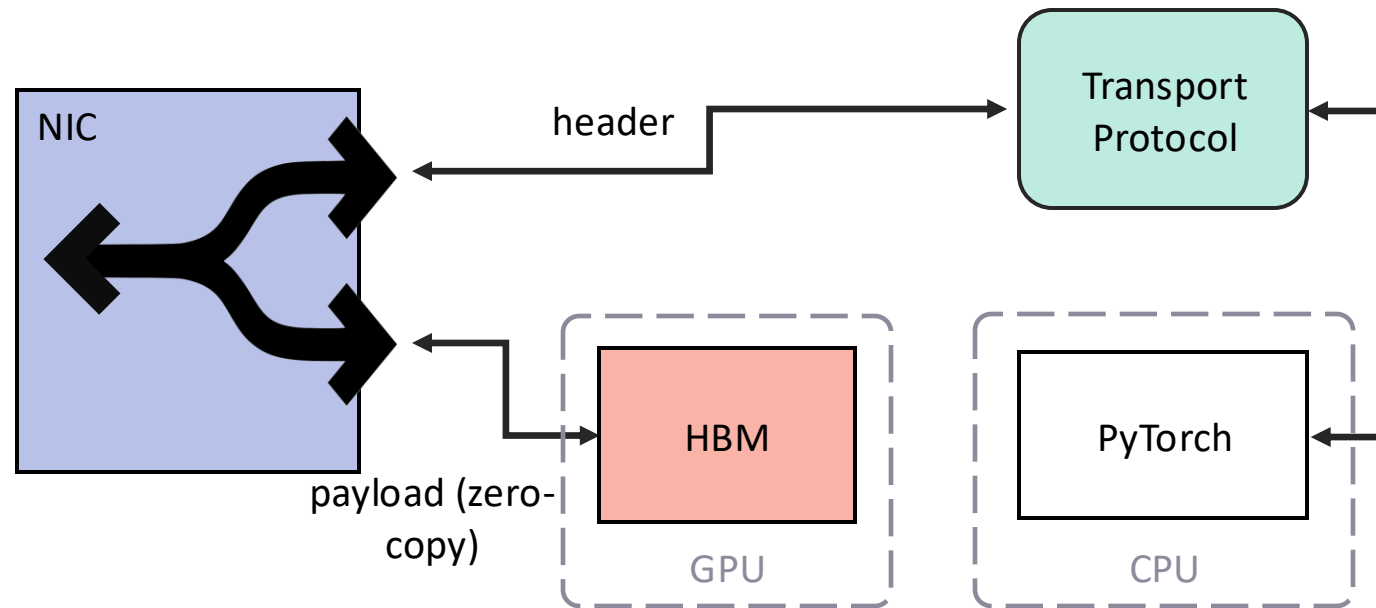
Physically separate the control and data paths
data: **zero-copy** control: **anywhere/anyhow**

Performant and Flexible Host Networking



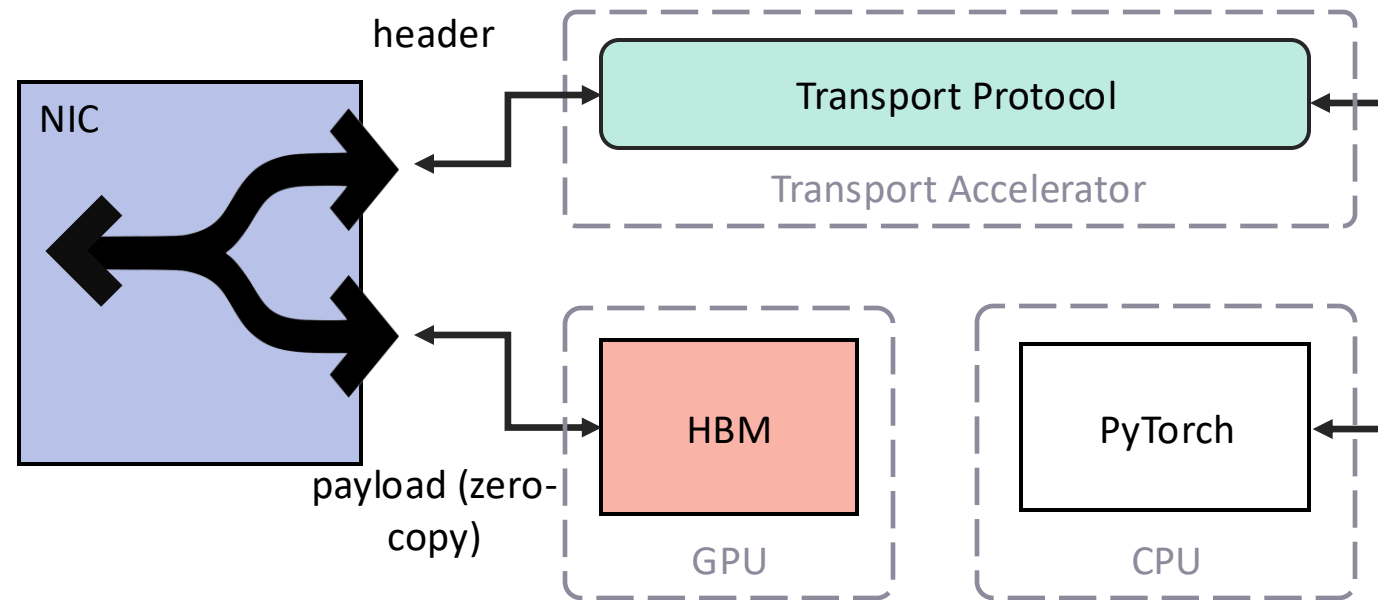
Physically separate the control and data paths
data: **zero-copy** control: **anywhere/anyhow**

Performant and Flexible Host Networking



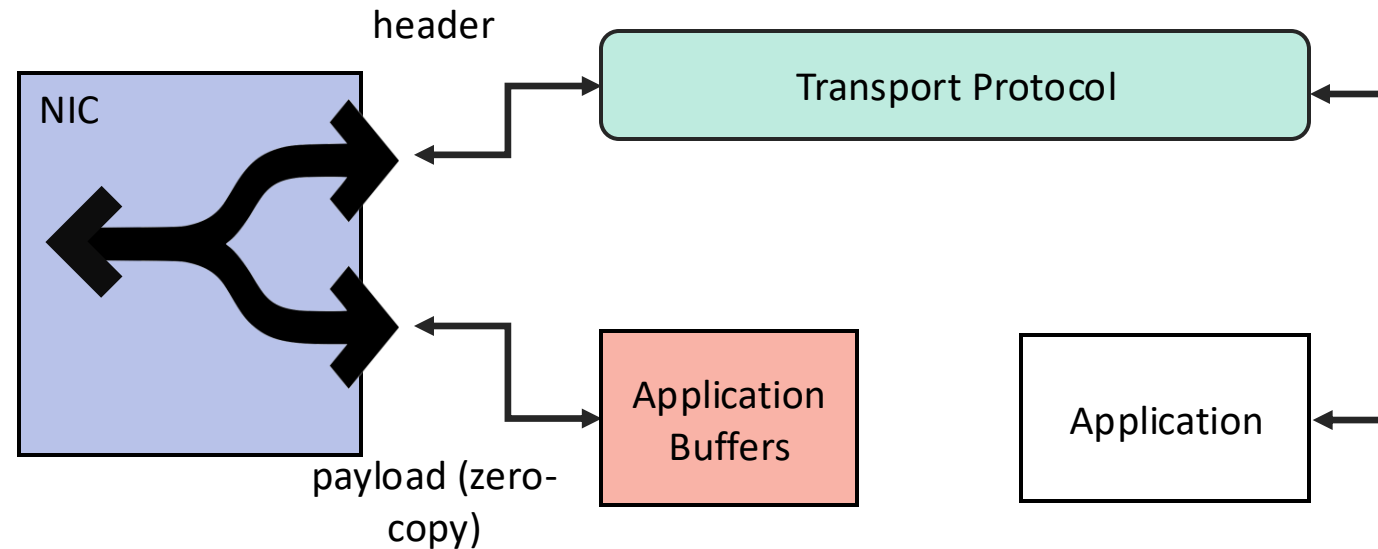
Physically separate the control and data paths
data: **zero-copy** control: **anywhere/anyhow**

Performant and Flexible Host Networking



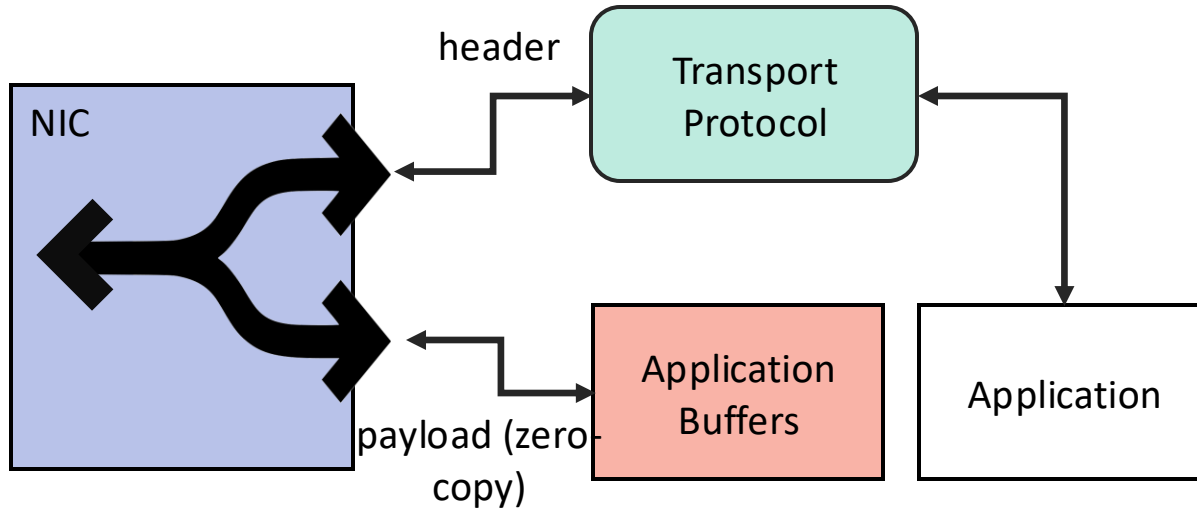
Physically separate the control and data paths
data: **zero-copy** control: **anywhere/anyhow**

Performant and Flexible Host Networking



Physically separate the control and data paths
data: **zero-copy** control: **anywhere/anyhow**

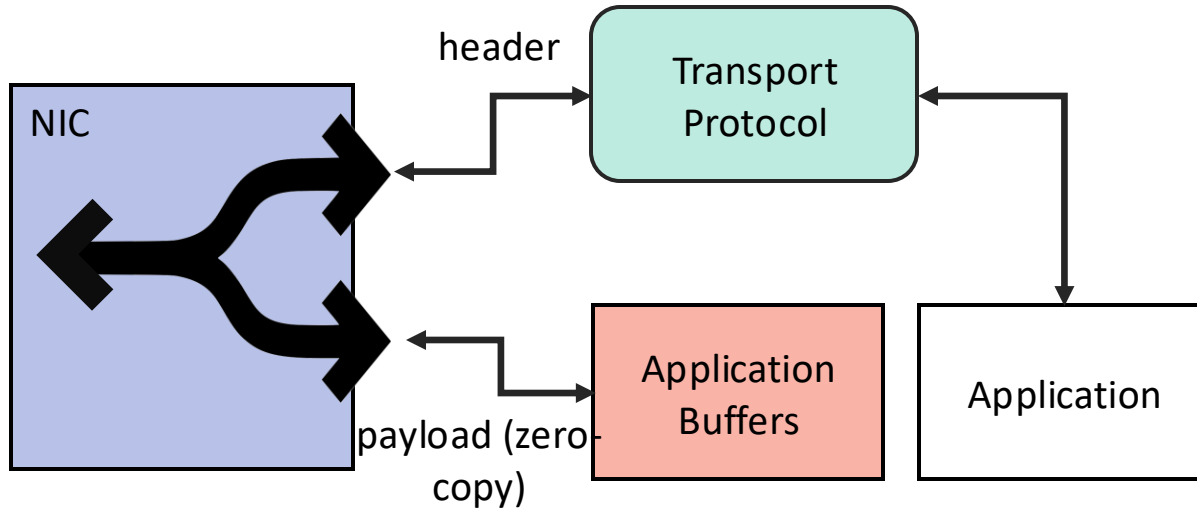
Performant and Flexible Host Networking



Prior work on zero-copy

- only send-side
- only unreliable (e.g., UDP)
- protocol offload to NIC
- change the API
- only for page-aligned payloads

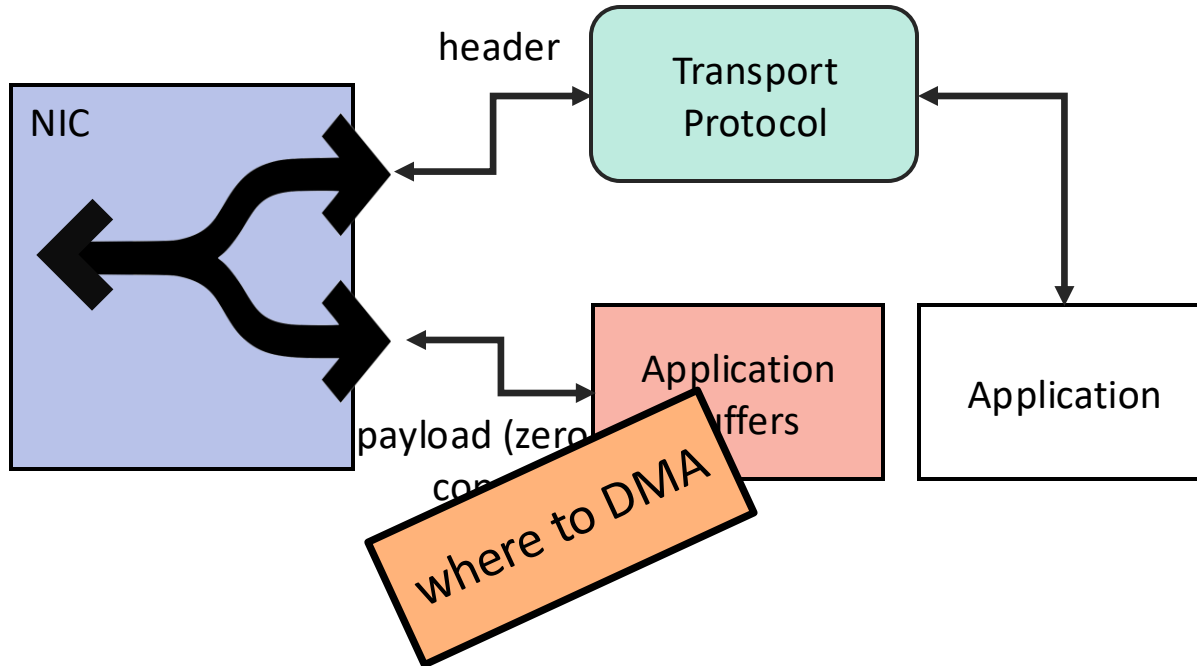
Performant and Flexible Host Networking



Challenge: reliable receiver

- user posts buffers (in-order)
- packet arrival $\neq$ in-order (drops, reorderings, ...)
- payload sizes $\neq$ buffer lengths
- protocol executes after DMA

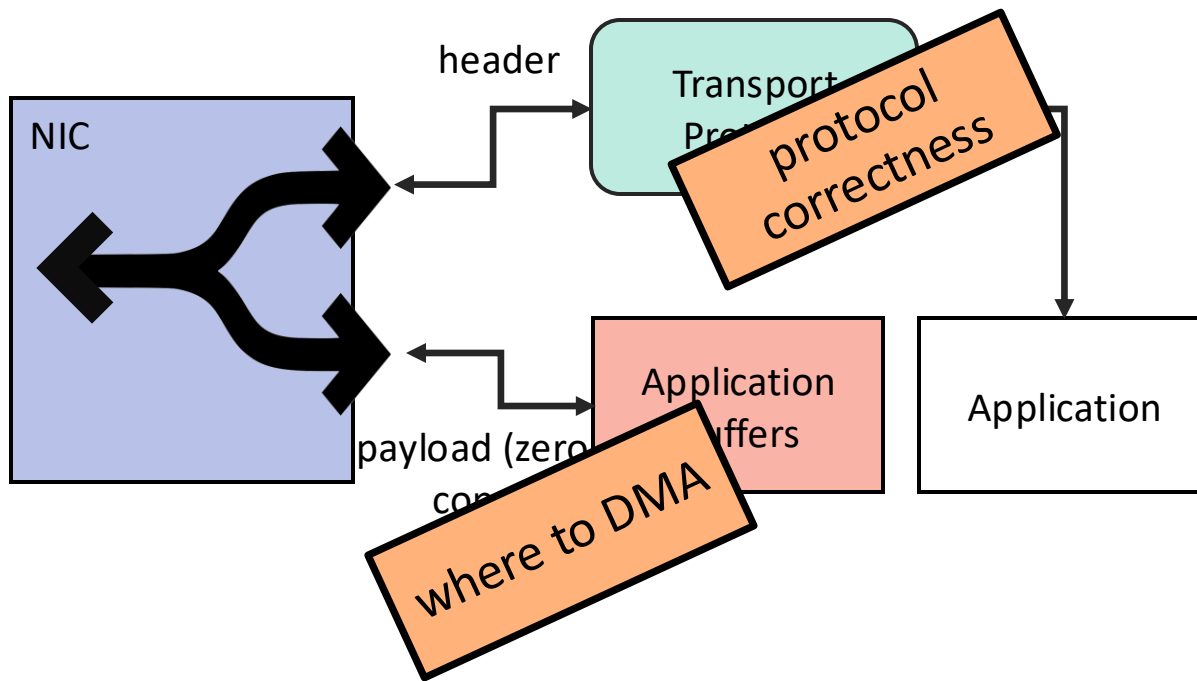
Performant and Flexible Host Networking



Challenge: reliable receiver

- user posts buffers (in-order)
- packet arrival $\neq$ in-order (drops, reorderings, ...)
- payload sizes $\neq$ buffer lengths
- protocol executes after DMA

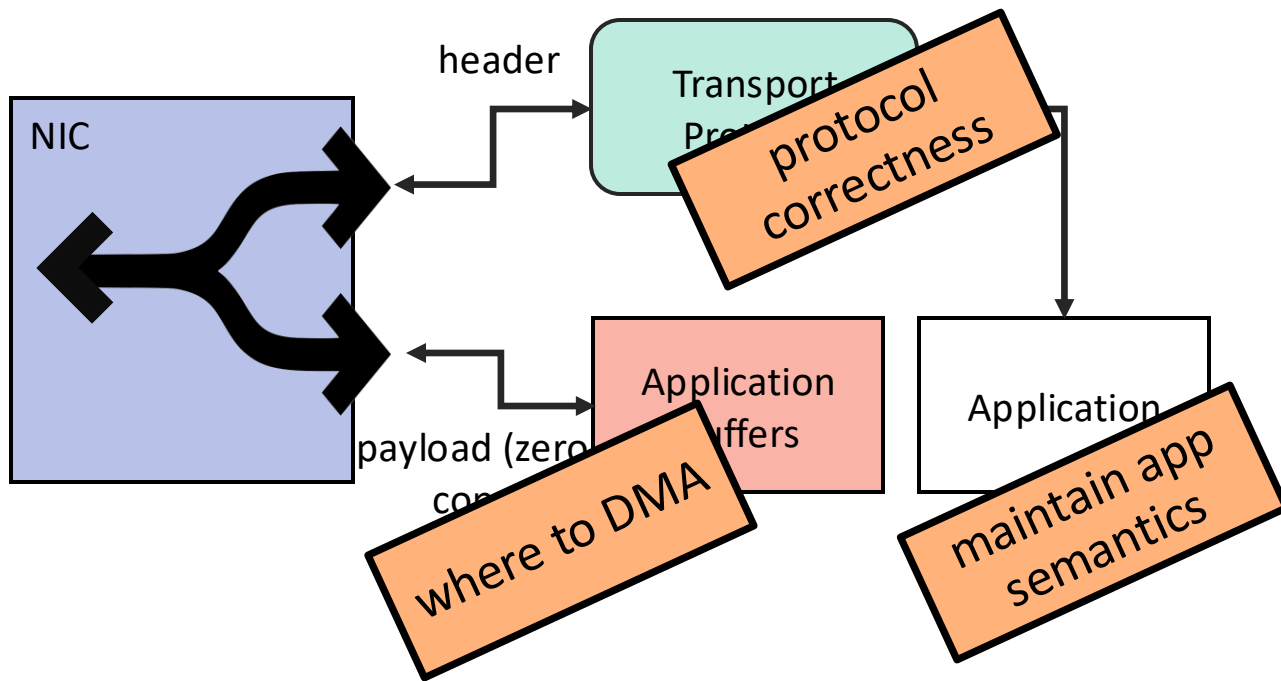
Performant and Flexible Host Networking



Challenge: reliable receiver

- user posts buffers (in-order)
- packet arrival $\neq$ in-order (drops, reorderings, ...)
- payload sizes $\neq$ buffer lengths
- protocol executes after DMA

Performant and Flexible Host Networking

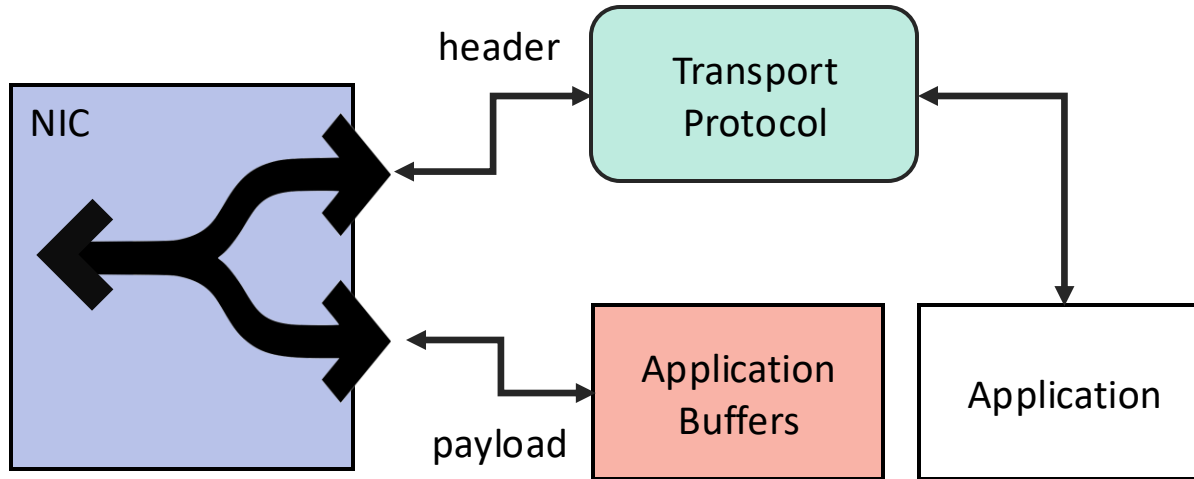


Challenge: reliable receiver

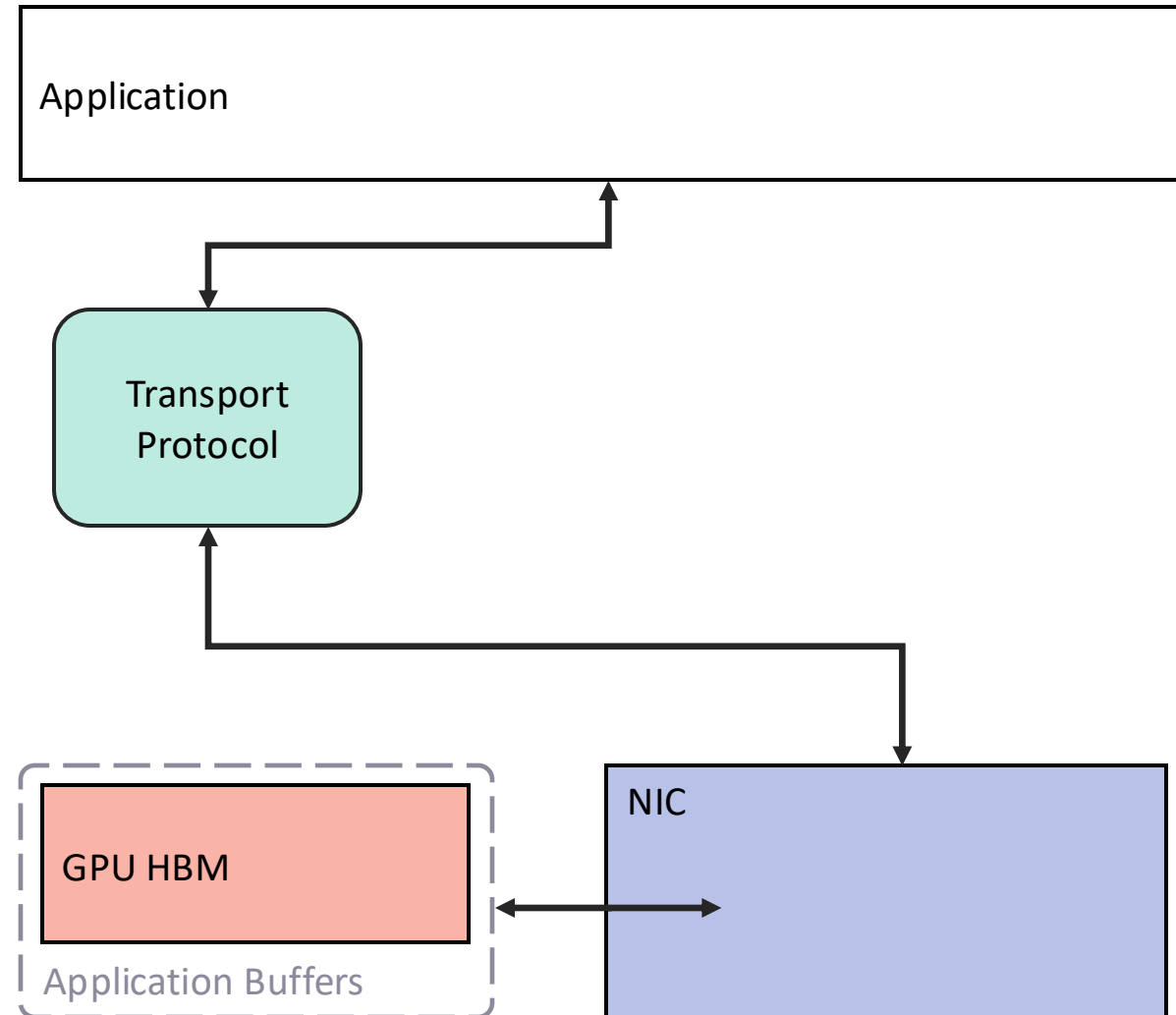
- user posts buffers (in-order)
- packet arrival $\neq$ in-order (drops, reorderings, ...)
- payload sizes $\neq$ buffer lengths
- protocol executes after DMA

ZeroNIC Design

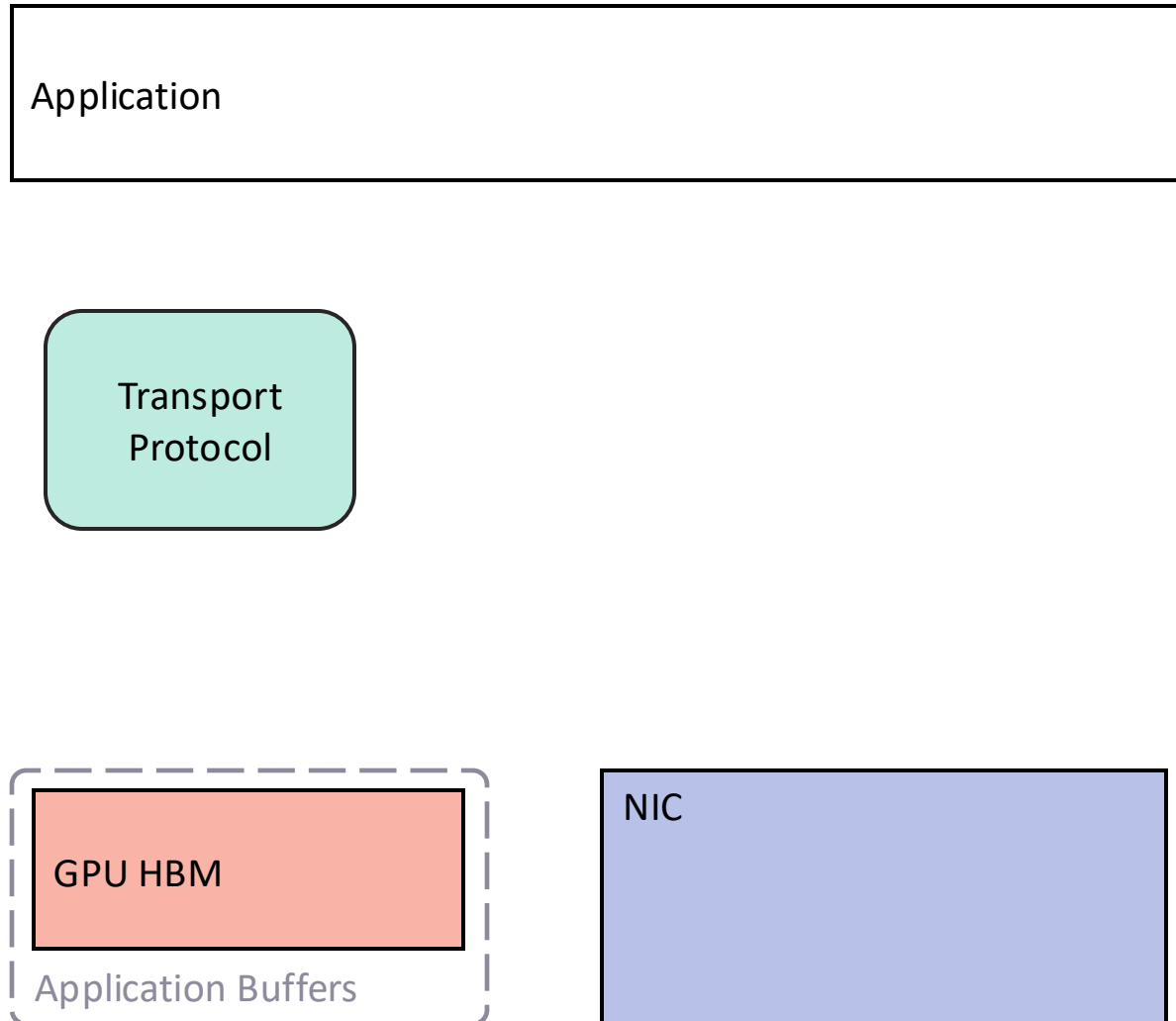
Design for Performance and Flexibility



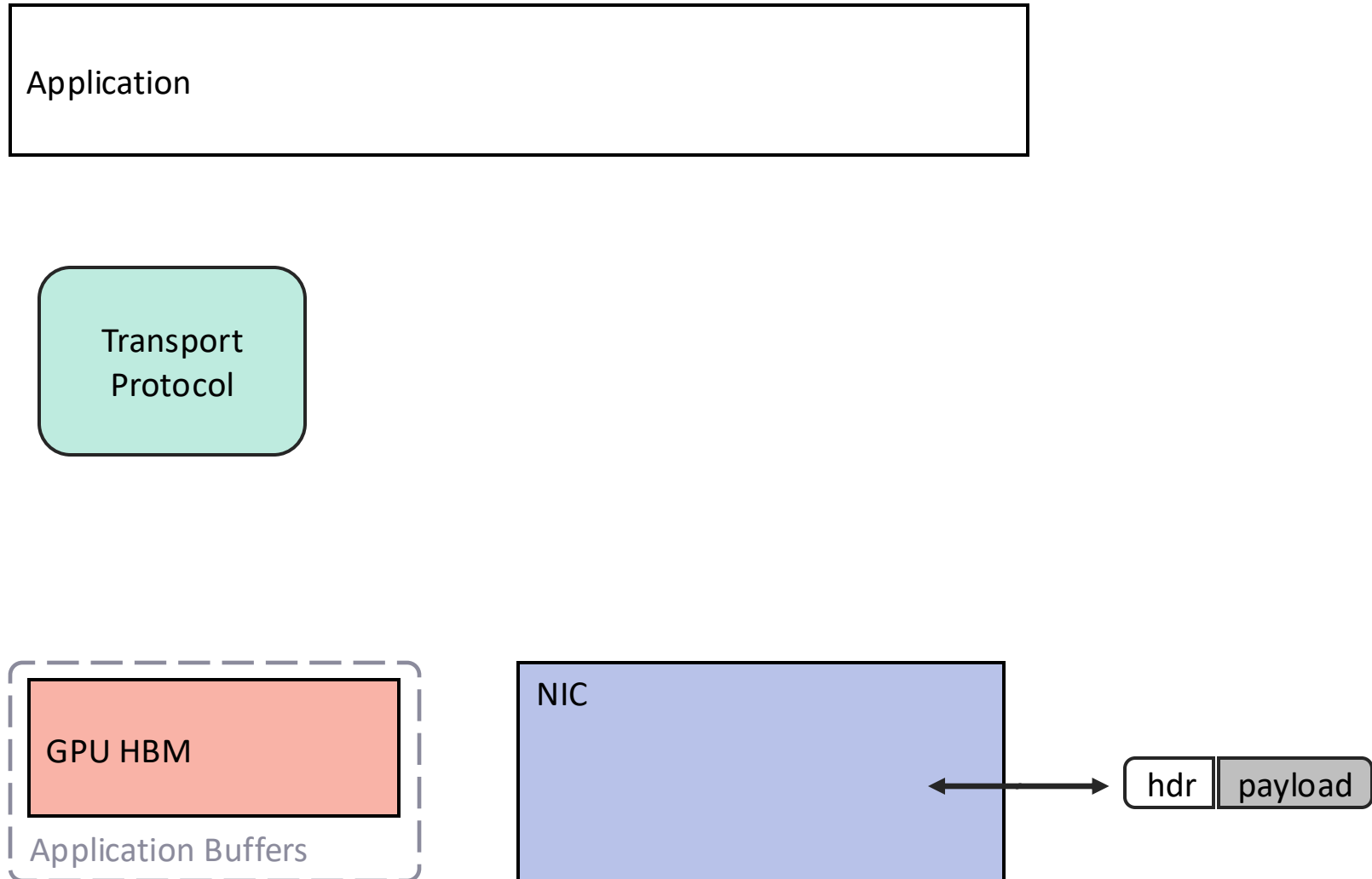
Design for Performance and Flexibility



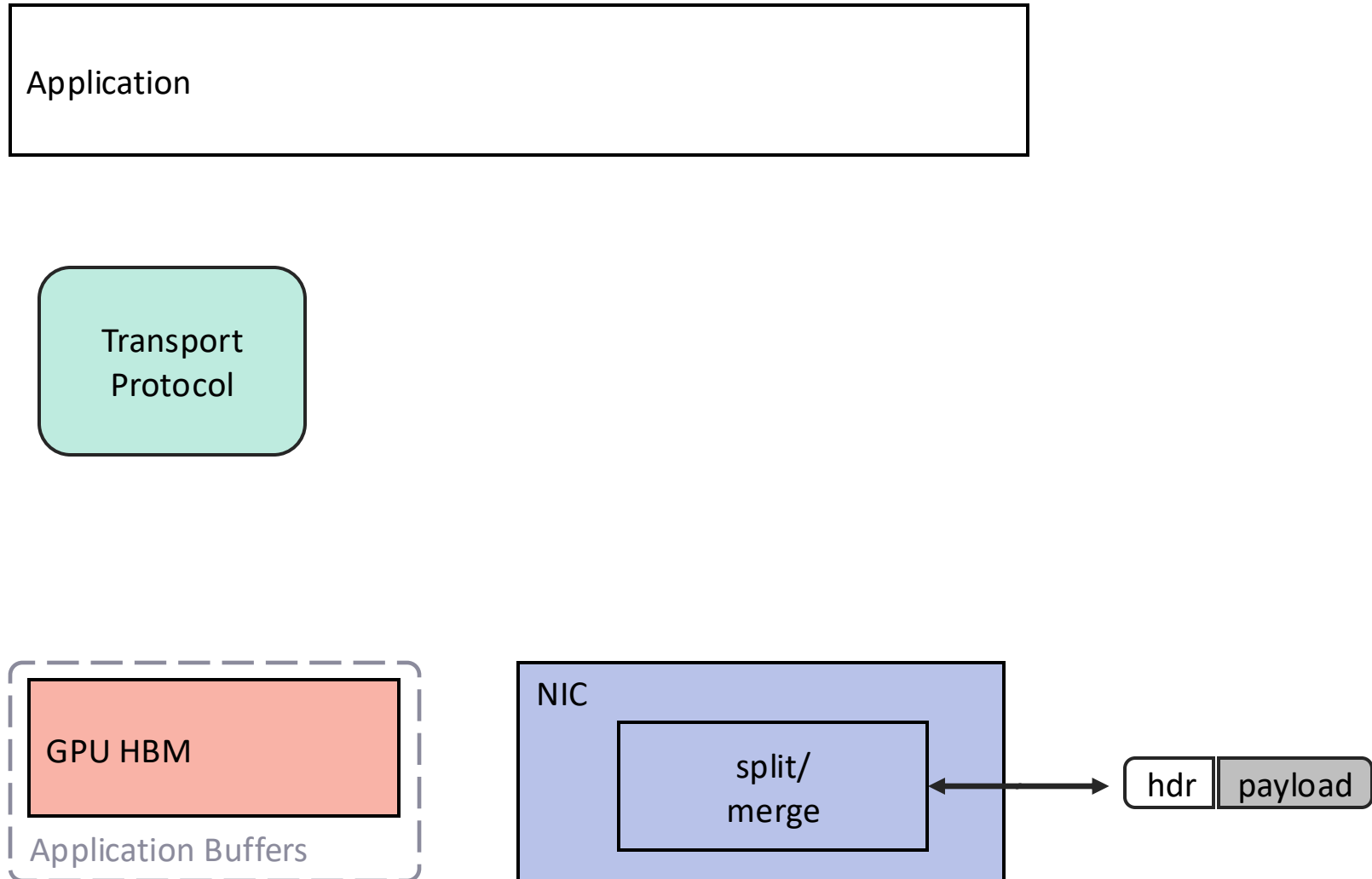
Design for Performance and Flexibility



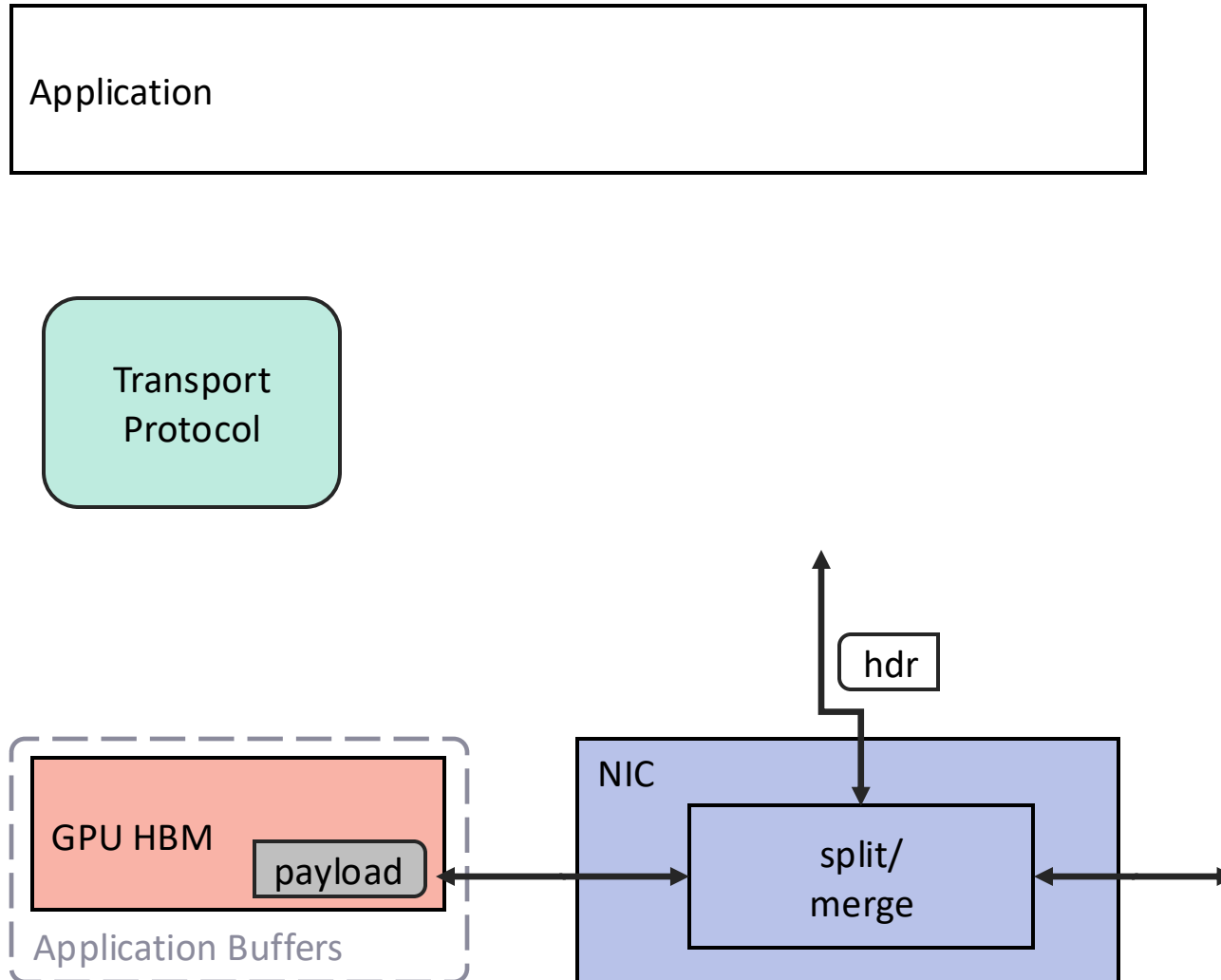
Design for Performance and Flexibility



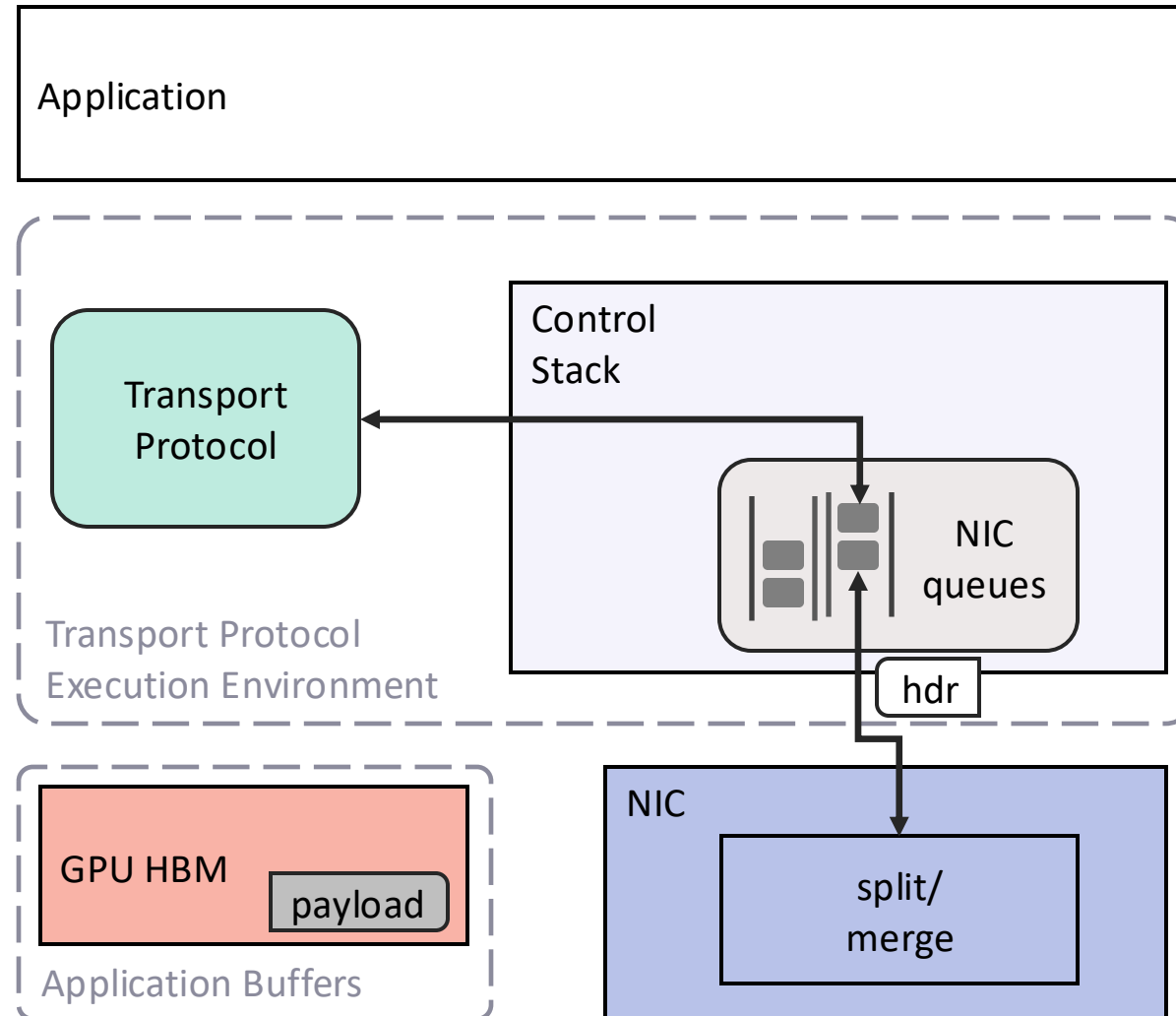
Design for Performance and Flexibility



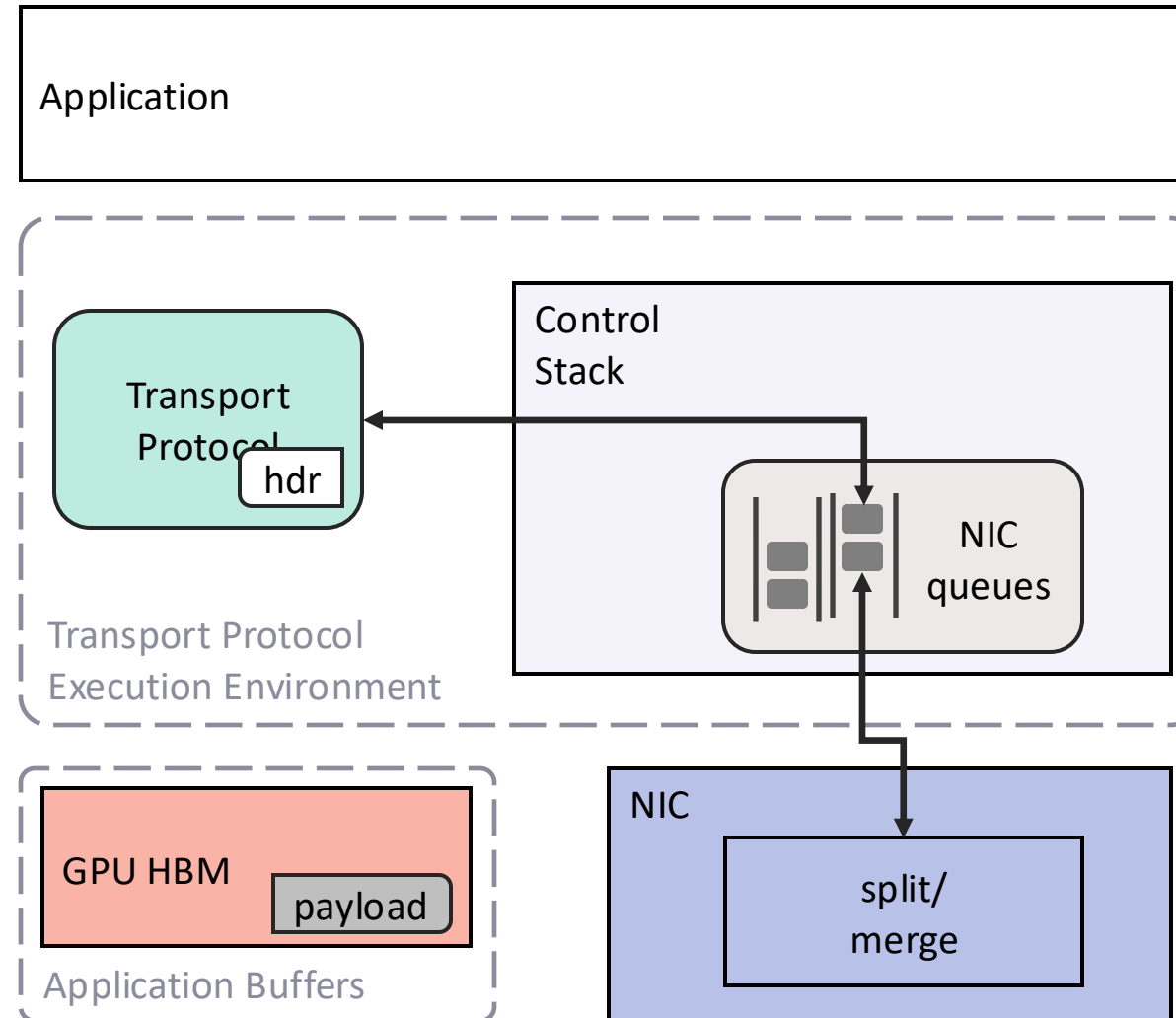
Design for Performance and Flexibility



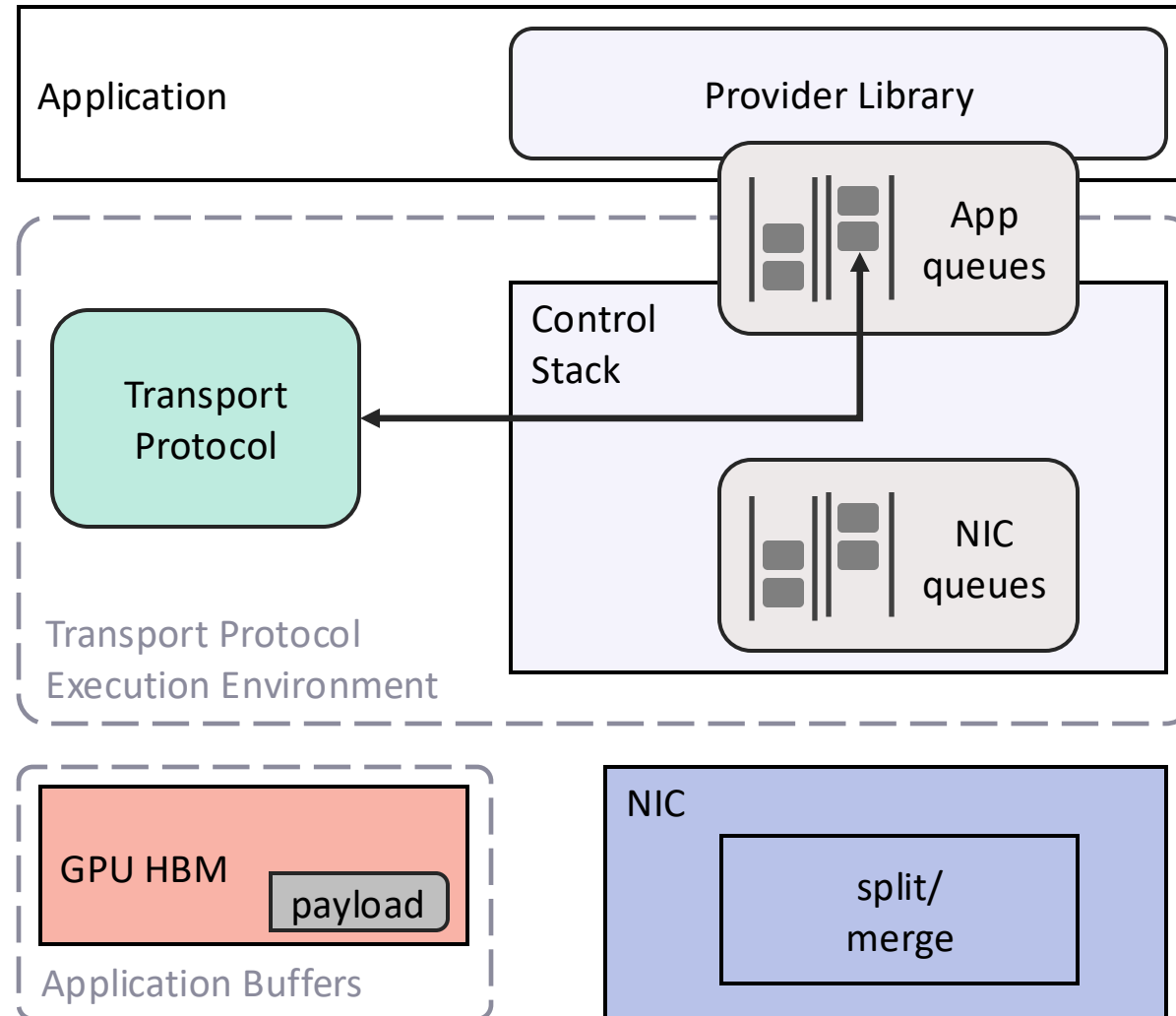
Design for Performance and Flexibility



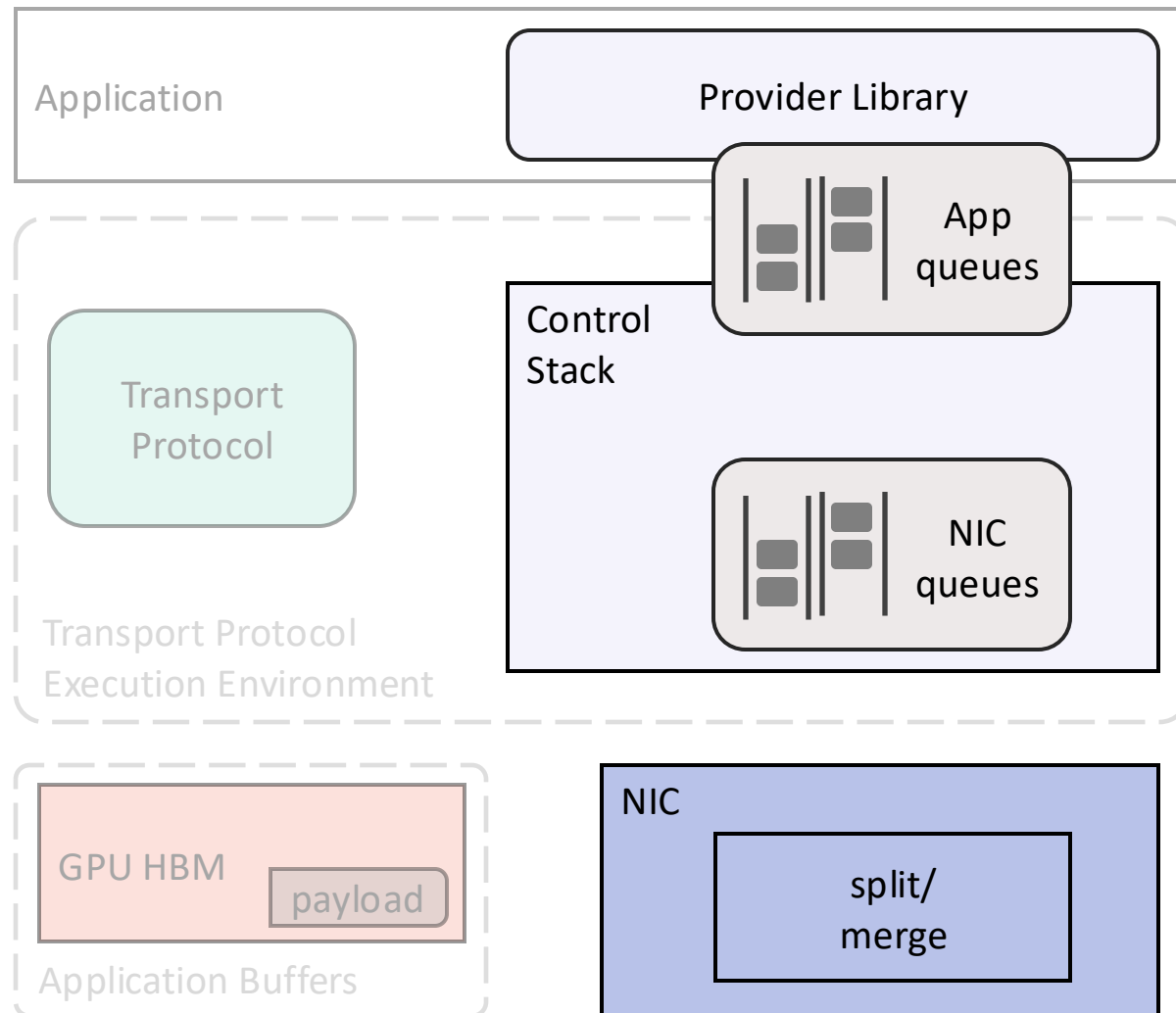
Design for Performance and Flexibility



Design for Performance and Flexibility



Design for Performance and Flexibility



implement popular APIs for transparent application integration

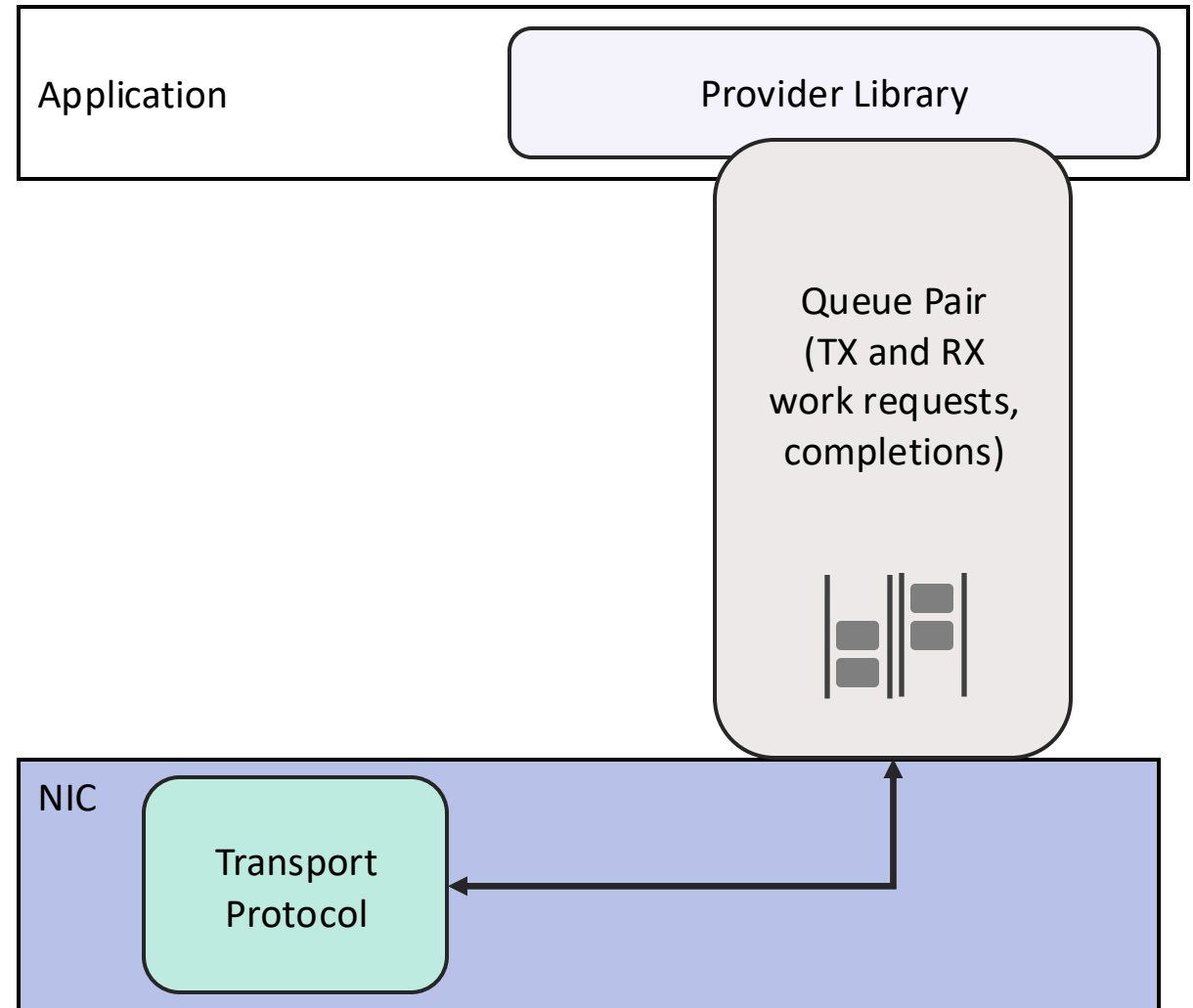
correctly interpose **any protocol** (e.g., reliable in-order) executing **anywhere** (e.g., kernel)

zero-copy data path to correct memory address on any endpoint (e.g., GPUs)

SW design: proxy protocol decisions

Prior RDMA stacks (IB/RoCE)

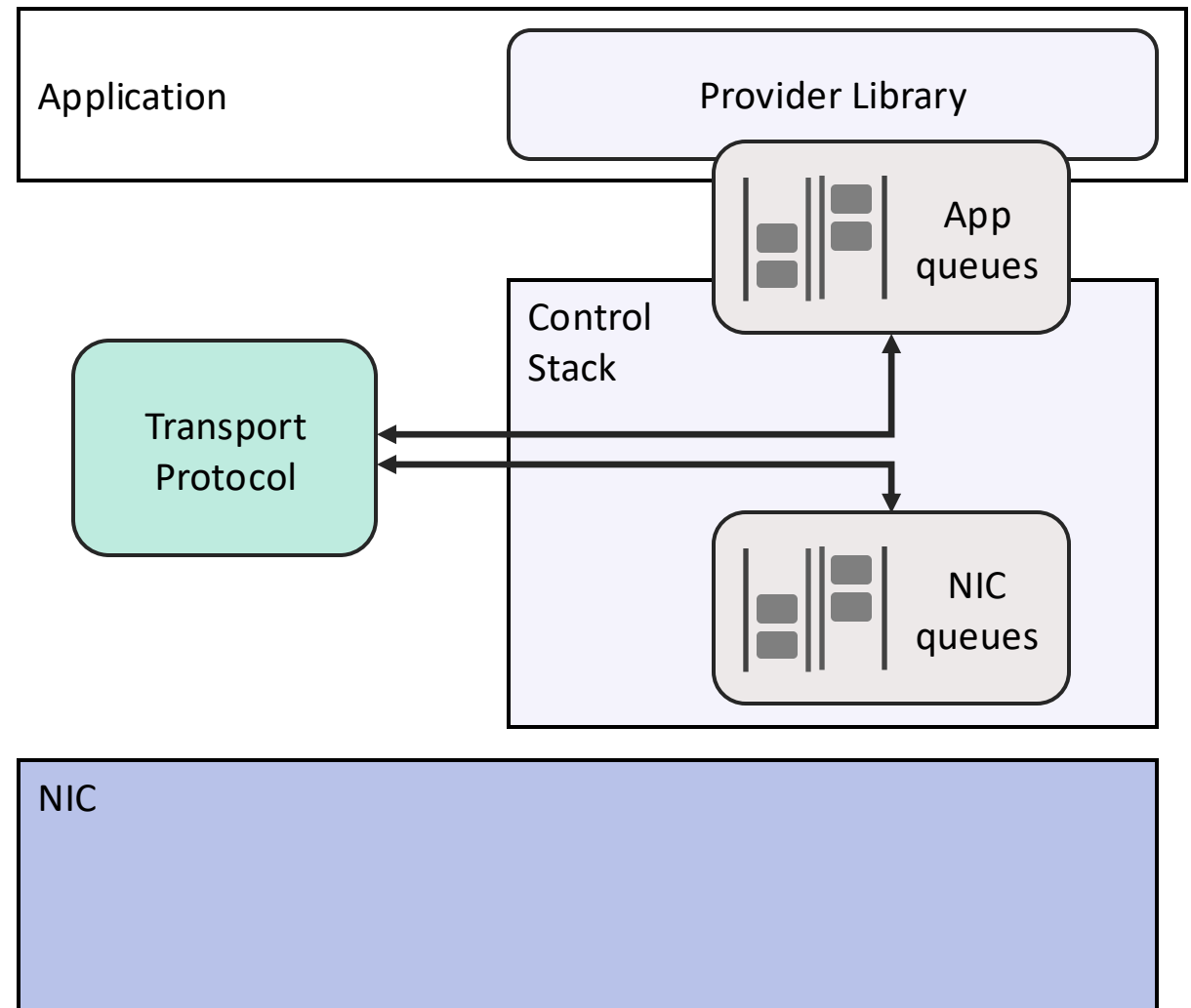
- 1 set of queues connects NIC and application layer
- Transport protocol in the NIC to enqueue receive completions



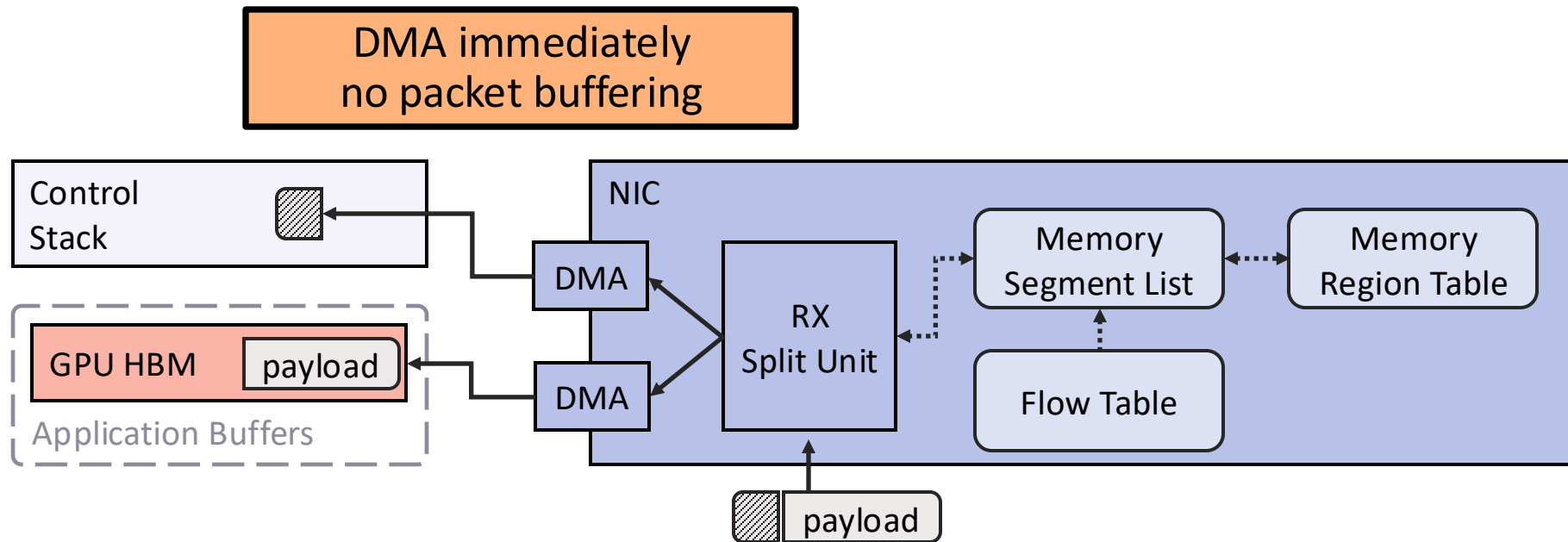
SW design: proxy protocol decisions

Our design

- 2 sets of queues interposing control stack and transport layer
- Transport protocol (anywhere) decides when to expose data to application

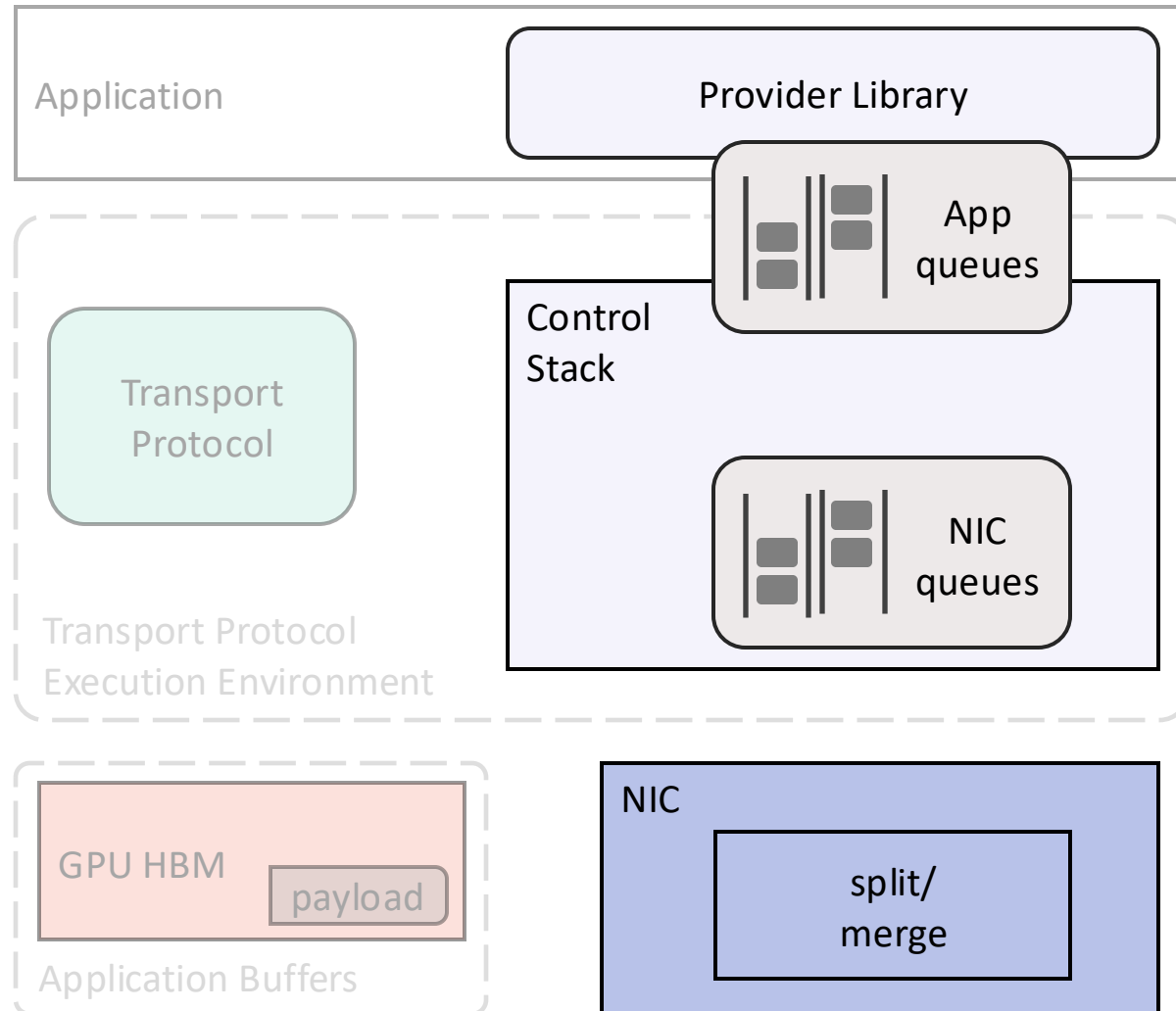


HW design: where to DMA data



Support 10,000 zero-copy flows with <10MB of NIC state

Receiving out-of-order packets

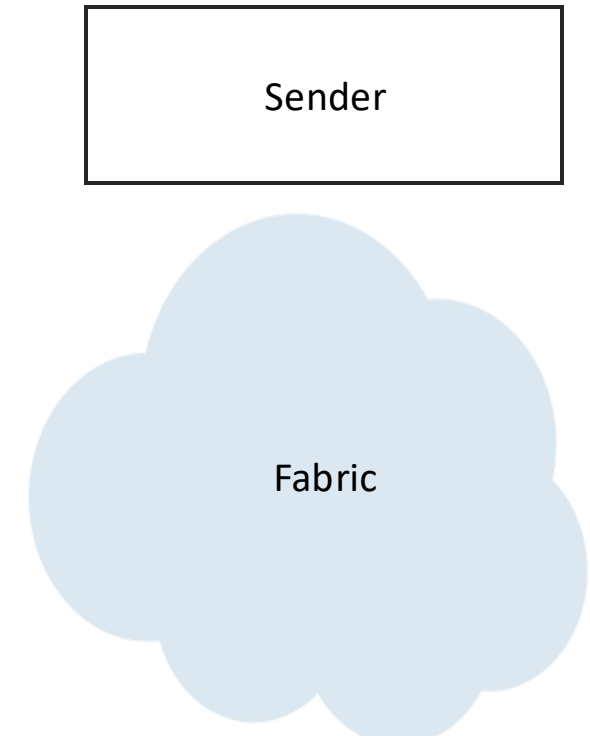
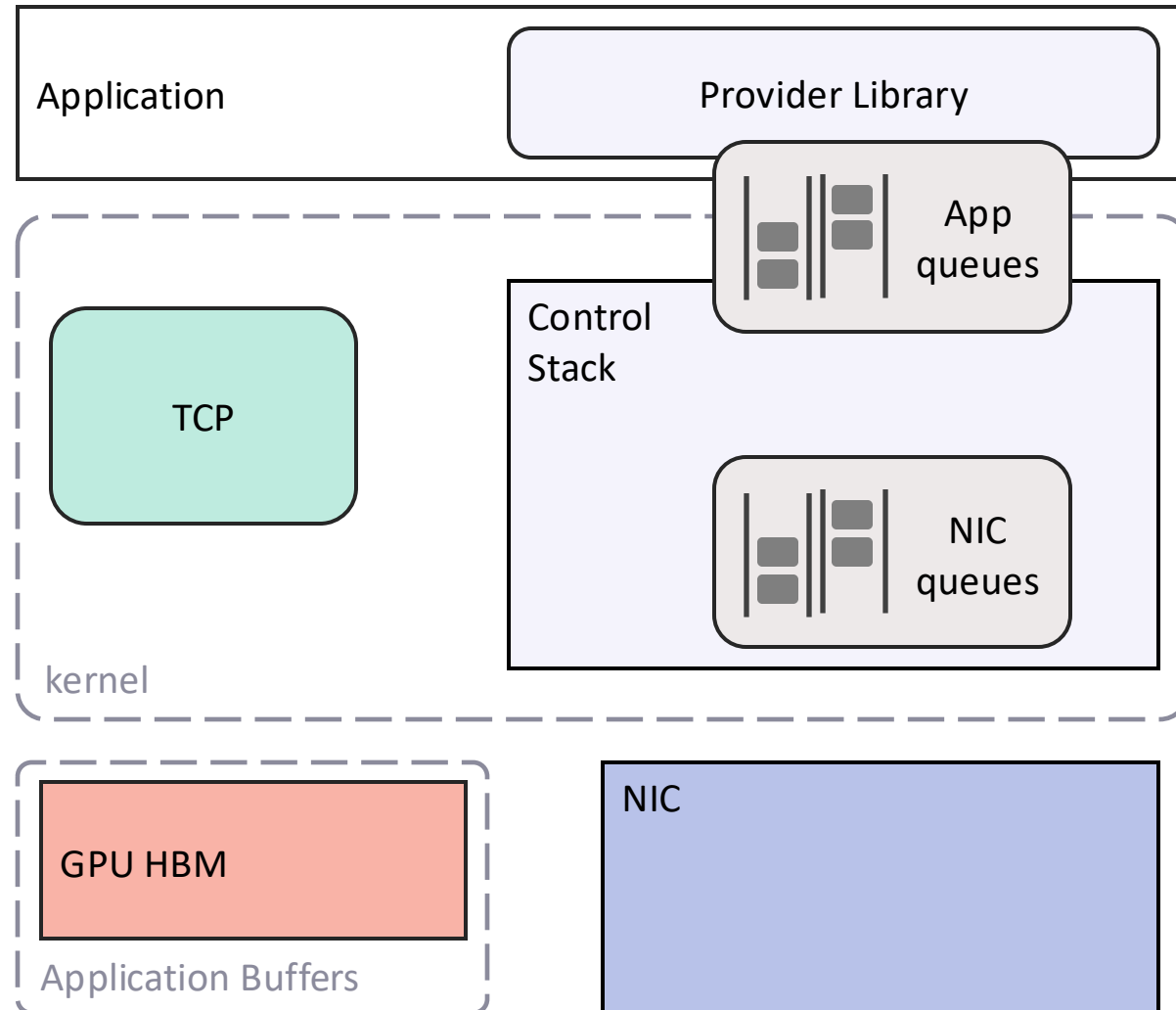


Prior RDMA stacks (IB/RoCE)

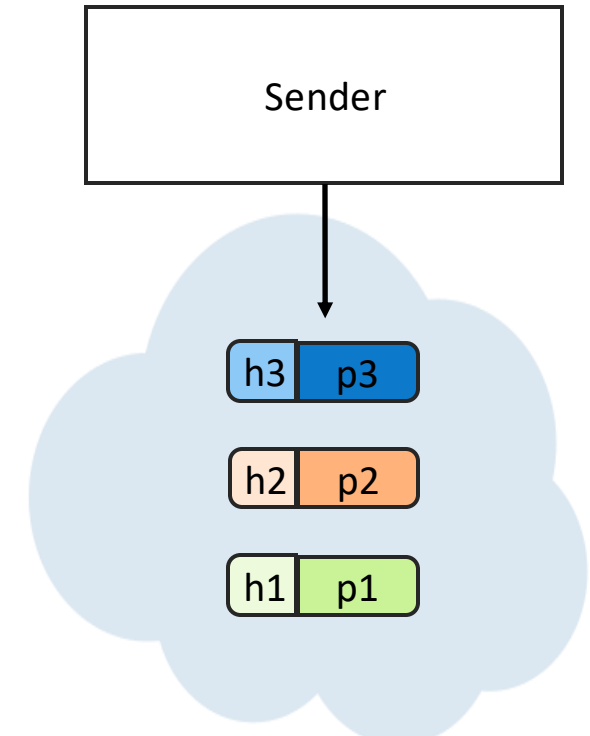
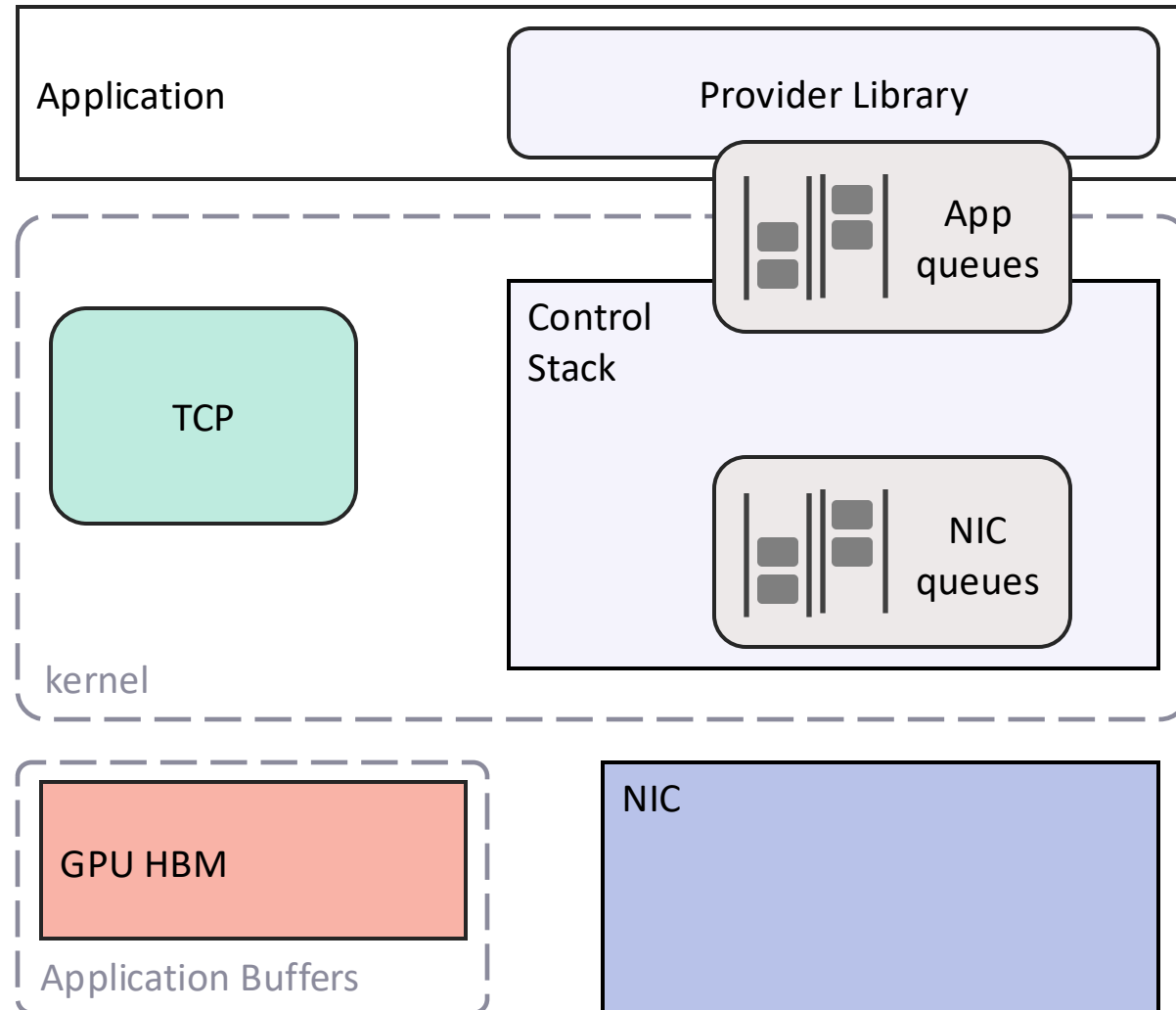
- Performance collapse under network perturbations (drops, retransmissions, reorderings, corruption, ...)
- L2 mechanisms to avoid them

Our design ~~avoids~~ **embraces** packet perturbations

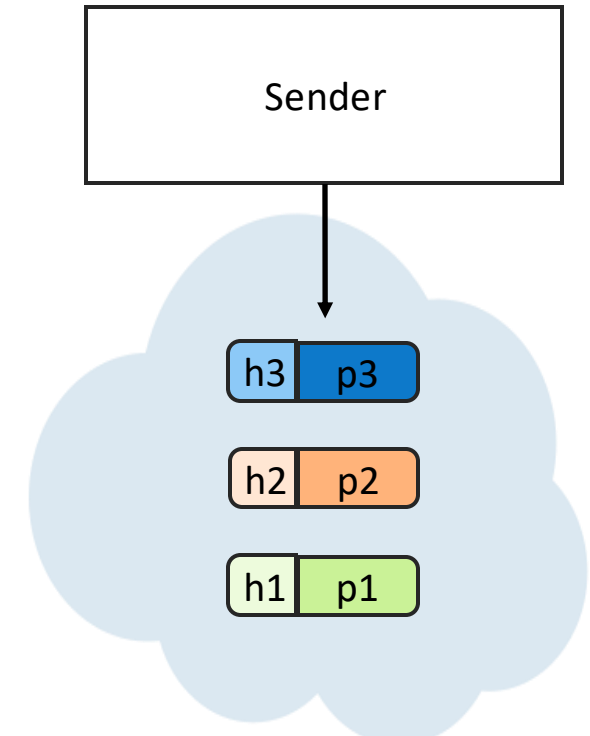
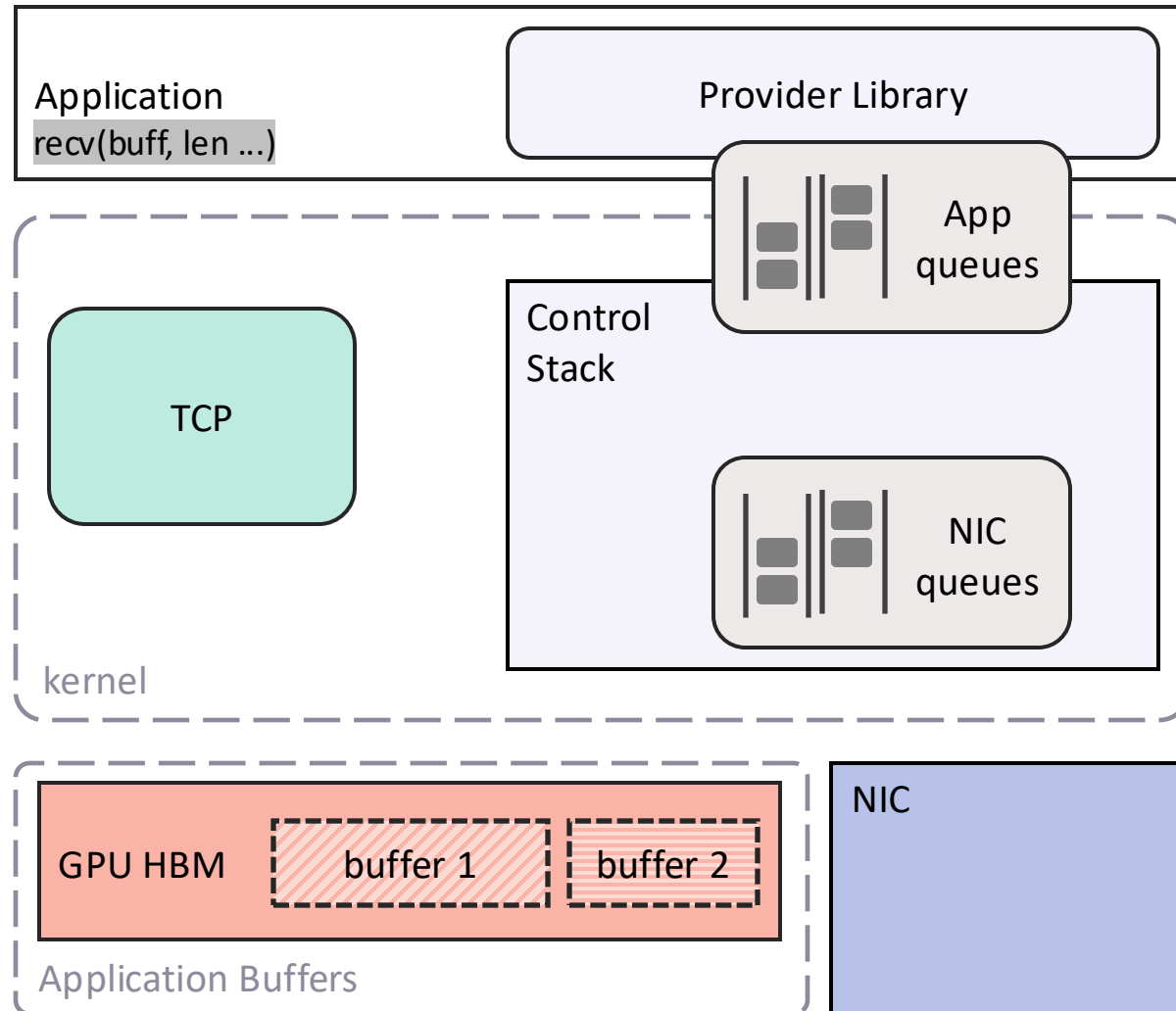
Receiving out-of-order packets



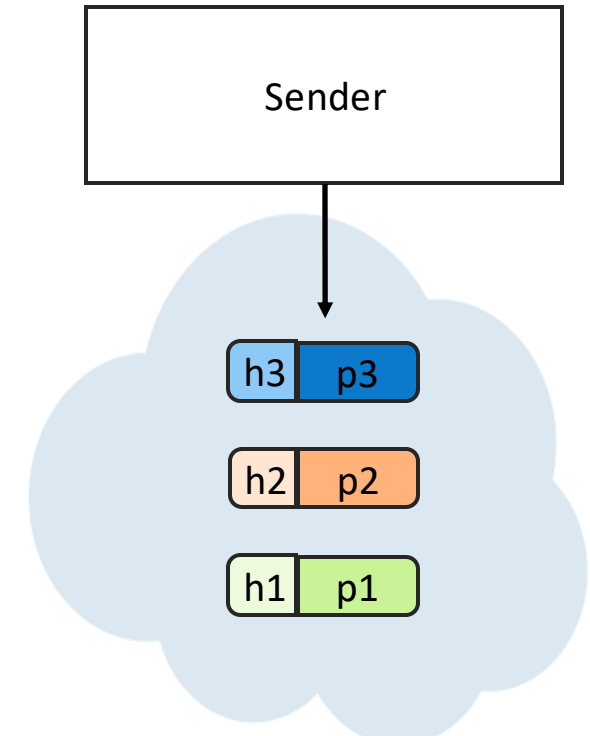
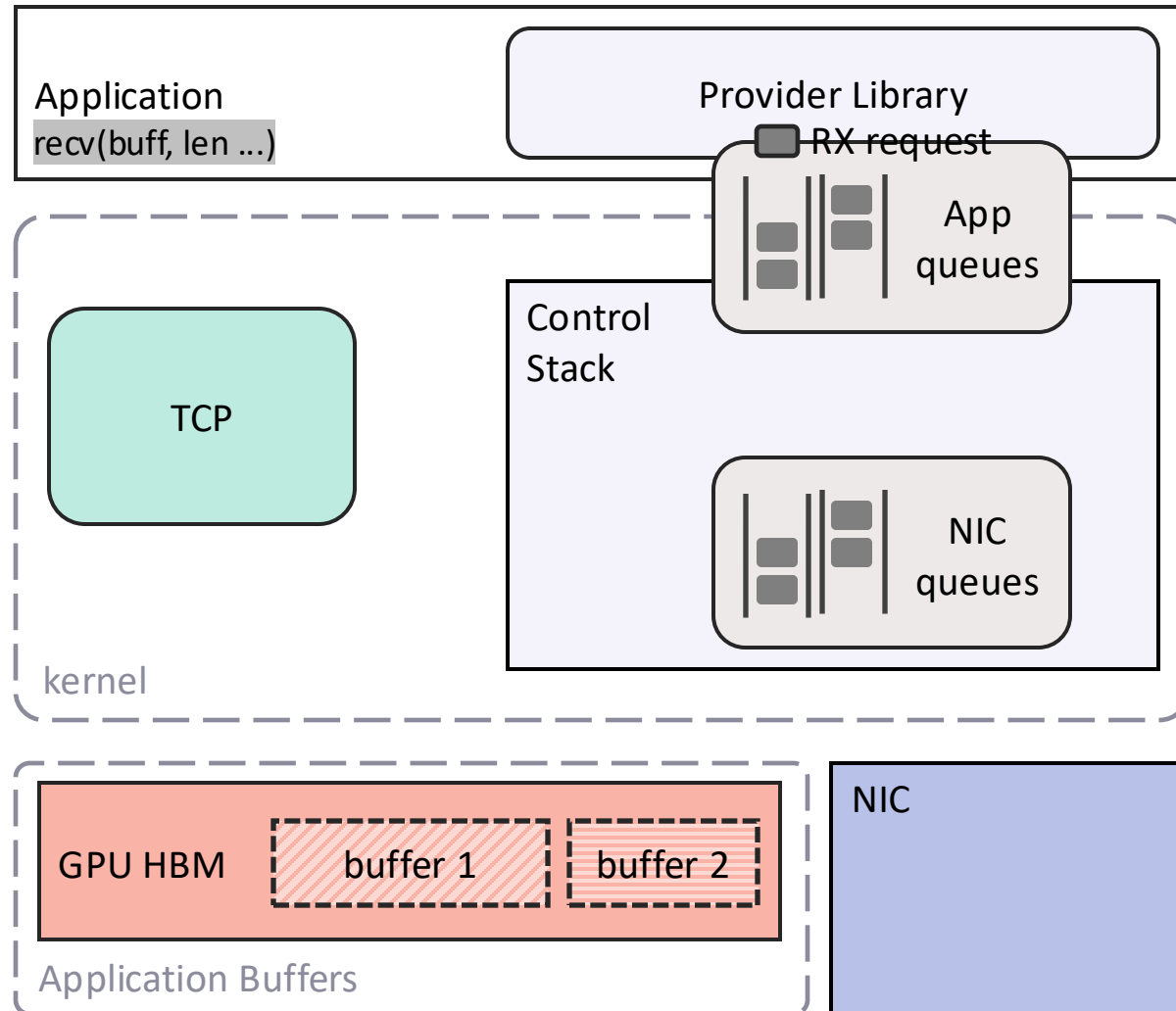
Receiving out-of-order packets



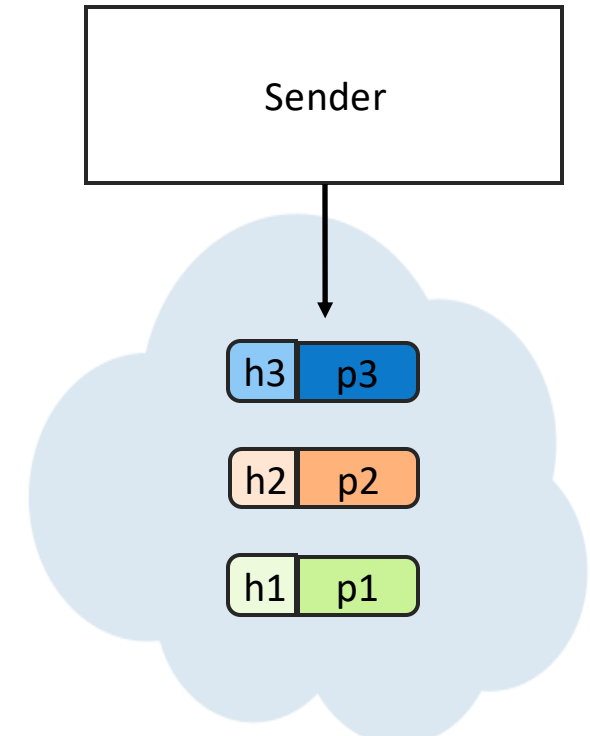
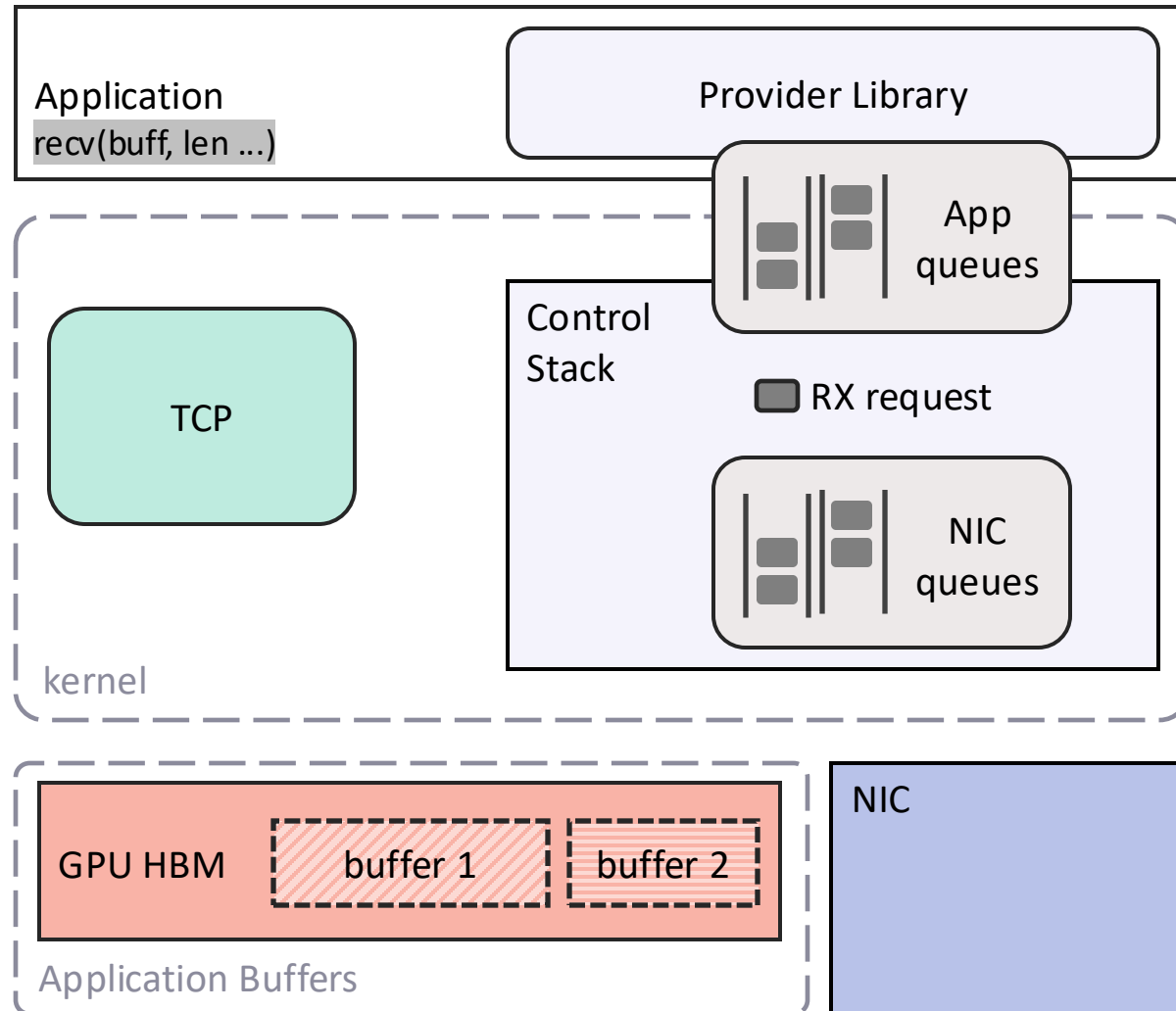
Receiving out-of-order packets



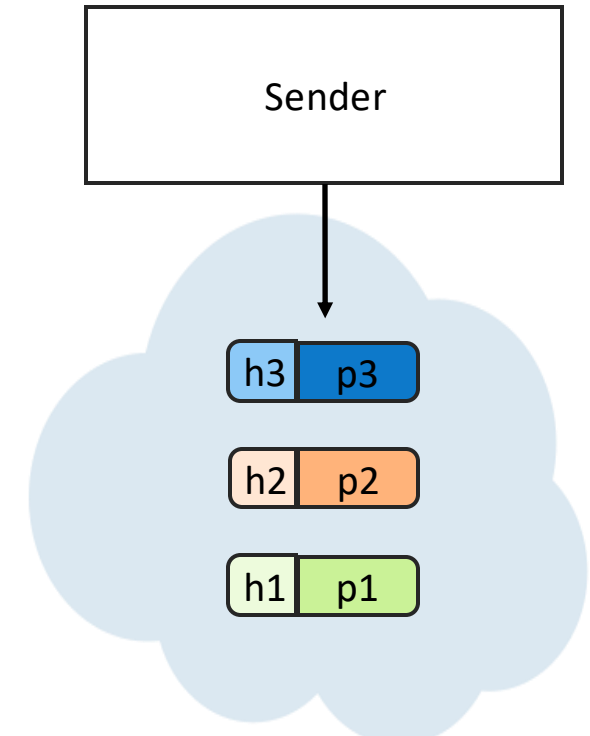
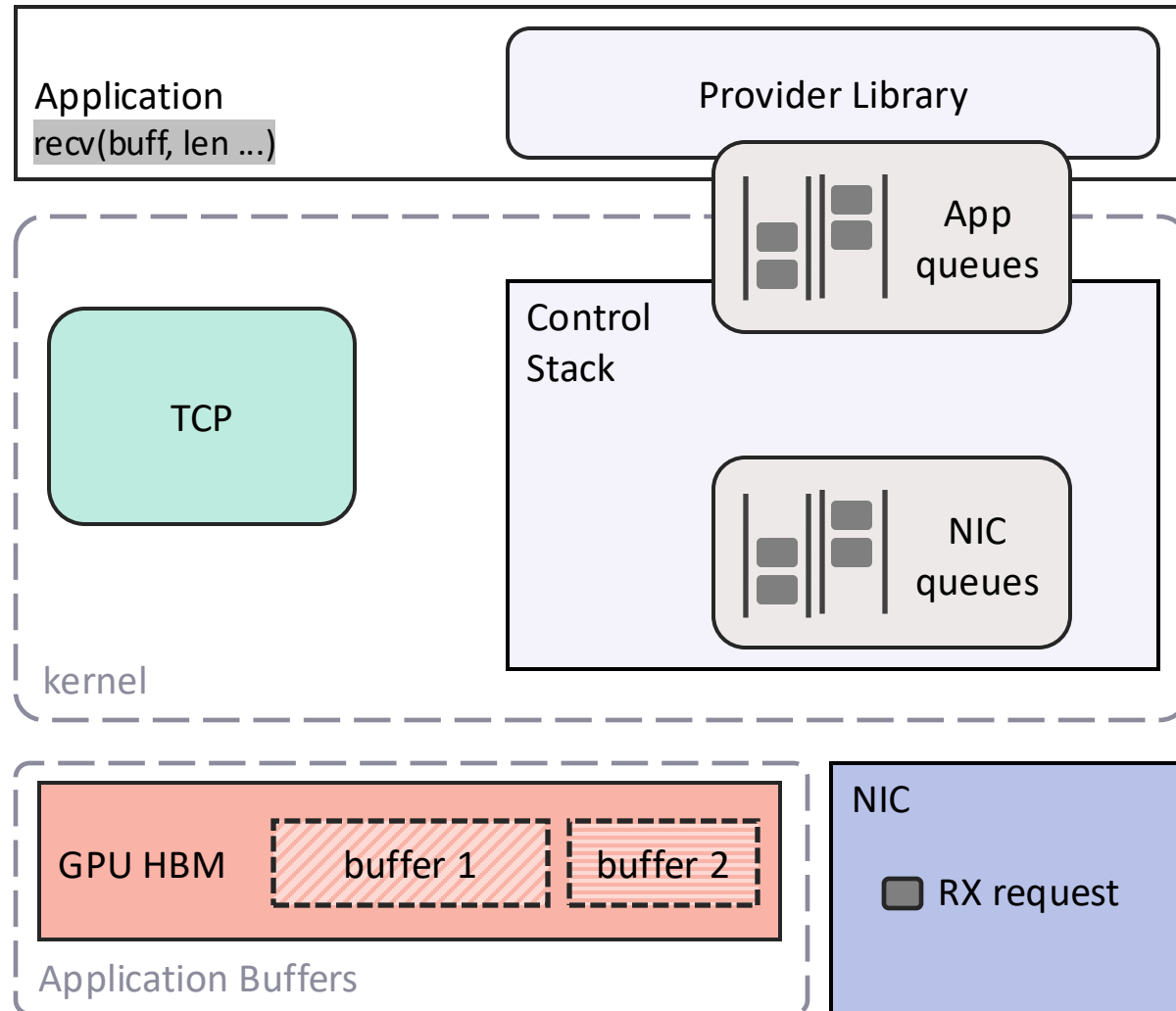
Receiving out-of-order packets



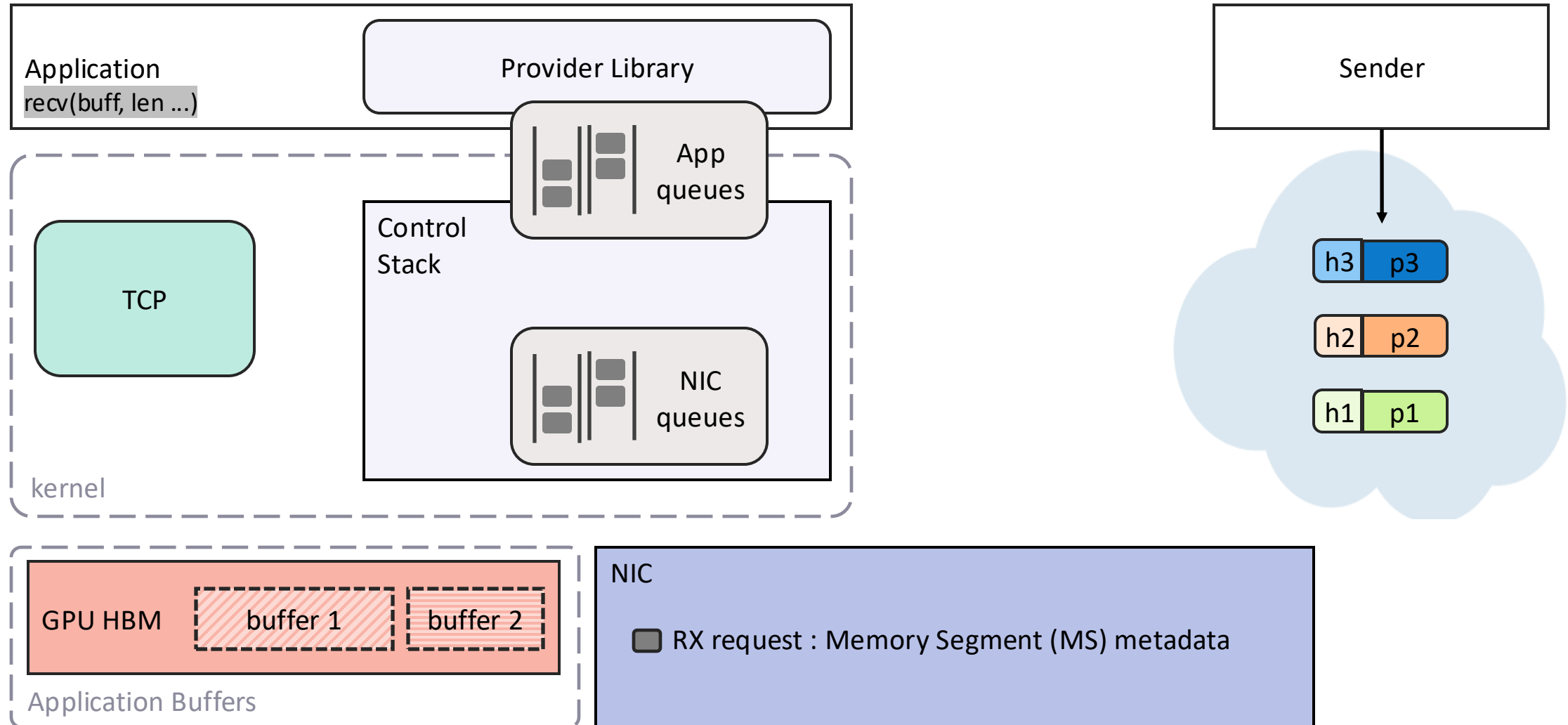
Receiving out-of-order packets



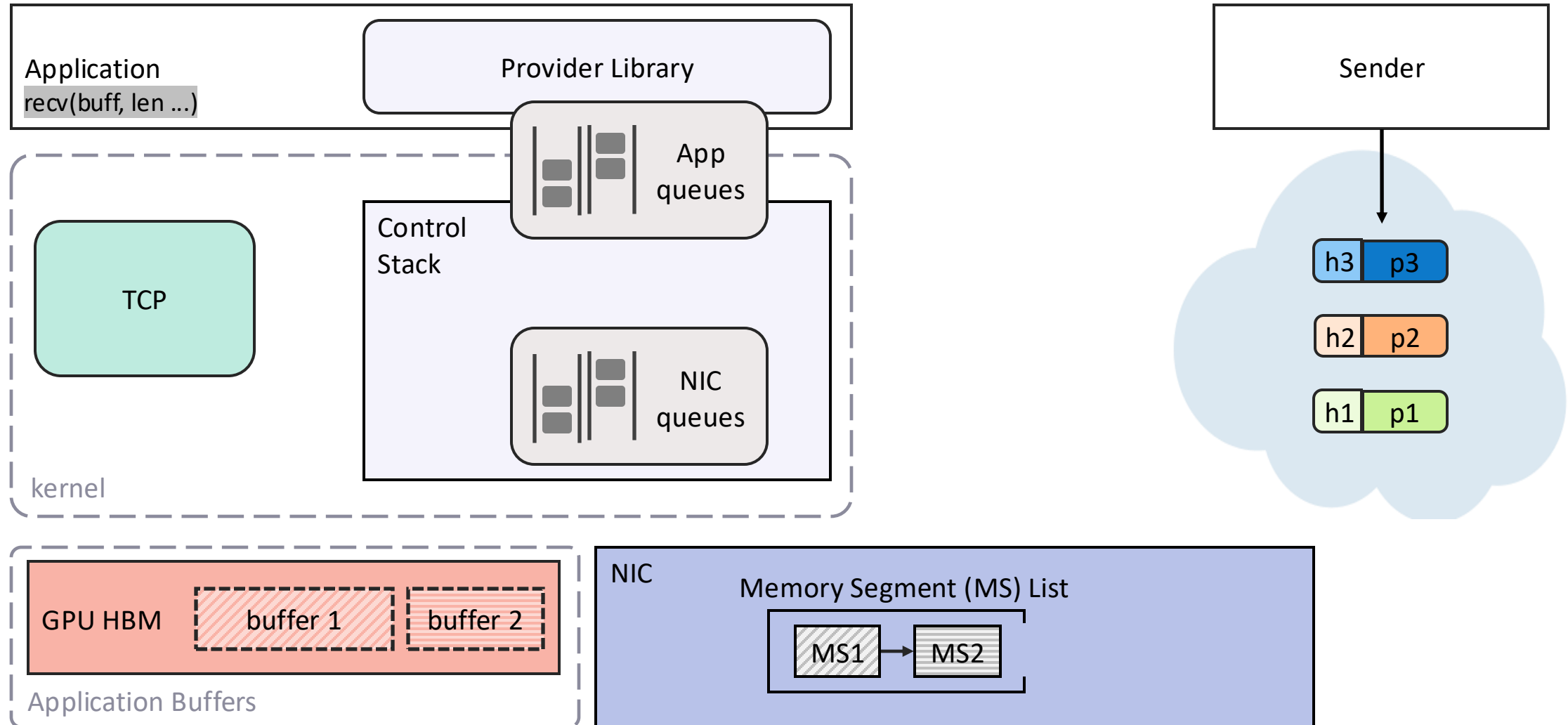
Receiving out-of-order packets



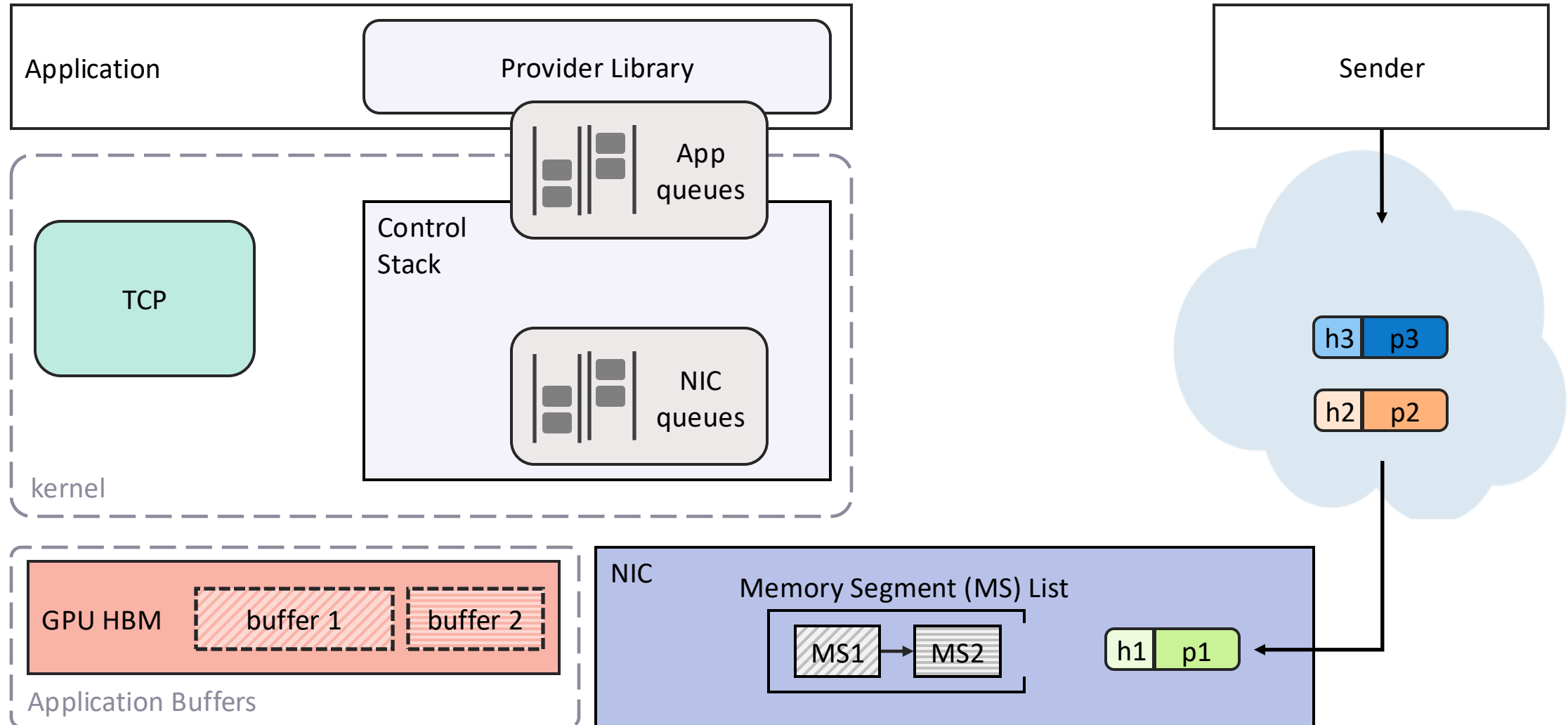
Receiving out-of-order packets



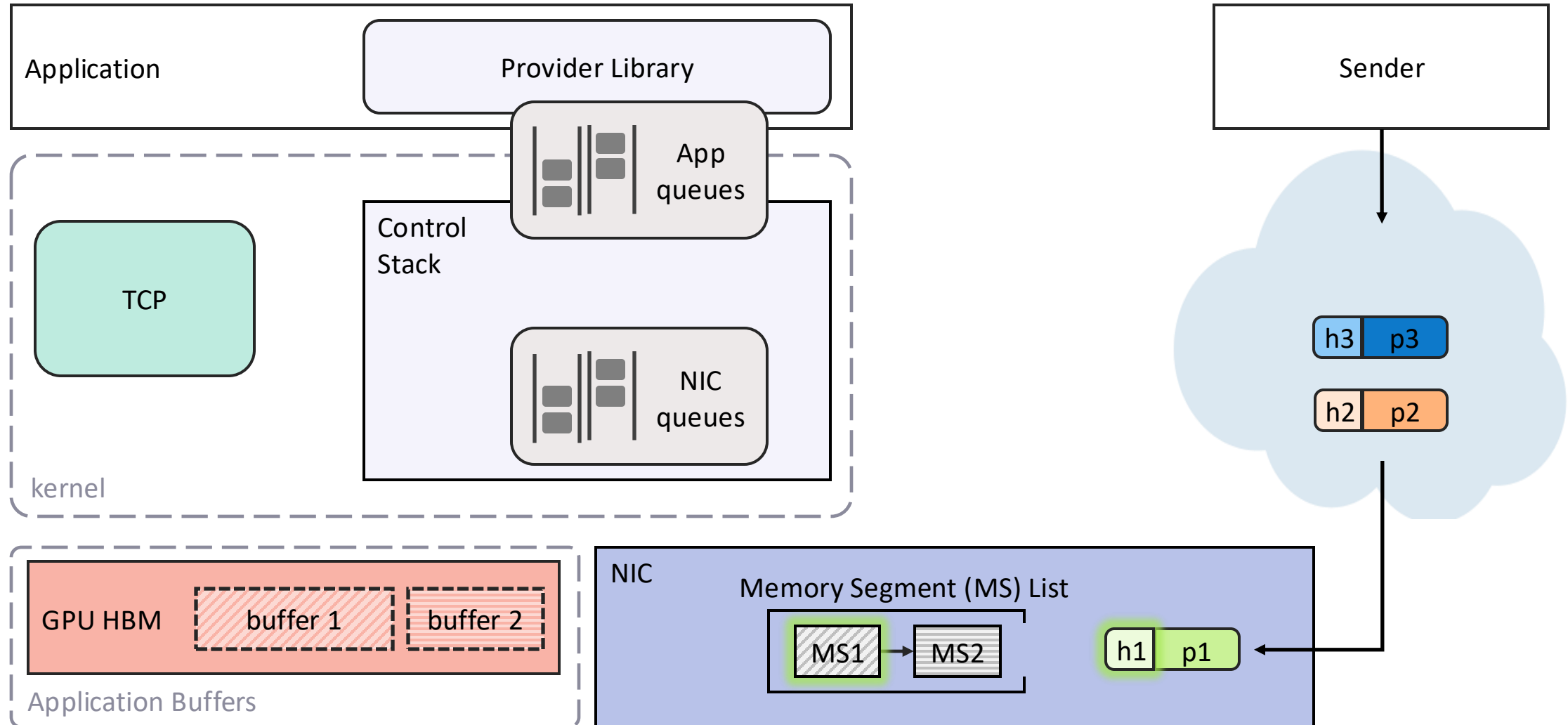
Receiving out-of-order packets



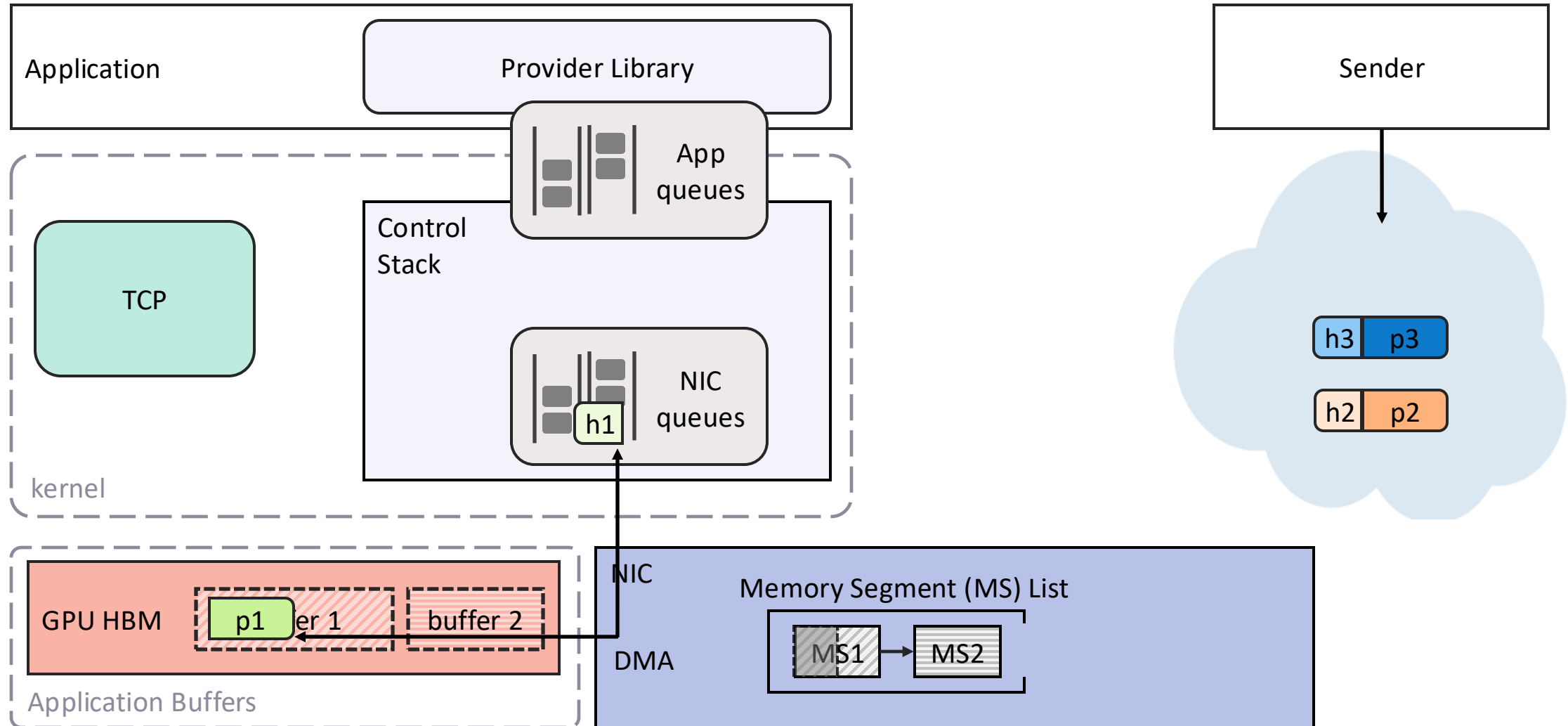
Receiving out-of-order packets



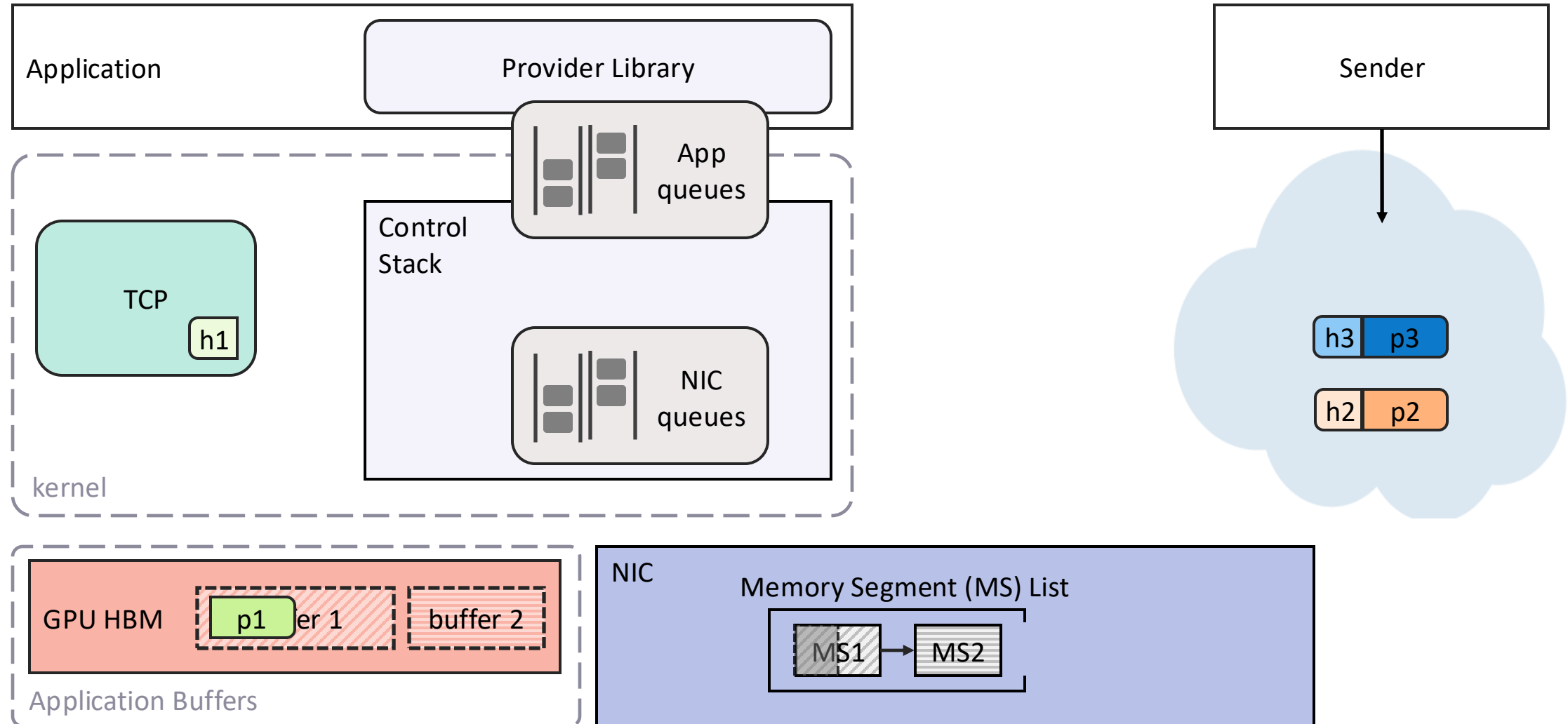
Receiving out-of-order packets



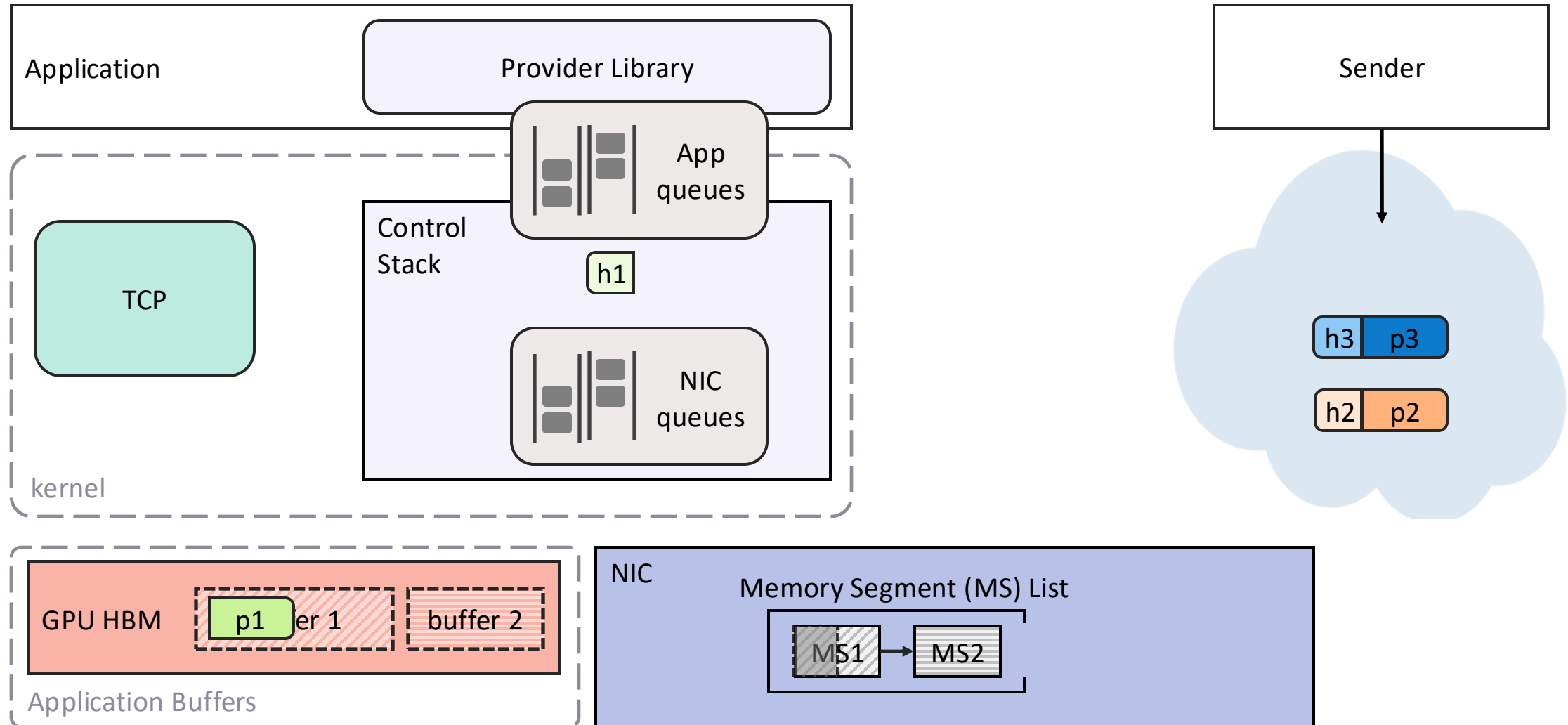
Receiving out-of-order packets



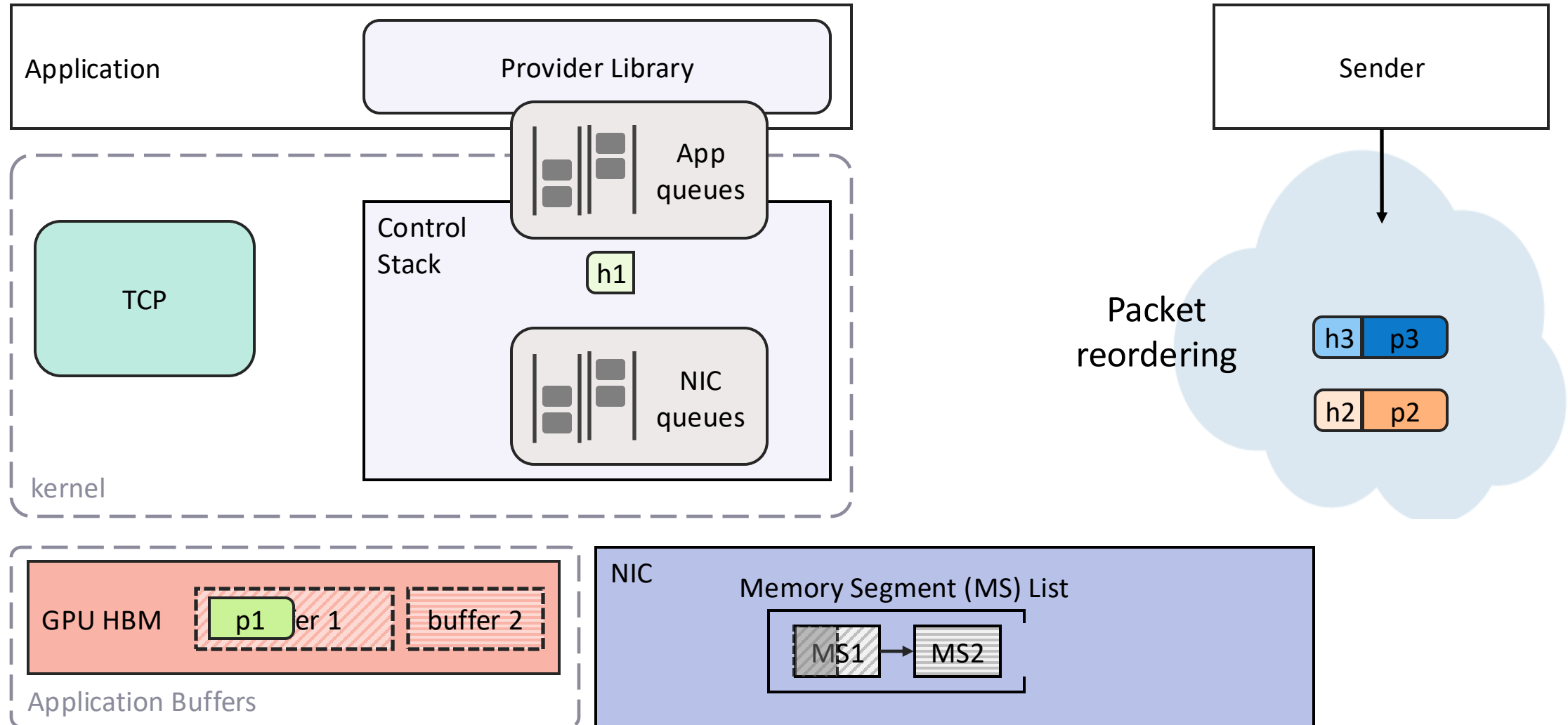
Receiving out-of-order packets



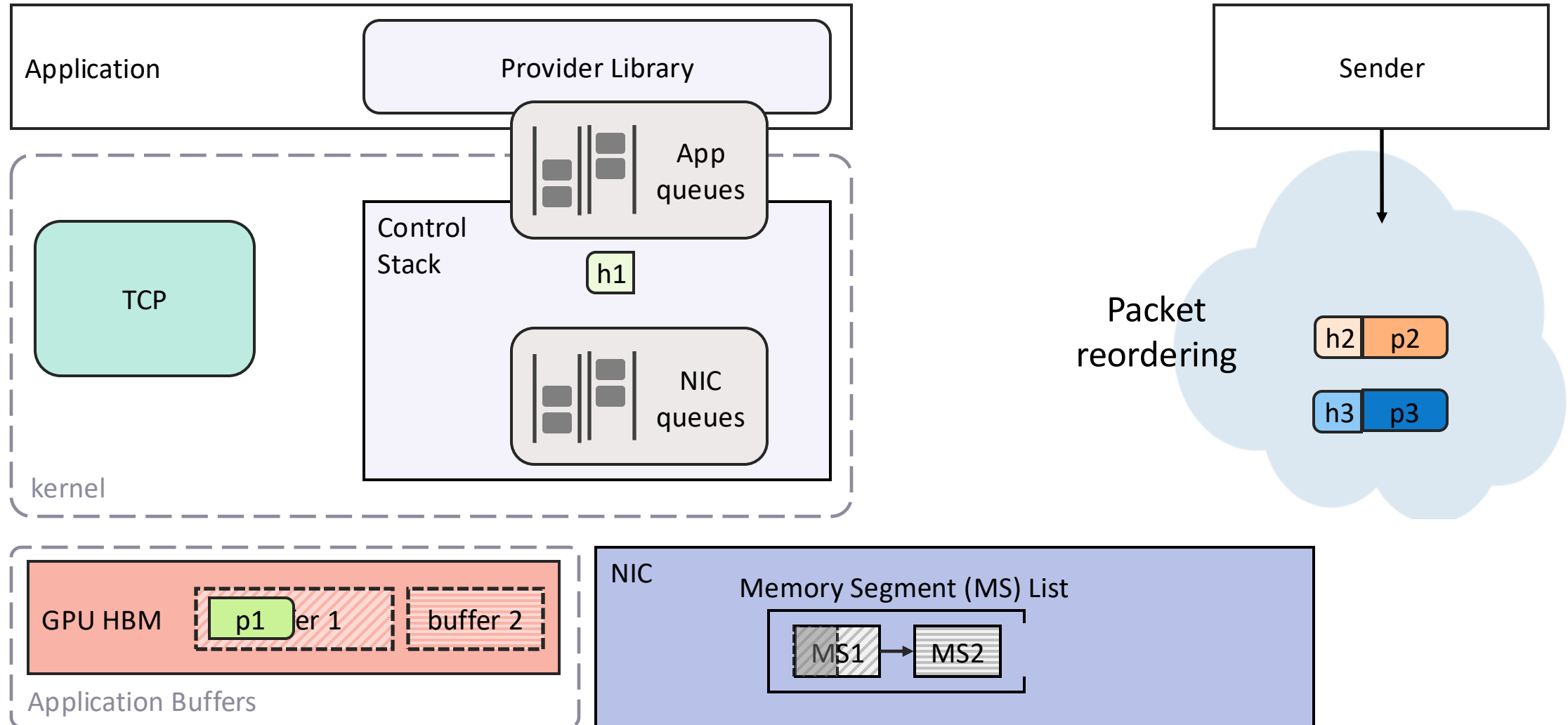
Receiving out-of-order packets



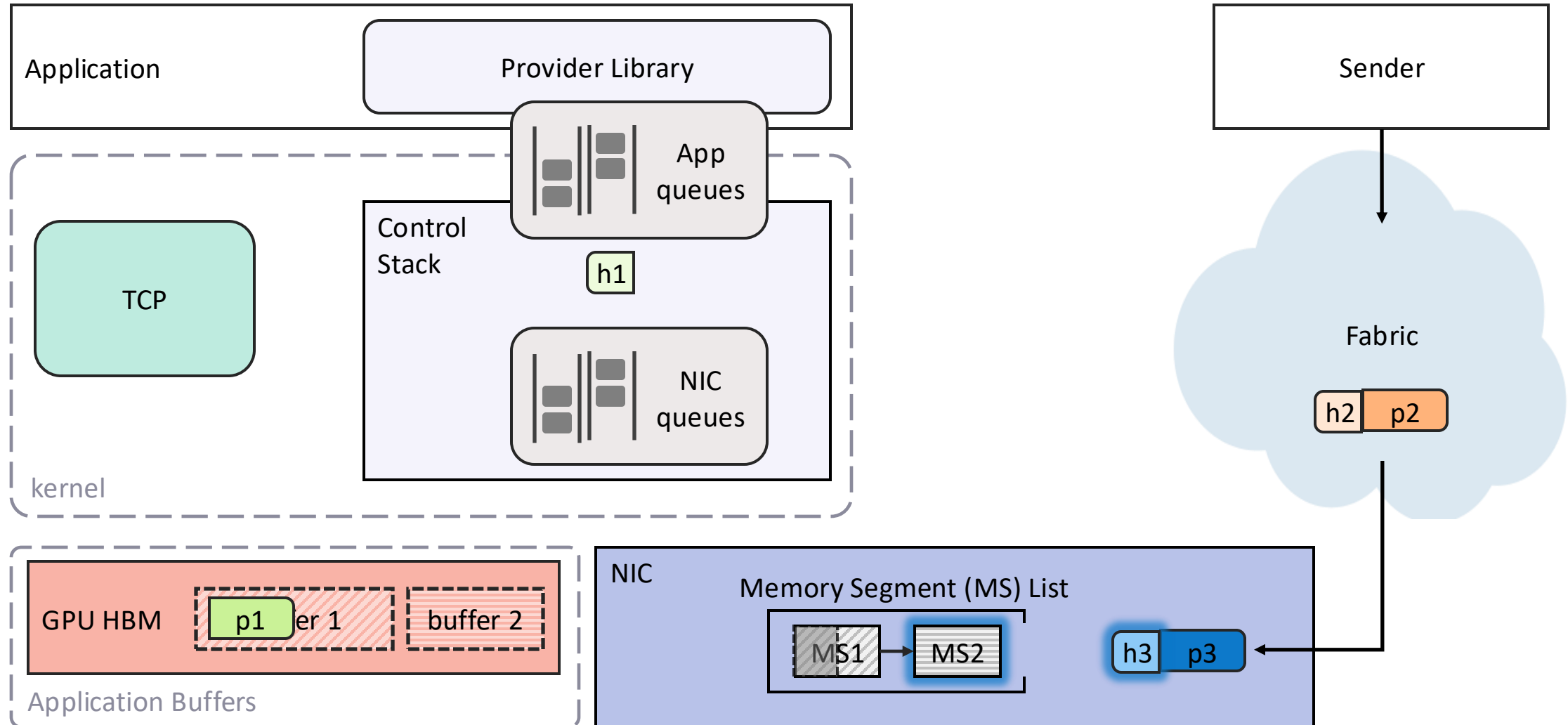
Receiving out-of-order packets



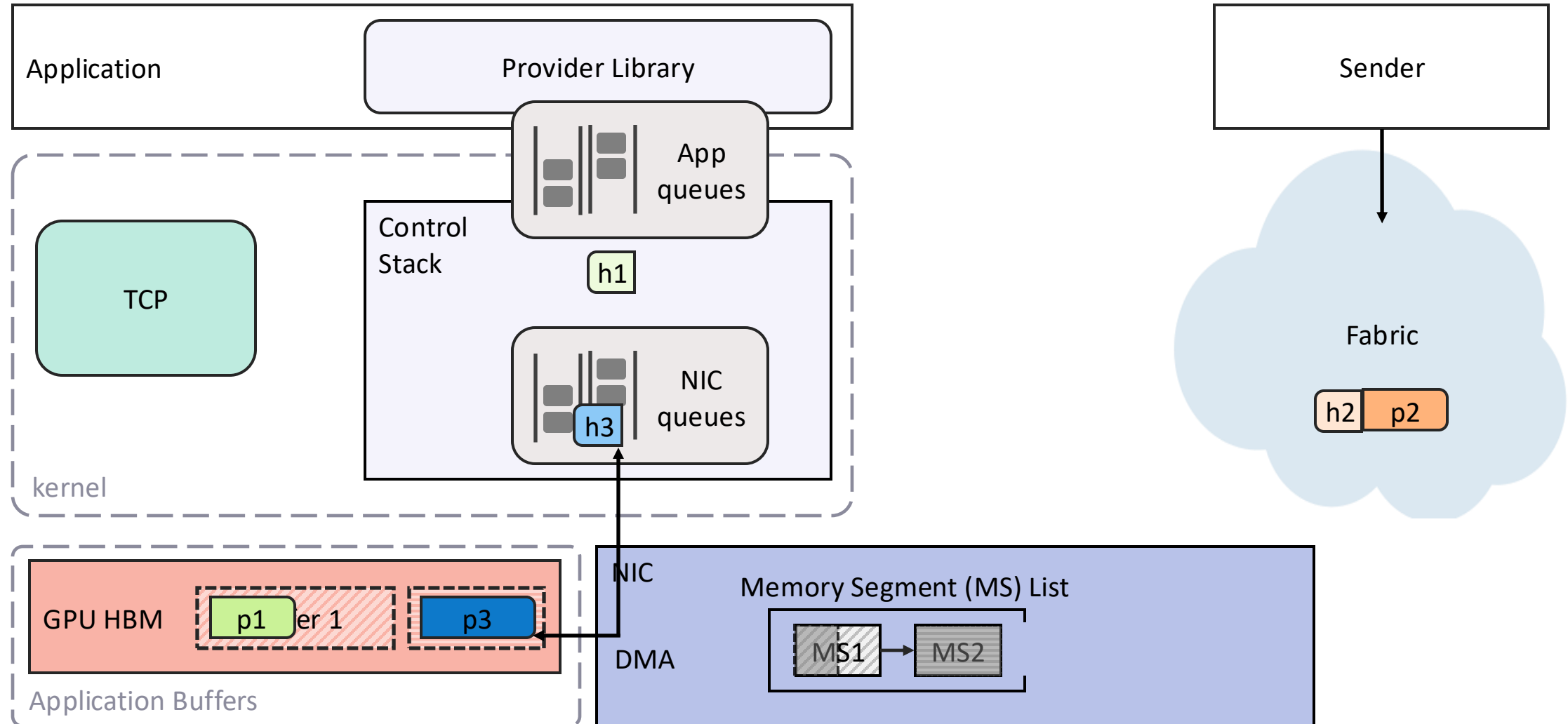
Receiving out-of-order packets



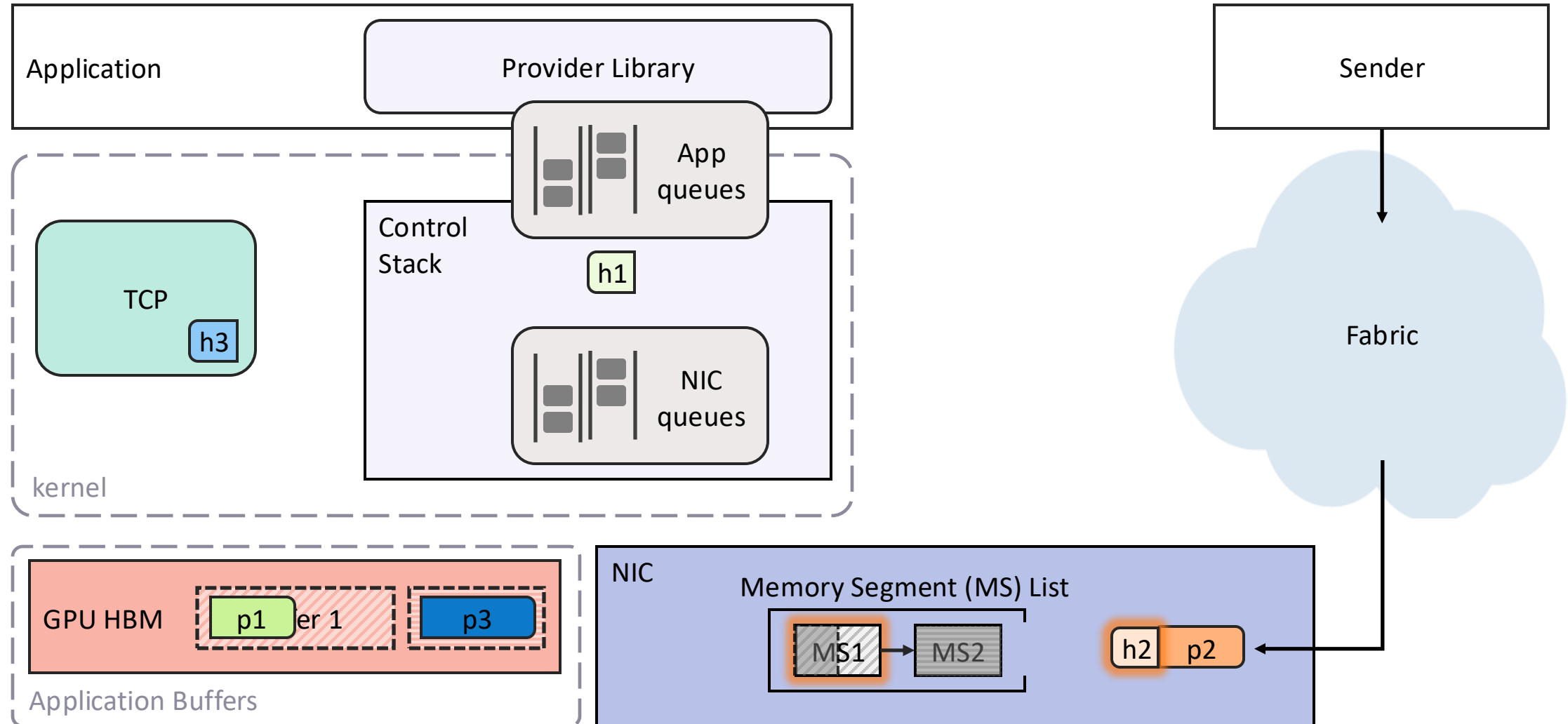
Receiving out-of-order packets



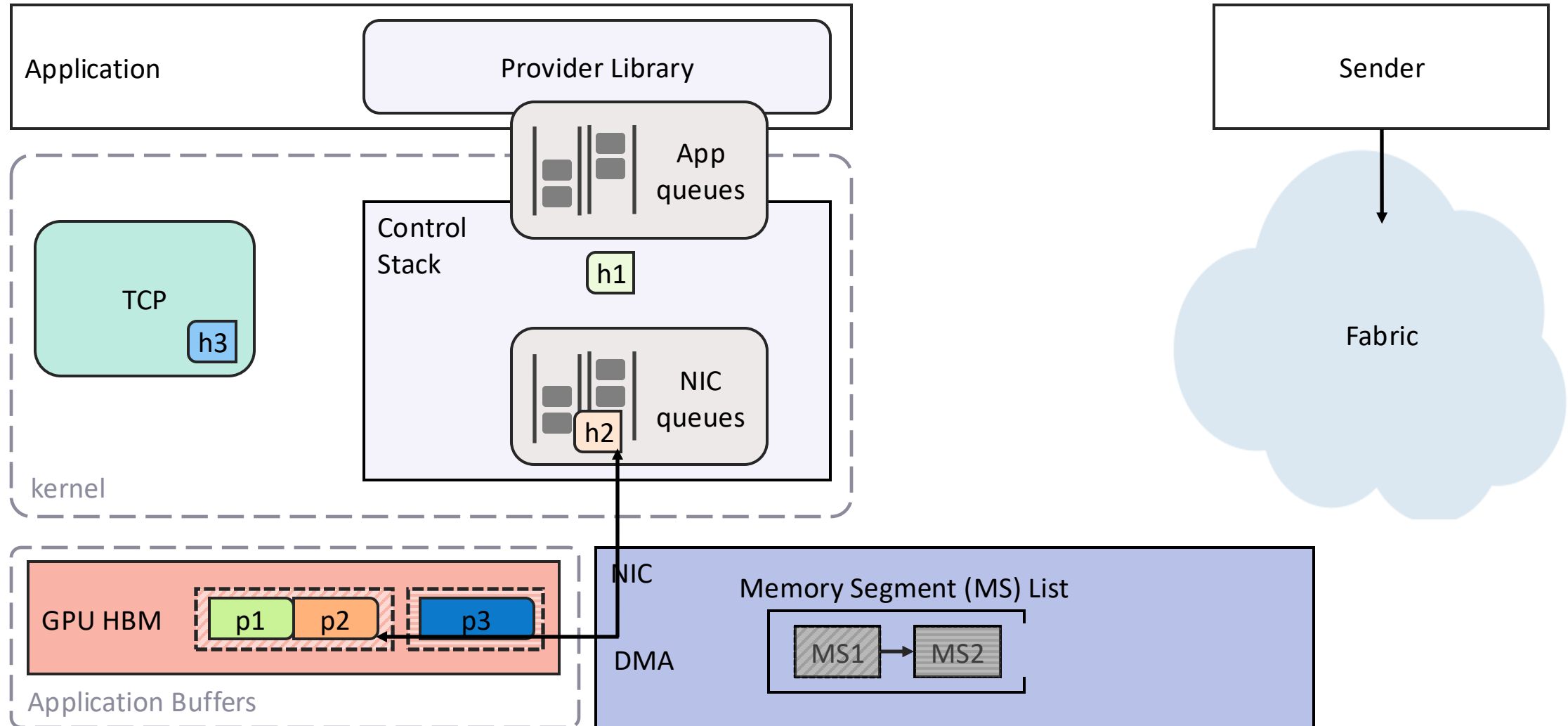
Receiving out-of-order packets



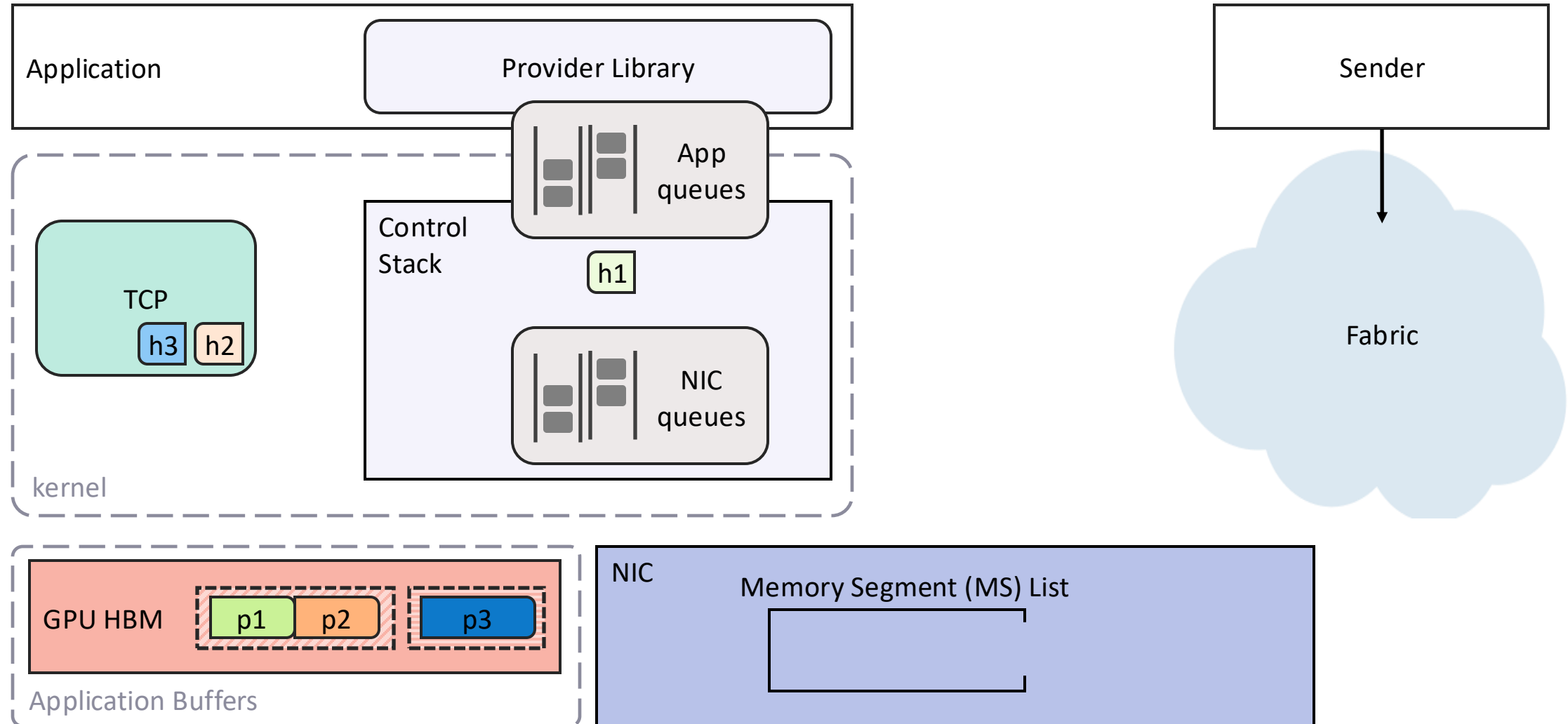
Receiving out-of-order packets



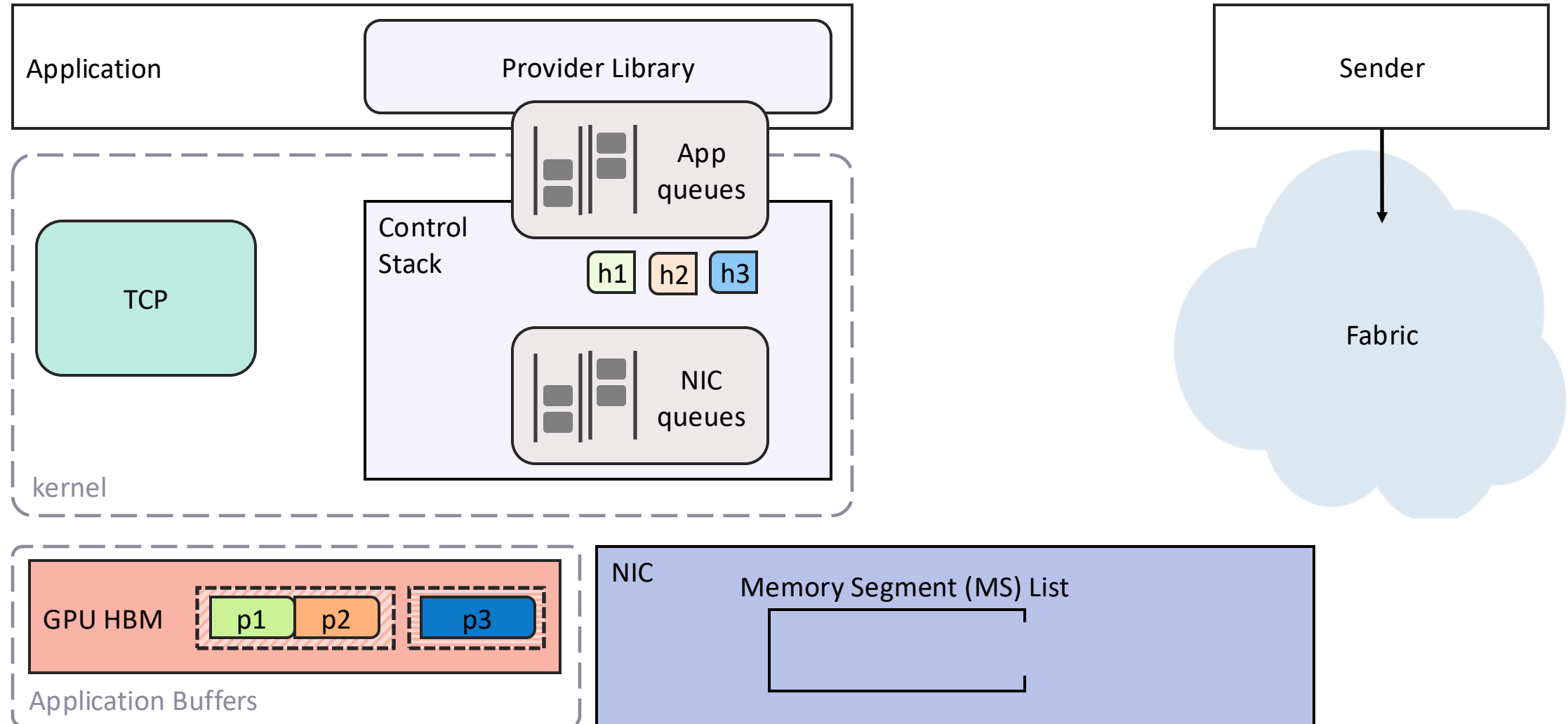
Receiving out-of-order packets



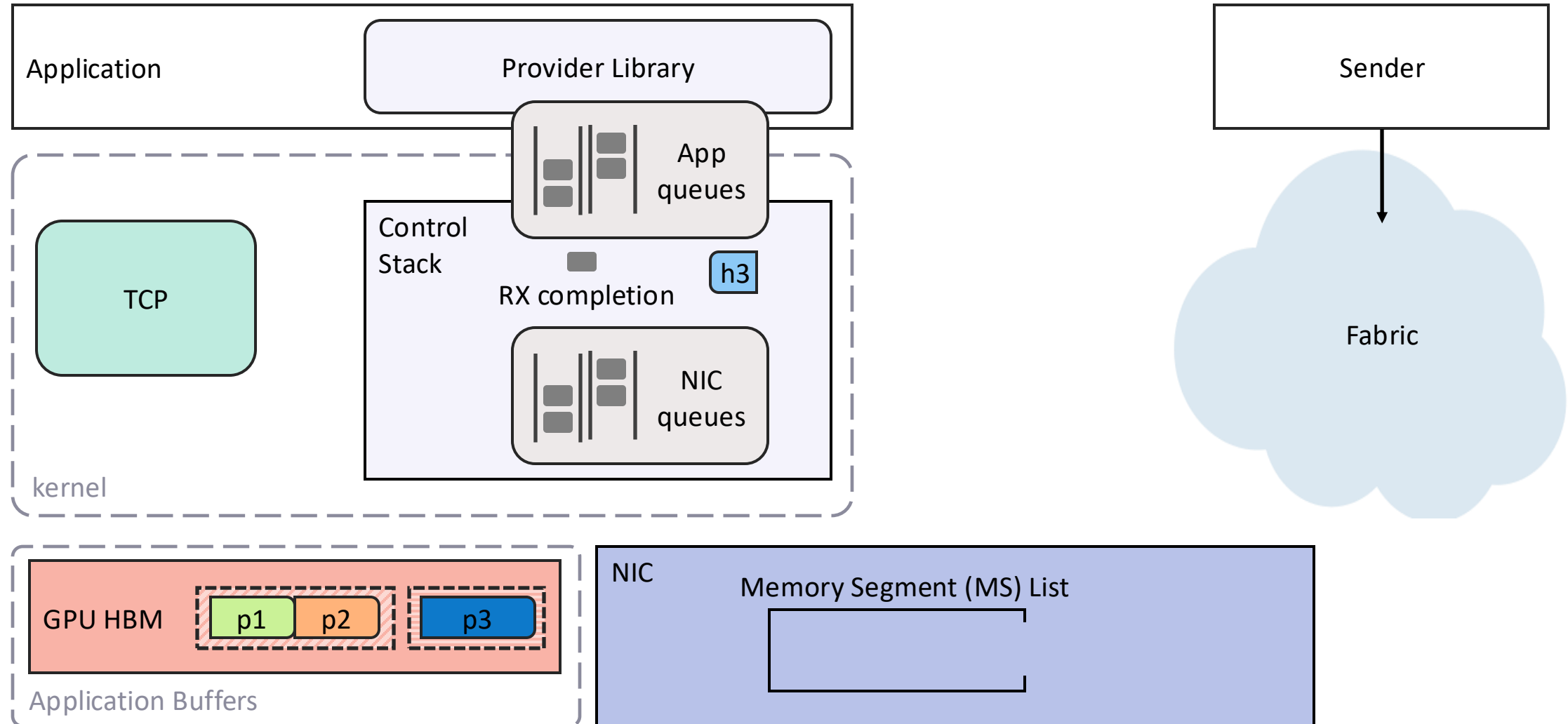
Receiving out-of-order packets



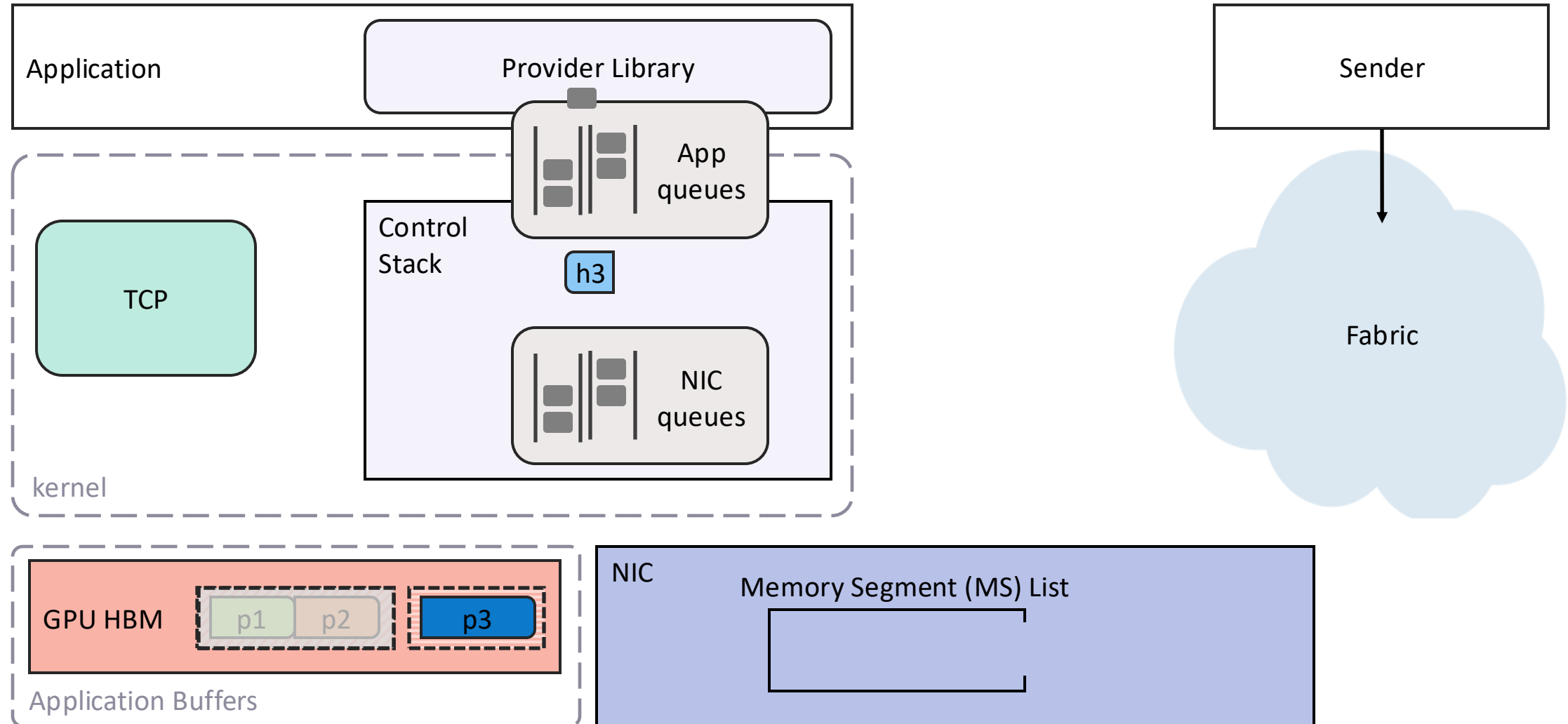
Receiving out-of-order packets



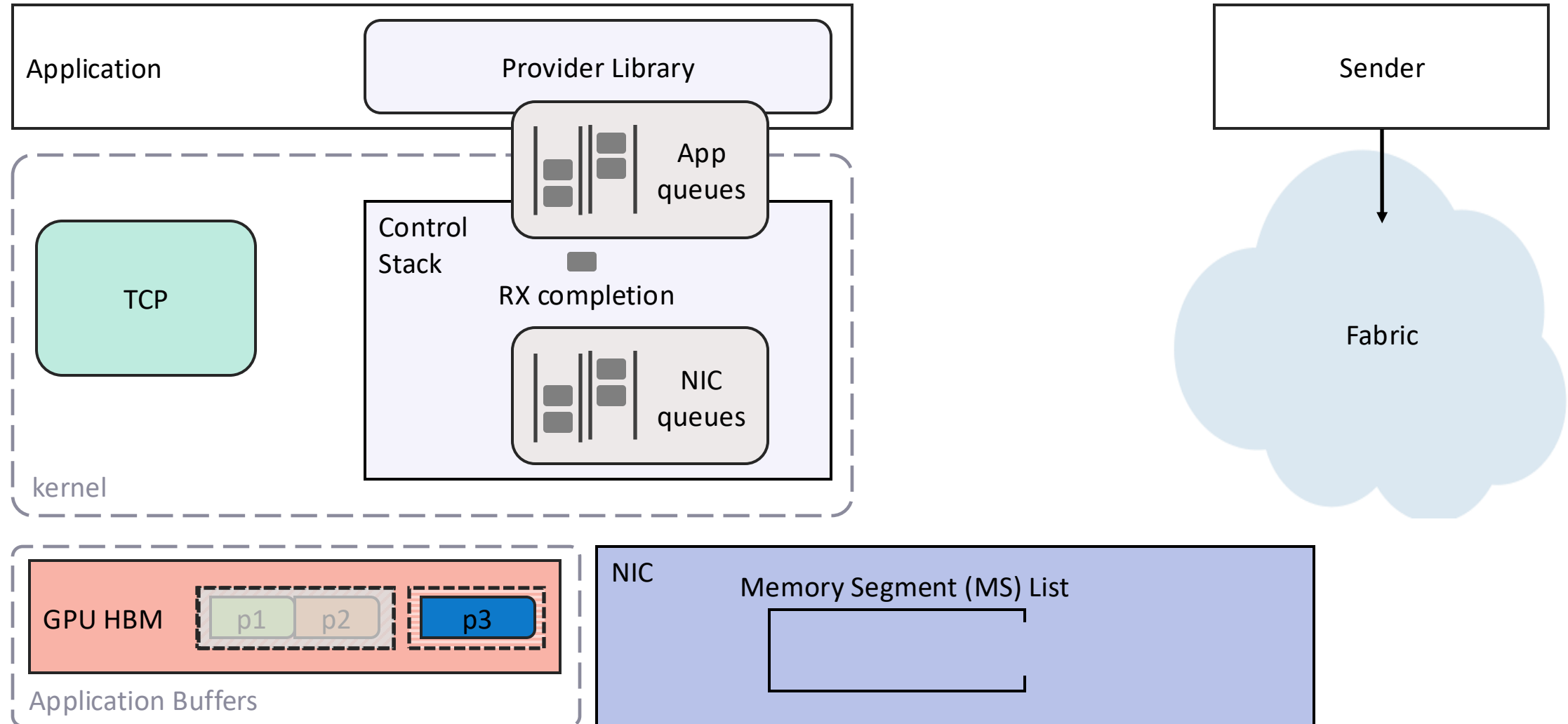
Receiving out-of-order packets



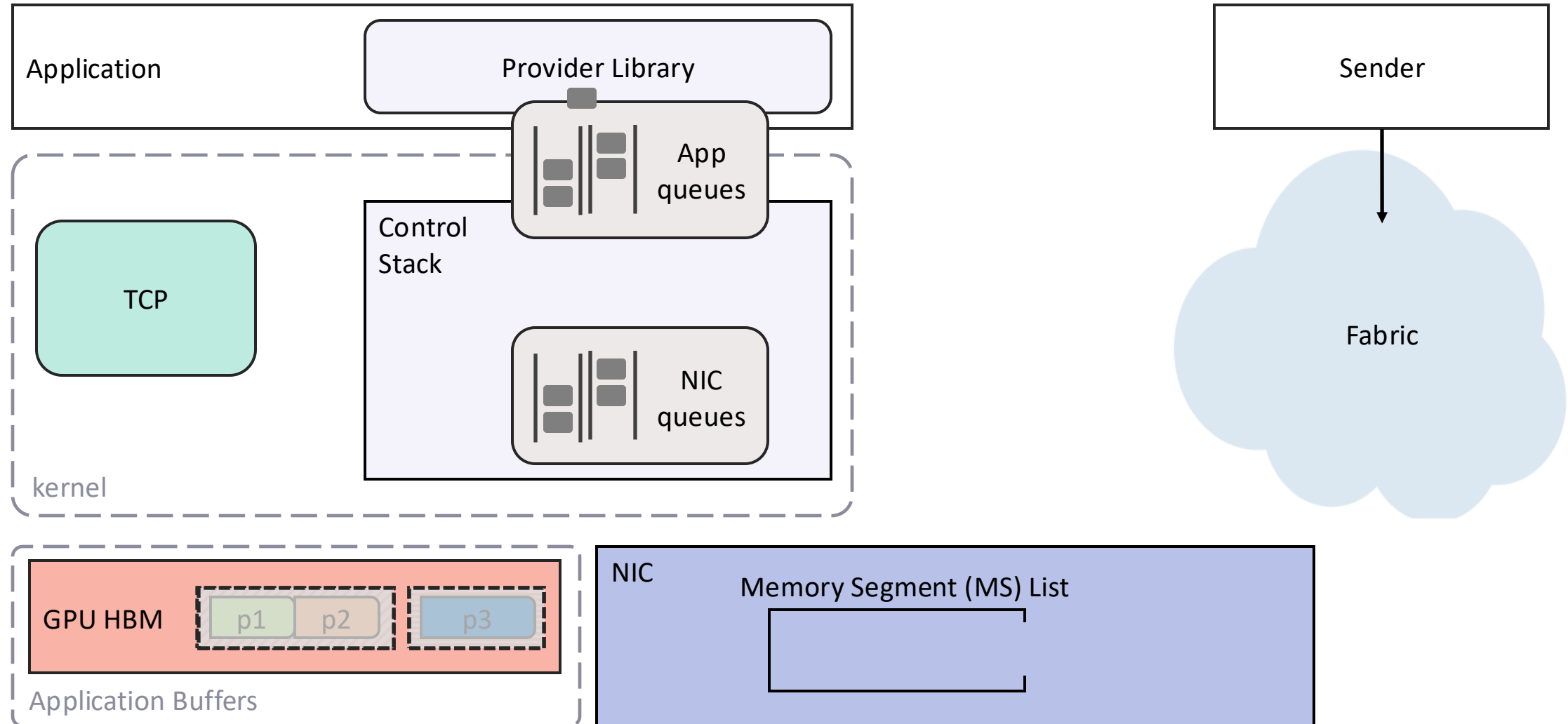
Receiving out-of-order packets



Receiving out-of-order packets



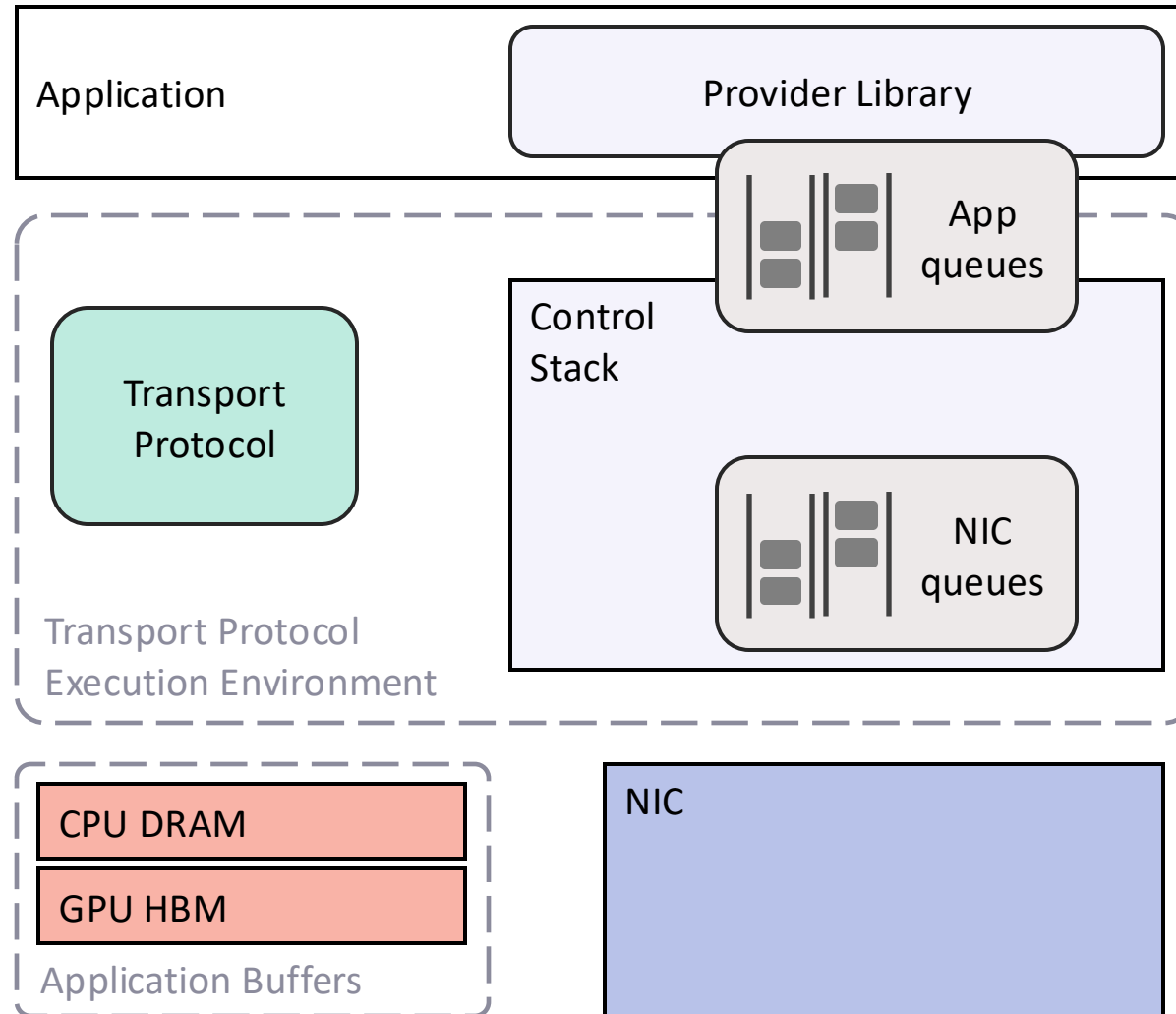
Receiving out-of-order packets



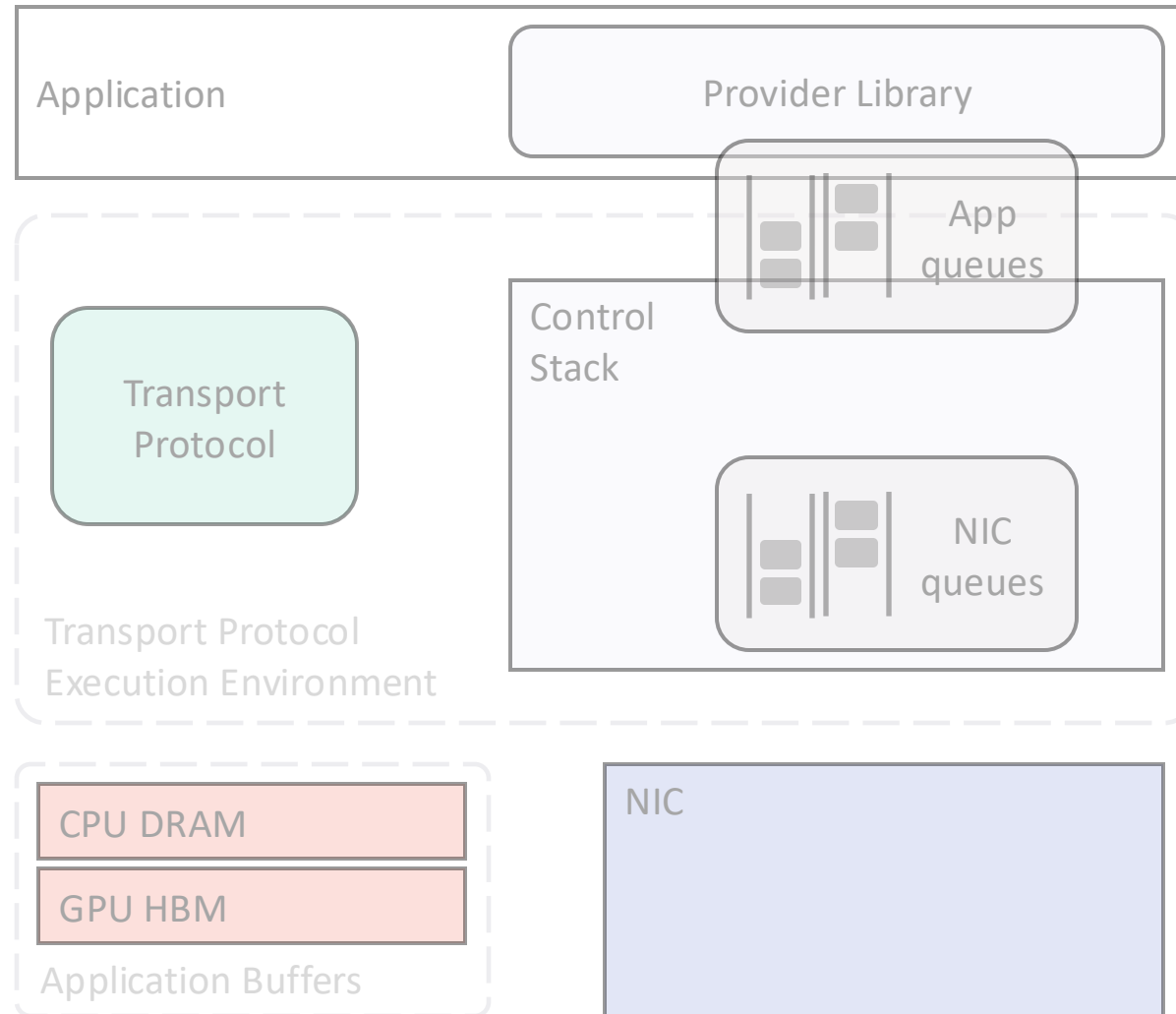
ZeroNIC

Implementation Prototype and Evaluation

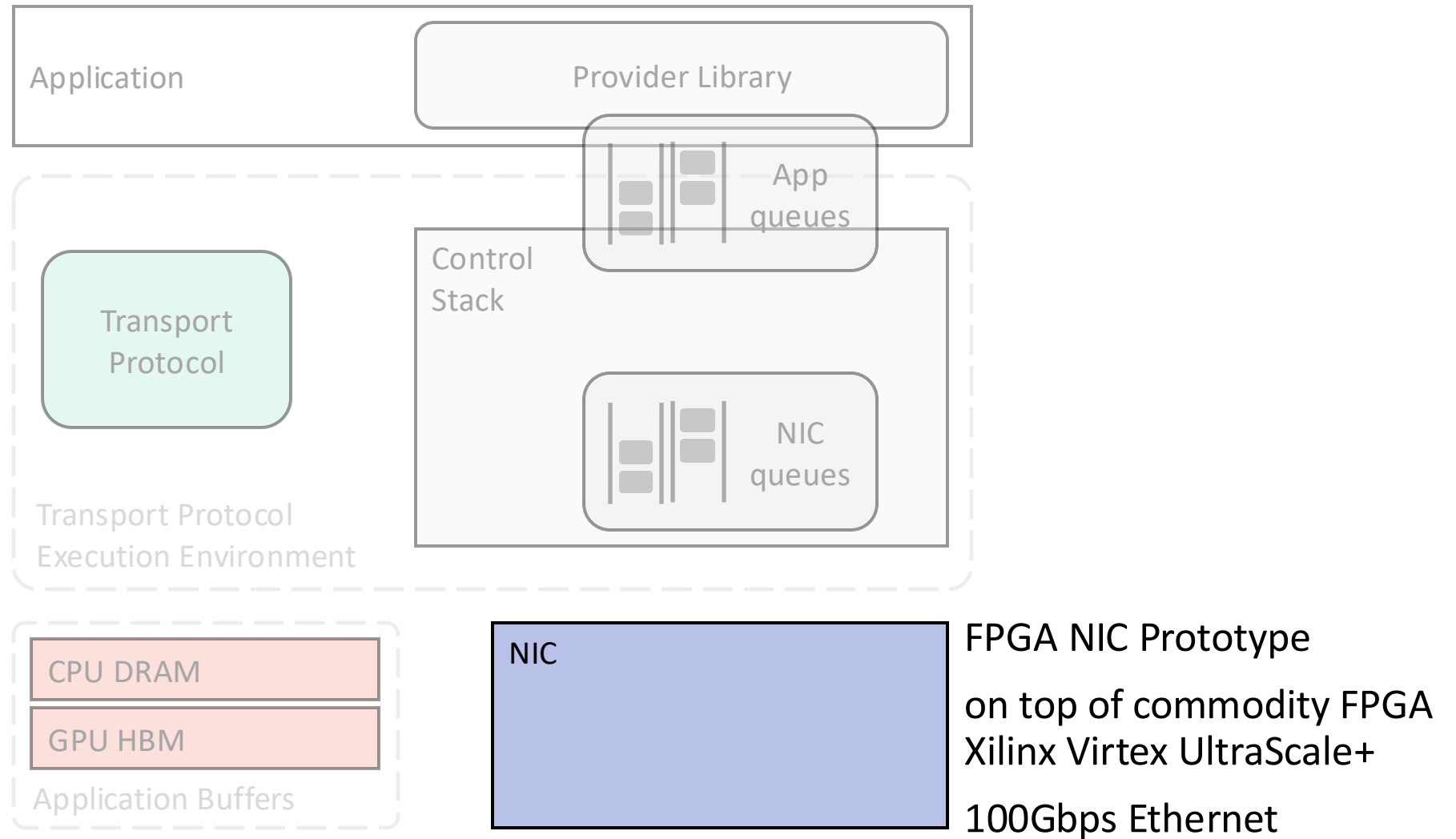
ZeroNIC end-to-end prototype



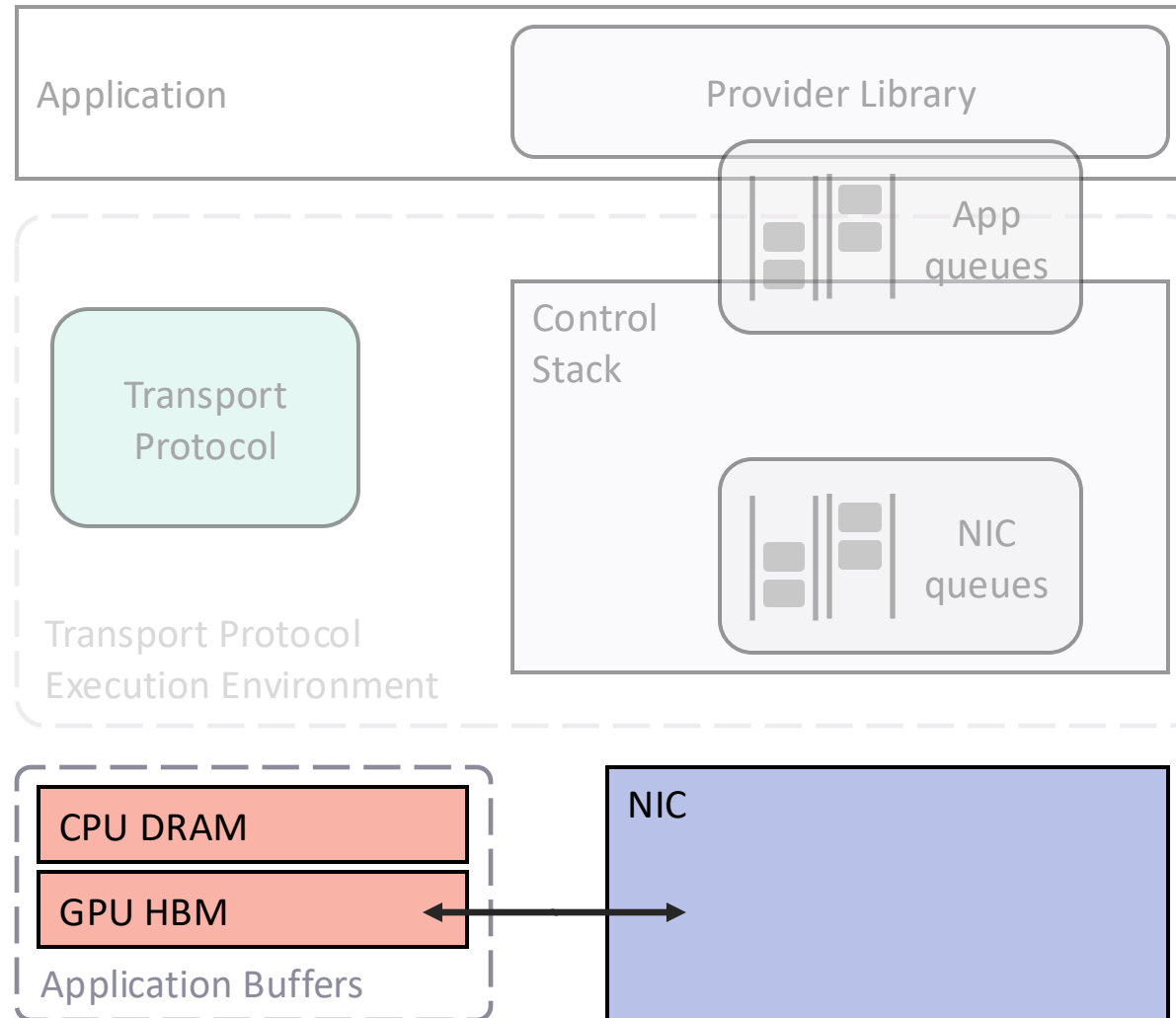
ZeroNIC end-to-end prototype



ZeroNIC end-to-end prototype



ZeroNIC end-to-end prototype



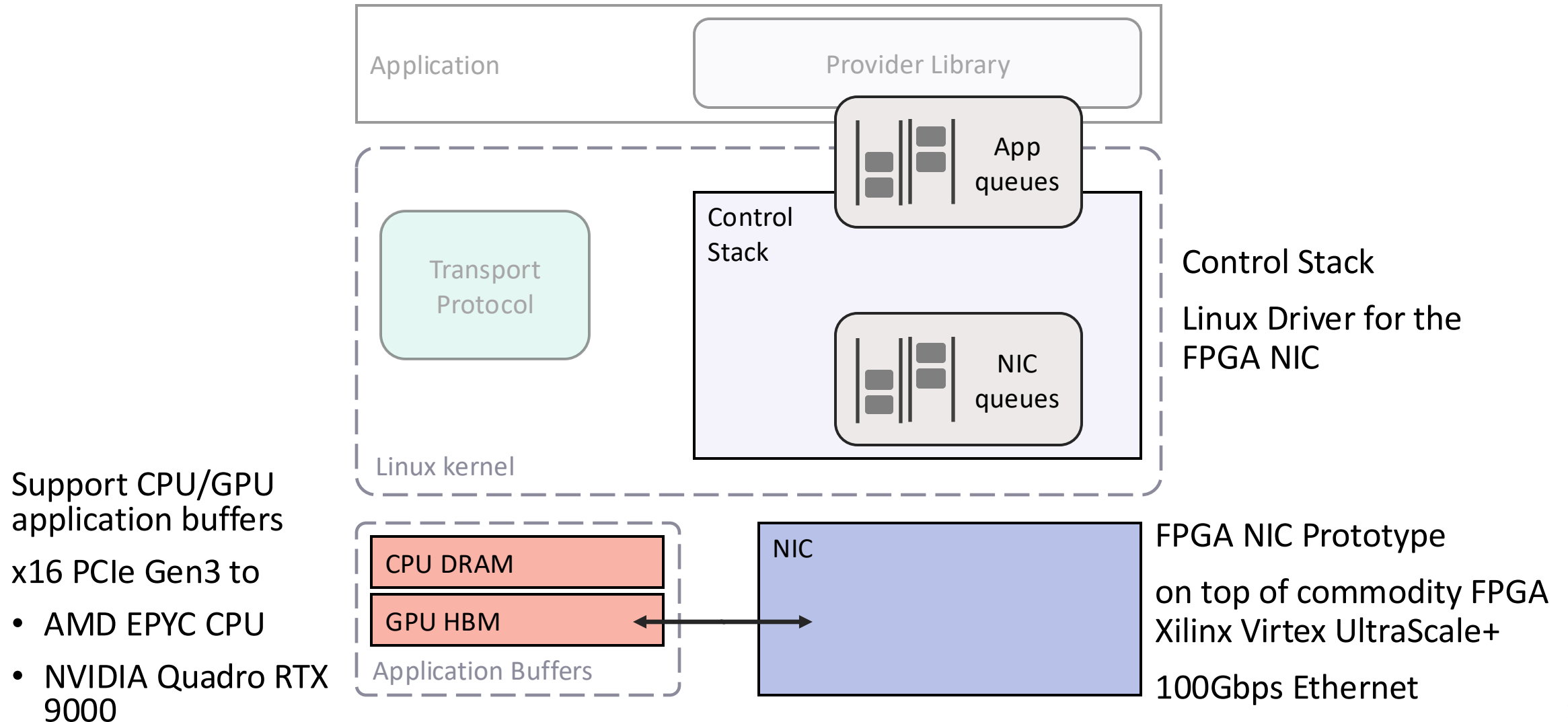
Support CPU/GPU application buffers

x16 PCIe Gen3 to

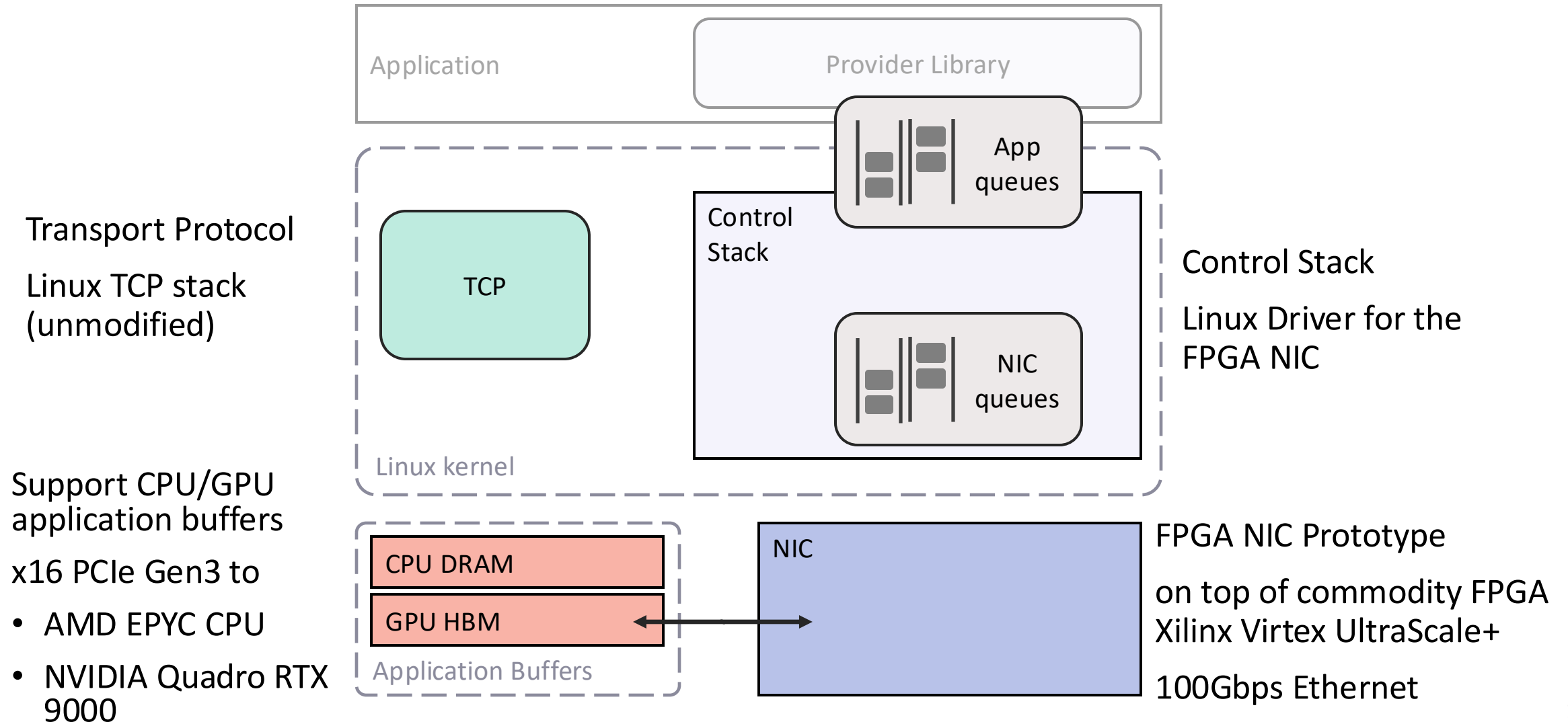
- AMD EPYC CPU
- NVIDIA Quadro RTX 9000

FPGA NIC Prototype
on top of commodity FPGA
Xilinx Virtex UltraScale+
100Gbps Ethernet

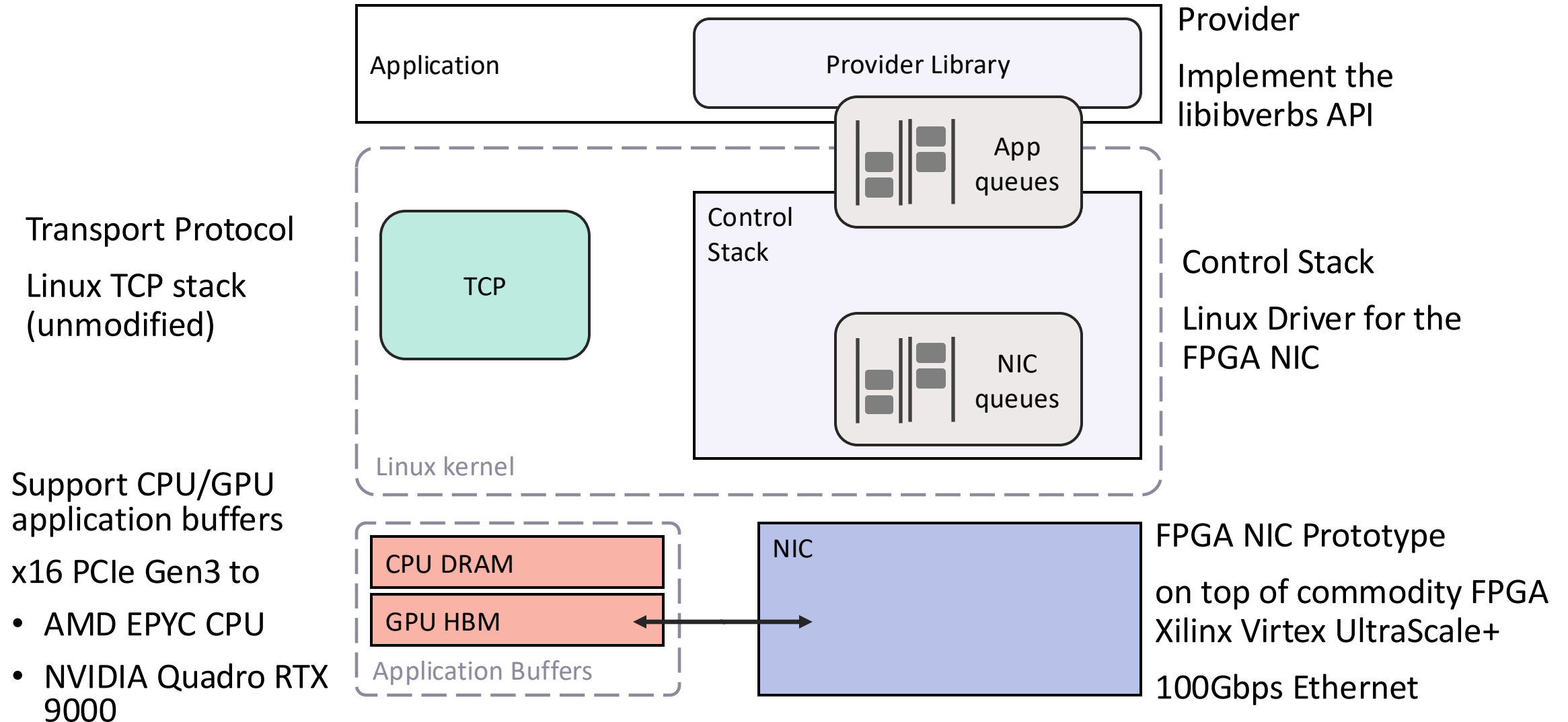
ZeroNIC end-to-end prototype



ZeroNIC end-to-end prototype

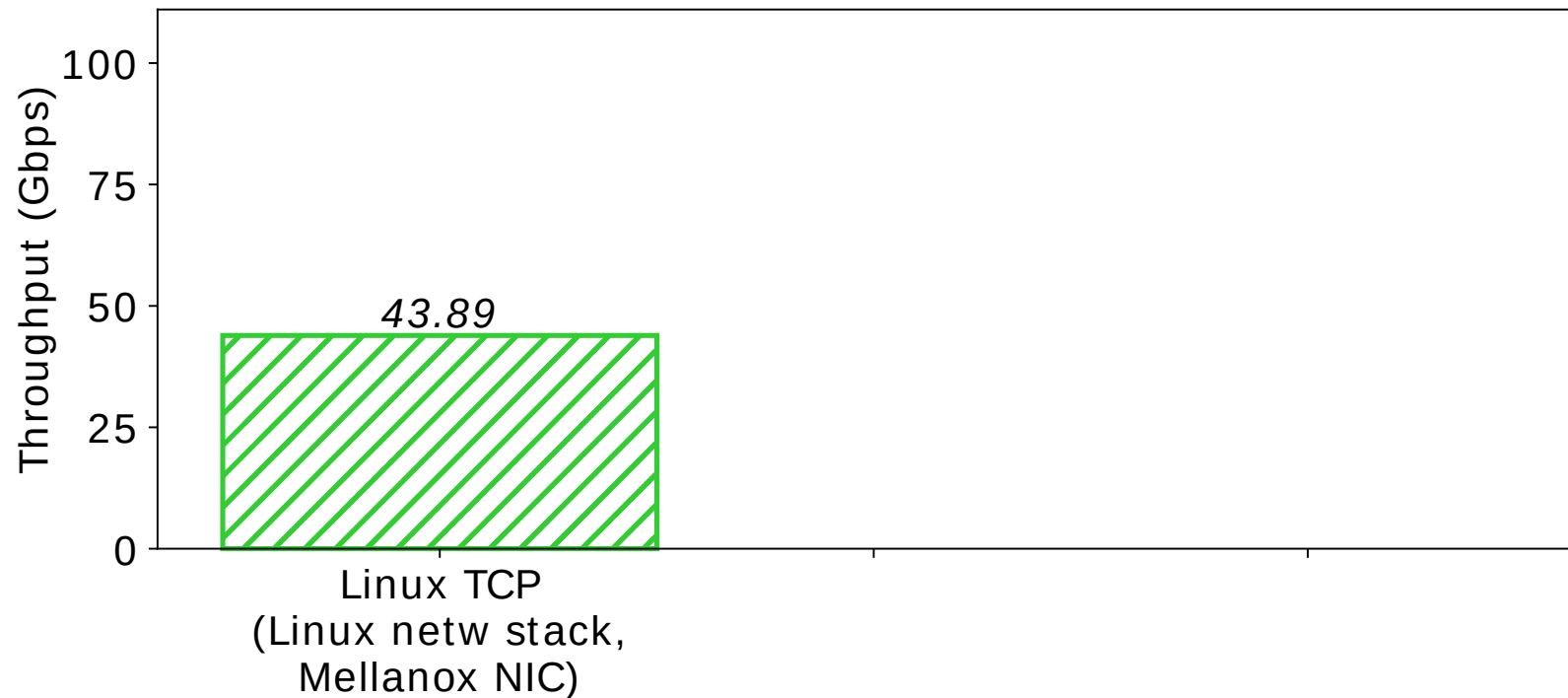


ZeroNIC end-to-end prototype



ZeroNIC: RDMA performance for TCP

single-flow microbenchmark (2 nodes)

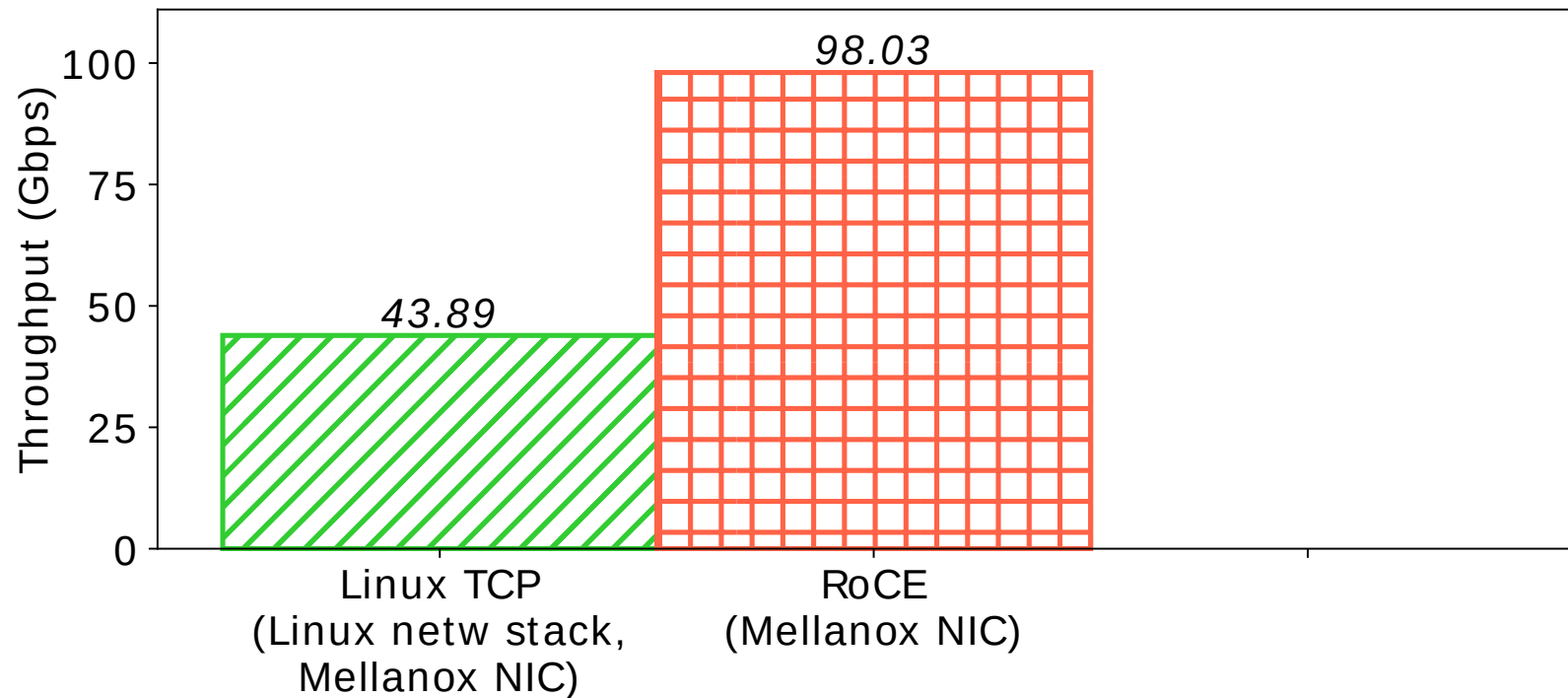


CPU utilization
(transport+driver)

Linux TCP 100%

ZeroNIC: RDMA performance for TCP

single-flow microbenchmark (2 nodes)



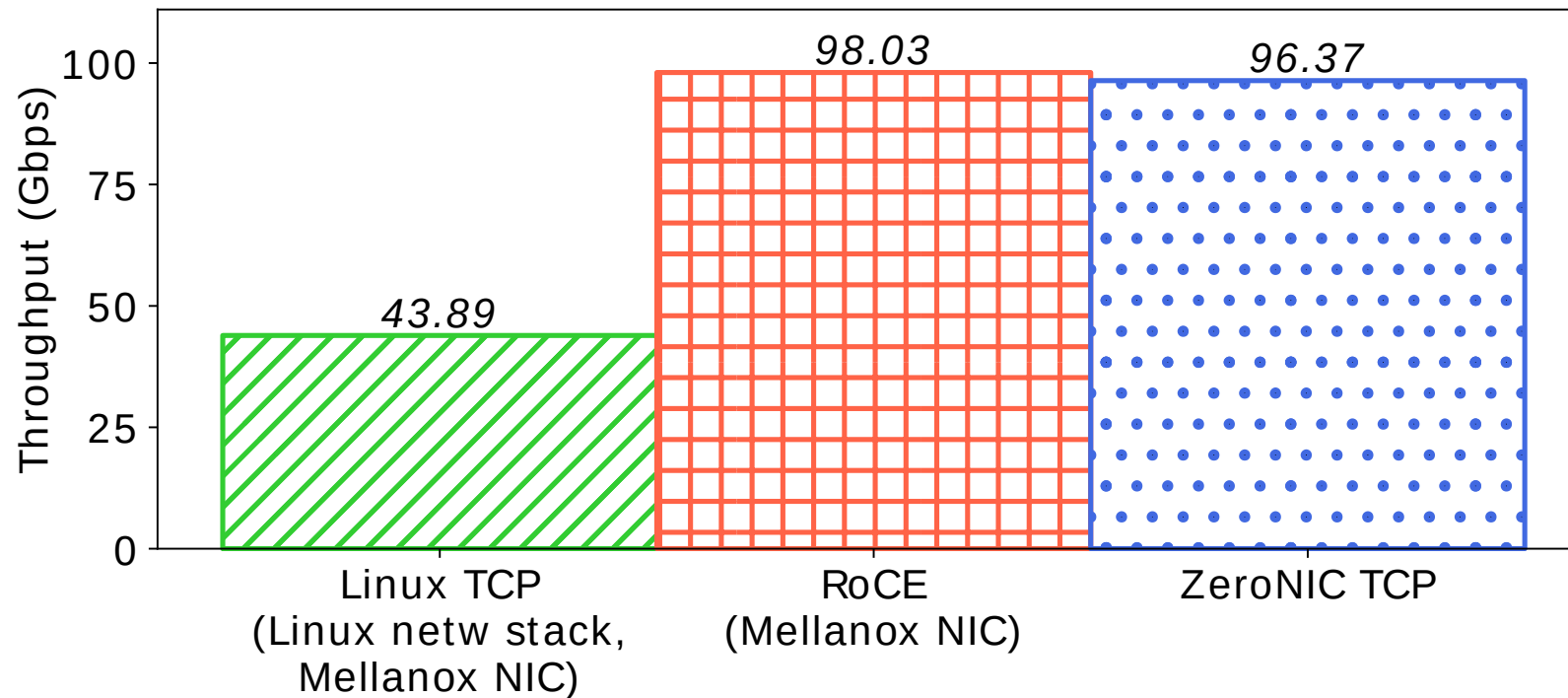
CPU utilization
(transport+driver)

Linux TCP 100%

MLX RoCE N/A

ZeroNIC: RDMA performance for TCP

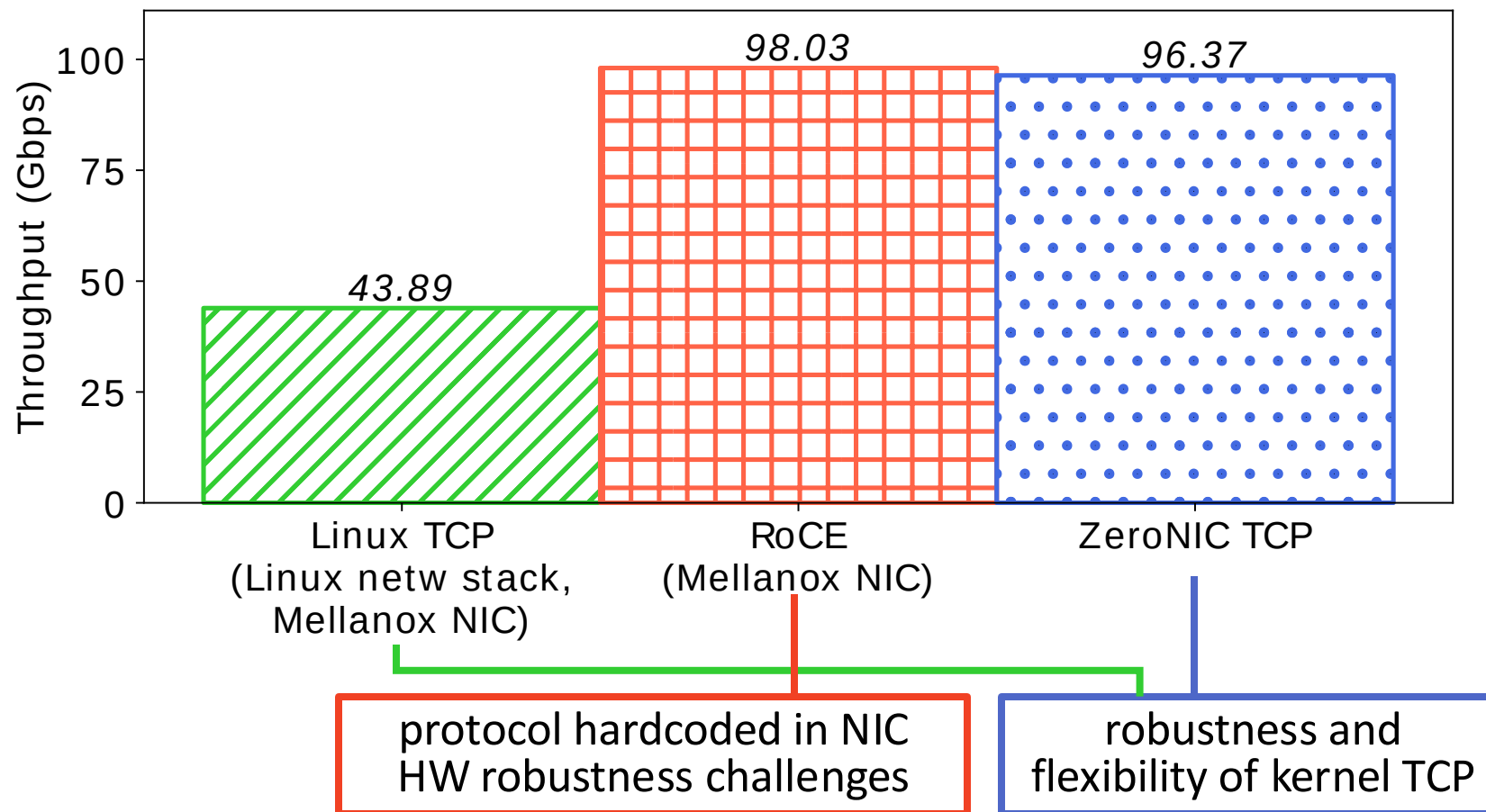
single-flow microbenchmark (2 nodes)



CPU utilization (transport+driver)	
Linux TCP	100%
MLX RoCE	N/A
ZeroNIC TCP	17%

ZeroNIC: RDMA performance for TCP

single-flow microbenchmark (2 nodes)



CPU utilization
(transport+driver)

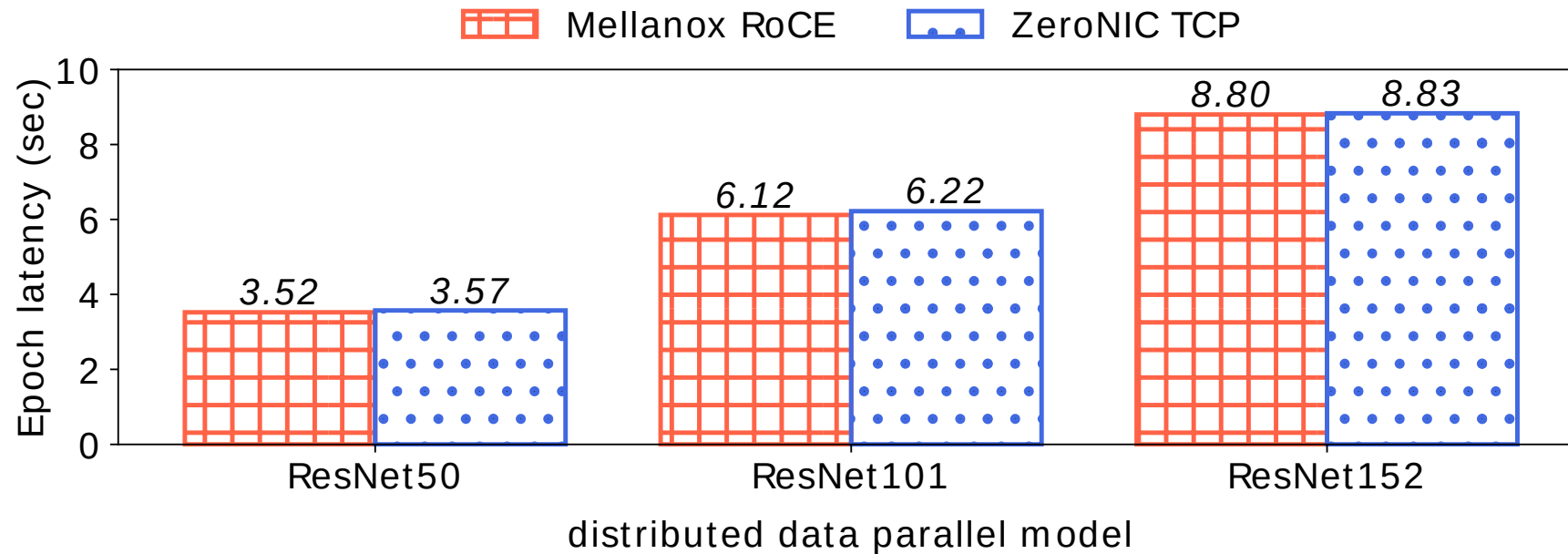
Linux TCP 100%

MLX RoCE N/A

ZeroNIC TCP 17%

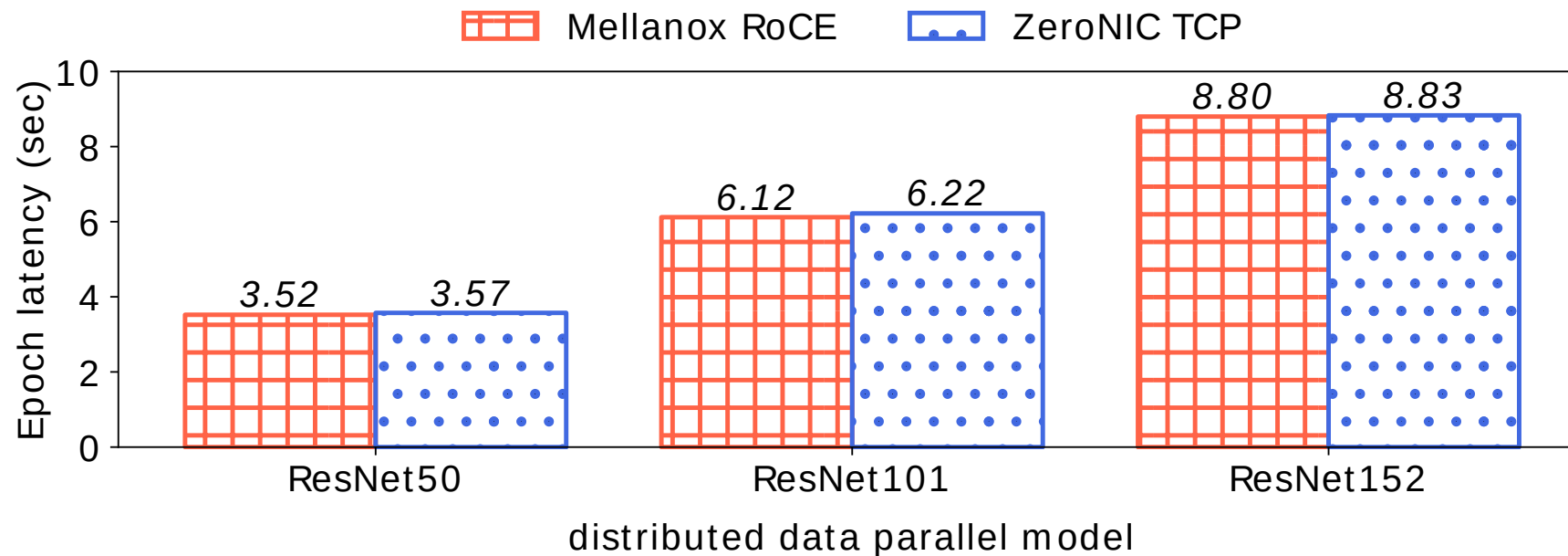
ZeroNIC: GPUDirect + TCP for ML Training

PyTorch end-to-end training benchmark (2 nodes)



ZeroNIC: GPUDirect + TCP for ML Training

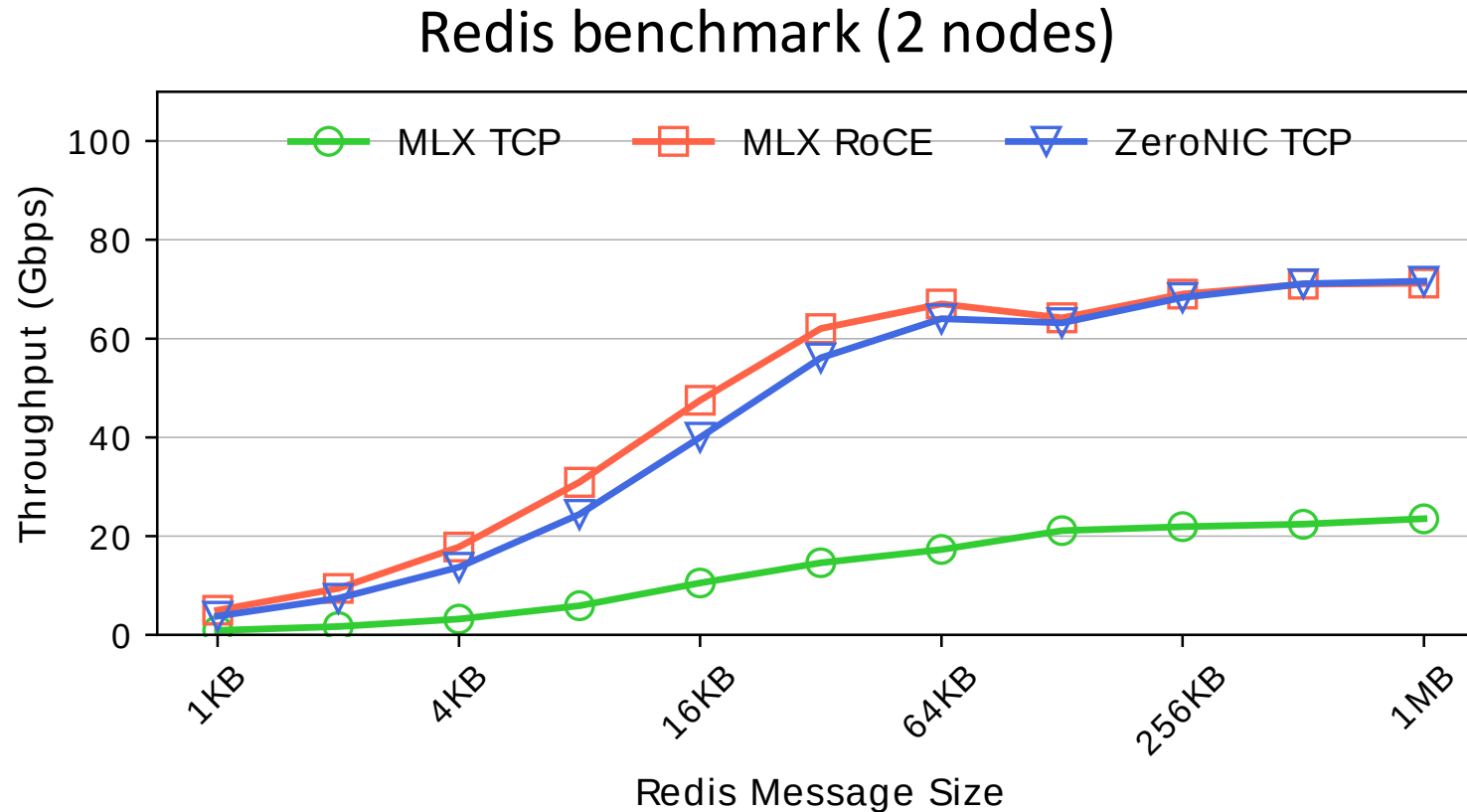
PyTorch end-to-end training benchmark (2 nodes)



protocol hardcoded in NIC
HW robustness challenges

robustness and
flexibility of kernel TCP

ZeroNIC transparently improves E2E apps



protocol hardcoded in NIC
HW robustness challenges

robustness and
flexibility of kernel TCP

ZeroNIC: **Performance** and **Flexibility** in Host Networking

